SEN381 - CIVICCONNECT PROJECT

Project Engineering Document (PED)

Milestone 4 – Finalisation, consolidation, and conclusion

Version 4.0

Team Name / Number: Proto

Team Members:

Jared Swanepoel - 601621

Michael Cheyne - 602441

Xander Oosthuyzen - 601256

Module: Software Engineering 381 (SEN381)

Governing document: SEN381 CivicConnect Master Project Brief

Submission Date: 18/10/2026

Table of Contents

Submission Information 7

Document Control 7

Change record 10

Baseline Approval 15

Baseline Approval (Milestone 2) 15

1. Problem Statement & Business Need 17

1.1 Problem Statement 17

1.2 Business Need 17

1.3 Intended Business Value 17

2. Stakeholder Analysis 17

3. Scope Baseline 18

3.1 In Scope 18

3.2 Out of Scope 19

3.3 Deferred / Future Scope 19

3.4 Defended Exclusion / Deferment 19

3.5 Scope baseline reviews and changes 19

4. Requirements & Acceptance Criteria 20

4.1 Functional Requirements 20

4.2 Non-Functional Requirements 22

4.3 Architecturally Significant Requirements and Quality Drivers 24

5. Constraints 26

5.1 Constraint Interaction / Trade-off 27

5.2 Assumptions and Dependencies 28

6.1 Matrix filled by Milestone 1 29

6.2 Matrix filled by Milestone 2 32

6.3 Matrix filled by Milestone 3 37

6.4 Matrix filled by Milestone 4 / final submission. 42

7. Risk Register 46
7.1 Index 46
Finalised risk register 60
8. Forward Engineering Considerations 62
9. Engineering Decision Log 65
9.1 Index 65
10. GitHub & Team Governance 85
10.1 Repository 85
10.2 Team Working Agreement 86
10.3 Repository structure and continuous integration 87
11. AI Usage Register 87
12. Baseline Readiness Checklist 91
Milestone 1 Readiness Checklist 91
Milestone 2 Readiness Checklist 91
Final readiness checklist 92
13. Solution Design Baseline 93
13.1 Architecture 93
13.1.1 Logical components 93
13.1.2 Considered alternatives 94
13.1.3 Justification of selected architecture 95
13.1.4 Physical deployment 95
13.1.5 Architectural diagrams 97
13.2 Data and persistence baseline 98
Intro 98
13.2.1 Core data entities 98
13.2.2 ERD 99
13.2.3 Data Ownership and Lifecycle 100
13.2.4 Persistence model choice 101
13.2.5 SPOF, scalability and bottlenecks 101

13.2.6 Milestone 3 Update: schema prototype and verification. 102
13.3 Tech Stack Analysis 102
13.3.1 Common web bundles 102
13.3.2 Candidates not recommended 103
13.3.3 Weight matrix 103
13.3.4 Weight matrix for database 104
13.3.5 Results of the matrices 105
13.3.6 Selected versions and compatibility 105
13.4 Design Decisions 106
13.4.1 Design Problem 1: Status-Change Side Effects 107
Problem 107
Assignment 2 Alternatives 107
Final Decision (adapts the Assignment 2 recommendation) 107
Trade-off 107
Class / Flow Diagram 107
13.4.2 Design Problem 2: Category-Specific Request Handling 109
Problem 109
Assignment 2 Alternatives 109
Final Decision 109
Trade-off 109
Class Diagram 109
13.5 UI & Information Architecture 111
13.5.1 Role Sitemap 111
13.5.2 Wireframes 111
13.5.3 Usability and Accessibility Rationale 112
13.6 Interfaces and Integration 114
13.6.1 Interface inventory 114
13.6.2 Page contracts 114
13.6.3 Failure behaviour 115

13.6.4 Internal event contract 116
13.6.5 External interface: mail provider 116
13.6.6 Change and versioning 117
13.6.7 Interactions not yet established 117
13.7 Deployment & Environments 117
13.7.1 Environment Structure 117
13.7.2 Configuration and Secrets 118
13.7.3 Queue worker and scheduler 118
13.7.4 Free-Tier Limits 118
13.7.5 Backup and Recovery 119
13.8 Open and Deferred Decisions 119
14. Milestone 3: construction, quality and release readiness 122
14.1 Overview 122
14.2 M2 baseline conformance 122
14.3 Controlled changes 123
14.4 Construction and configuration 123
14.5 CI and automated verification 124
14.6 Quality strategy applied 124
14.7 Black-box, integration and system validation 124
14.8 Performance and reliability 125
Result: every request was well under the 500 ms target, with no errors, so NFR-004 is met in this environment. This does not, however, include the latency added by the distance due to the NEON hosted database being hosted in London. 125
14.9 Defects and quality interpretation 126
Ten defects were recorded (Section 16) and all are fixed. Most came from environment and tooling (Windows server start-up, pnpm versions, the type-check, clashing test data) or from test timing, rather than from business rules, and CI or the test suites caught them before they reached Development. The most serious, DF-04, could have shown requests to the wrong staff member. RequestVisibilityTest found it and now guards against it. The main remaining concern is untested behaviour rather than known defects: PostgreSQL-only constraints and row locks (including the lock that stops two simultaneous offer declines) are not exercised by the SQLite test run, and the status-change, offer and category-handler code was merged last, so it has the least time in use beyond its automated tests. 126
14.10 Release readiness 126

15. Technical Debt Register....127
16. Defect Register....128
17. Final delivery and conclusion....129
17.1 Final product release baseline. 129
17.2 M1 scope baseline to final delivery 129
17.3 Scope Completion Percentage....130
17.4 Final deployment state and release decision 133
17.5 Closure of M3 issues 134
17.6 Known limitations....134
17.7 Project reflection and conclusion....135
References....136

Submission Information

Item	Information
Team	Proto: Jared Swanepoel (601621), Michael Cheyne (602441), Xander Oosthuyzen (601256)
Final release	v3.0-rc1: main, commit 288849fd91f39c35ecdd15fdeec31455884f32c6 (PR #25)
Repository	https://github.com/Destroyer1819/SEN381_Project
PED version	V4.0
Final RTM	PED Section 6.4 (final status), history in 6.1 to 6.3
CI evidence	GitHub Actions workflow "Tests", run #21 on the final commit: https://github.com/Destroyer1819/SEN381_Project/actions/runs/38919834328
Final product access	https://civicconnect-staging-kski.onrender.com (staging, final deployment state per Section 17.4, verified 18/10/2026)
Date submitted	18/10/2026

Document Control

Version	Date	Author(s)	Summary of Change	Reviewed By
0.1	07/09/2026	Xander	Initial Template	Michael, Jared
0.2	07/09/2026	Xander	Sections 5, 7, 8, 9, 10 filled out	Michael
0.3	09/09/2026	Michael	Sections 4, 6 added	
0.4	09/09/2026	Xander (AI-assisted)	AI Usage Register entries added for Sections 5/7/8/9/10 AI assistance. Risk Register owners assigned. Team Working Agreement drafted. FR-001 wording aligned between Section 4 and the RTM	Michael
0.5	09/09/2026	Jared	Sections 1, 2, 3 added to the main assignment	Michael

Version	Date	Author(s)	Summary of Change	Reviewed By
1.0	09/09/2026	Michael, Jared, Xander	Initial PED version 1.0 for Milestone 1 signed off	Michael, Jared, Xander
1.2	22/09/2026	Michael	Spelling corrections. Section 2: lecturer added as stakeholder. Section 3: expanded the WhatsApp exclusion, added the reasoning for excluding multi-language support and for the 5-year retention. Section 5: rationale now cited. Section 6: restored a missing functional requirement	Jared, Xander
1.3	23/09/2026	Michael	Section 4: added 4.3 Architecturally Significant Requirements. Section 5: added constraint IDs and CON-007 technology environment. Section 6: split the RTM into 6.1 (M1) and 6.2 (M2)	Jared, Xander
1.4	24/09/2026	Jared	Section 2: added the ASR influence column. Section 3: scope reviewed for v2.0, change record started	Michael, Xander
1.5	25/09/2026	Michael	Section 13.1 architecture and 13.2 data and persistence baseline drafted	Jared, Xander
1.6	25/09/2026	Xander	Sections 13.4 design decisions, 13.5 UI and information architecture, 13.7 deployment and environments drafted	Michael, Jared
1.7	26/09/2026	Jared	Section 13.3 technology stack analysis and weighted matrices added	Michael, Xander
1.8	26/09/2026	Xander	Section 7 risk register restructured per risk, R-008 to R-016 added. Section 8 Milestone 2 status added and FEC 7 and 8 raised. Section 9 decision log restructured per decision, D-006 to D-015 added. Section 10 governance updated	Michael, Jared
1.9	26/09/2026	Jared	Change record completed (CR-001 to CR-016). Section 11 AI Usage Register updated for Milestone 2. Section 12 Milestone 2 readiness checklist added. References extended	Michael, Xander
1.9.1	27/09/2026	Jared	Team member names corrected, bundle ID B-002 corrected, checklist symbols removed, Section 10.3 repository structure added, NFR-009 and ASR-004 review wording changed, D-001 category list changed, 13.2.1 requirement links updated	Michael, Xander
1.10	27/09/2026	Jared	Consistency review applied: CR-017 recorded and SPA wording replaced in 13.1, 13.5 and 13.7, CR-009, CR-012 and CR-013 applied, CR-018 and CR-019 raised, NFR-009, ASR-004 and D-001 restored to the baseline wording, 13.2.4 completed, 5.2 assumptions and 13.3.6 versions added, references corrected	Michael, Xander

Version	Date	Author(s)	Summary of Change	Reviewed By
1.11	27/09/2026	Jared	Consistency review applied (part 2): 13.5 and 13.6 updated for Inertia. All six figures redrawn (Figures 13.1 to 13.6). Section 13 heading restored. References corrected in Source Manager	Michael, Xander
2.0	30/09/2026	Jared	Grammar fixes such as section 13.7.2 was missing a full stop. Change document name to SEN381 Proto PED Milestone 2 V2.0	Michael, Xander
2.2	12/10/2026	Michael	Added sections 6.3 and 13.2.6 which is filled by Milestone 3's work. Updated the AI register. Updated 13.7.4 and 13.7.5. recorded the database provider for OD-01.	Jared, Xander
2.3	15/10/2026	Michael	Added the decisions of the Emailing service, database hosting site and application hosting site. Updated all section impacted by this.	Jared, Xander
3	16/10/2026	Michael, Jared, Xander	Milestone 3 version. Added the submission information block, Section 14 (construction, quality and release readiness), Section 15 Technical Debt Register and Section 16 Defect Register. Added the Milestone 3 status column to 13.8 and the M3 rows to RTM 6.3. Added D-020 (offer and decline rule) and the Render deployment evidence to D-018. Corrected the D-019 heading. Document renamed to SEN381 Proto PED Milestone 3 V3.0.	Jared, Xander
4	18/10/2026	Michael, Jared, Xander	Final version. Added Section 6.4 (final RTM status) and Section 17 (final release, M1 scope to delivery, Scope Completion Percentage, deployment decision, M3 closure, limitations, reflection). Added D-021, D-022, CR-032 to CR-038, TD-08 and TD-09 and the final readiness checklist. Updated the risk, decision and open-decision statuses. Filled the M3 placeholders. Document renamed to SEN381 Proto PED Final V4.0. Grammar fixes.	Michael, Jared, Xander

Change record

CR	Date	Raised by	Item	Change	Reason	Impact	Decision
CR-001	22/09/2026	Michael	FR-010	List the access levels explicitly	Requirement was not testable while the levels were undecided	Requirements, RTM	Accepted
CR-002	22/09/2026	Michael	FR-011	Elaborate on which access levels may see what	Same reason as CR-001	Requirements, RTM	Accepted
CR-003	23/09/2026	Michael	Section 6 RTM	Split the matrix into 6.1 (M1) and 6.2 (M2)	Readability, and to show M2 evidence separately from the M1 baseline	Traceability only	Accepted
CR-004	23/09/2026	Michael	Section 5	Added constraint IDs and CON-007 technology environment	Constraints could not be referenced. Master Brief section 25 makes the team responsible for checking technology availability on the BC Desktop	Constraints, ASRs, D-007	Accepted
CR-005	23/09/2026	Jared	Section 2	Added the ASR influence column	Links stakeholders to the drivers they cause, as Milestone 2 section 5.2 requires	Stakeholder analysis, section 4.3	Accepted
CR-006	23/09/2026	Michael	Section 4	Added 4.3 Architecturally Significant Requirements and Quality Drivers	Milestone 2 requires ASRs to be identified and linked to decisions	Requirements, architecture, RTM	Accepted
CR-007	26/09/2026	Xander	Section 7	Restructured the register one risk per table and raised R-008 to R-016	Register must be reviewed at every milestone. New risks arise from the M2 decisions	Risk, deployment, testing	Accepted
CR-008	26/09/2026	Xander	Section 9	Restructured the log one decision per table, closed D-004 and D-005, added D-006 to D-015	Decisions made in M2 had no records, and the log was unreadable as a single table	Architecture, data, technology, design, deployment	Accepted
CR-009	26/09/2026	Jared	FR-010, FR-011	Access levels reduced to three: Requester, Staff, Management. Administrative functions are handled by Management for the	The data model, policies and UI in Section 13 are built on three roles,	Requirements, data model (users.role), D-012, RTM, UI	Accepted

CR	Date	Raised by	Item	Change	Reason	Impact	Decision
				MVP. A separate Admin role moves to deferred scope	and a fourth undefined role cannot be tested		
CR-010	26/09/2026	Xander	FR-017, Section 8 FEC 2	Adopt the status set open, assigned, in_progress, resolved, closed, with transitions seeded in a lookup table	FR-017 requires pre-defined options, and D-008 cannot enforce transitions without a fixed set	Requirements, data model, UI, D-015	Accepted
CR-011	26/09/2026	Jared	Section 3.1, FR-004, FR-007	Confirm that email notification of a status change is in scope. Push notifications remain deferred	Section 3.4 already names email as the affordable replacement for WhatsApp, and D-009 and D-011 depend on it	Scope, design, deployment (queue worker), R-009	Accepted
CR-012	26/09/2026	Michael	FR-008, FR-016, FR-020	One retention wording: records are purged 5 years after the last update, not 5 years after creation	Section 4 and the RTM contradicted each other. The full history must be kept until the whole record expires.	Requirements, data model, retention job	Accepted
CR-013	26/09/2026	Michael	NFR-001	Reword to bcrypt password hashing with a cost factor of 12	“12 rounds of encryption” is not testable and confuses hashing with encryption	Requirement wording, M3 test evidence	Accepted
CR-014	26/09/2026	Xander	Section 8	Added Milestone 2 status and resolution columns, and raised FEC 7 and FEC 8	Forward considerations must show progress, and deferring construction created a new one	Forward considerations, risk	Accepted
CR-015	26/09/2026	Jared	Section 12	Added the Milestone 2 readiness checklist beside the Milestone 1 list	The M1 checklist does not cover the M2 baseline	Checklist only	Accepted
CR-016	26/09/2026	Michael	Section 6.2	Extended the M2 matrix with implementation evidence, verification evidence and status columns	Milestone 2 section 4.2 requires these columns, with a controlled status where evidence does not yet exist	Traceability, M3 planning	Accepted
CR-017	26/09/2026	Jared	D-006, D-007, Sections 13.1, 13.5, 13.6,	Presentation layer changed from a separate React SPA over a Laravel JSON API to React and	The section 13.3.3 matrix ranked B-009 (Laravel + Inertia) highest. One deployable with server-side session authentication removes CORS and	Architecture, deployment topology, authentication (D-012), interfaces, UI structure,	Accepted

CR	Date	Raised by	Item	Change	Reason	Impact	Decision
			13.7, R-010, R-016	TypeScript rendered through Inertia.	token handling, and no requirement needs an independent client or independent scaling (Assignment 2, Task 3).	R-010 reduced, R-016 raised.	
CR-018	27/09/2026	Jared	FR-005, FR-013, FR-021, FR-022, NFR-007, NFR-008	Wording changed so each requirement is testable and matches the design.	FR-005's title and criteria disagreed. FR-013 said filters remove relevant entries. FR-021 described deletion although resolution is a status change. FR-022 was not testable. The 95% targets were not supported by the cited source.	Requirements, RTM 6.1, M3 test design.	Accepted
CR-019	27/09/2026	Jared	Section 4.1 note, ASR-005, CR-012 reason, references	Basis for the 5-year retention changed from RICA to a stakeholder decision balanced against POPIA section 14.	RICA section 30 applies to telecommunication service providers, which CivicConnect is not.	Requirement wording, ASR-005, reference list.	Accepted
CR-020	10/10/2026	Michael	Section 6.2	Changed an error in that all functions should be within three clicks of the main screen, not two.	A mistyping of the NFR, as it contradicted the NFR in section 4.2.	Potential miscommunication has been removed	Accepted
CR-021	12/10/2026	Michael	Section 13.2.2	Updated the ERD to reflect the actual tables and entities.	Laravel makes use of standard tables and entity names which was not accounted for in the initial ERD.	Can influence the SQL statements of the application.	Accepted
CR-022	12/10/2026	Michael	Sections 6.3 and 13.2.6	Added new sections to contain the work from milestone 3	Test data became available and decisions were made.	Additional evidence.	Accepted
CR-023	13/10/2026	Michael	Section 13.2.1	Included template tables added by Laravel.	Laravel added additional template tables not considered.	Additional information	Accepted
CR-024	13/10/2026	Michael	Sections 9, 13.7.4, 13.7.5, 13.3.6	D-016 added: NEON selected as database host,	D-013 deferred the provider for hosting the Application	Closes the database section of OD-01	Accepted

CR	Date	Raised by	Item	Change	Reason	Impact	Decision
CR-025	13/10/2026	Michael	Section 13.7.1, AS-002, R-013	Development database changed from Docker Compose to a locally installed PostgreSQL 18	A locally installed PostgreSQL will match the NEON database's version directly	Development/staging version parity is kept	Accepted
CR-026	13/10/2026	Michael	R-002, R-0 Section 9 13	Milestone 3 reviews added of the NEON limits and the verification of the staging.	Database provider has been chosen and tested	Causes the risk to reflect the real limits	Accepted
CR-027	13/10/2026	Michael	Sections 9 13.7 intro and 13.7.1	D-017 added, deferring the hosting of the application to milestone 4.	Milestone 3 allows for a simulated Staging of the application and database.	Application host part of OD-01 open until M4	Accepted
CR-028	13/10/2026	Michael	Section 6.1	Acceptance criteria corrected to match Section 4.1	FR-003 repeated FR-004's criterion	Traceability only	Accepted
CR-029	15/10/2026	Michael	Sections 9, 5.2, 7, 13.7, 13.8	D-019 added.	Resend selected as mail provider, closing OD-02.	Clarity in scope, design and deployment	Accepted
CR-030	15/10/2026	Michael	Sections 9, 13.6.5, 13.7.2, 13.8	Sections 13.6.5, 13.7.2 updated for Resend over HTTPS	New information became available.	Clarity in scope, design and deployment	Accepted
CR-031	16/10/2026	Michael	Submission Block. Sections 14-16	Added the submission block and sections 14-16	New information became available.	Traceability	Accepted
CR-032	16/10/2026	Jared	FR-015	Acceptance criteria extended: a staff member may decline 3 offers per calendar month. After that the next offer is accepted automatically. A declined offer passes to the staff member with the fewest open requests, or back to the open pool.	FR-015 did not say what happens after a decline (D-020)	Requirements, RTM, OfferService	Accepted

CR	Date	Raised by	Item	Change	Reason	Impact	Decision
CR-033	18/10/2026	Michael	Section 6.4	Final RTM status table added. 6.3 kept as M3 history	Final brief requires final status per requirement	Traceability only	Accepted
CR-034	18/10/2026	Michael	Section 17	Final delivery and conclusion section added	Final brief: scope conclusion, deployment decision, closure, reflection	Documentation	Accepted
CR-035	18/10/2026	Michael	Section 15	TD-08 and TD-09 added	Found while closing M3 issues	Technical debt register	Accepted
CR-036	18/10/2026	Michael	OD-10	D-022 records role assignment through the database only	Closes OD-10	Decision log, 13.8	Accepted
CR-037	18/10/2026	Michael	D-017, D-018, 13.7	Application host changed from deferred to M4 (D-017) to Render free tier (D-018). Recorded at M4, decided 15/10/2026.	M3 was postponed while the final deadline stayed fixed, so hosting moved into M3	Deployment, R-009, TD-05, TD-08	Accepted
CR-038	18/10/2026	Michael	D-008	Take uses a single conditional update instead of the version check (CC-3). Recorded at M4.	Two simultaneous Takes could both pass a read-then-write version check	Design, R-008	Accepted

Baseline Approval

Project	CivicConnect
Baseline Type	Engineering Baseline (PED)
Version	1.0
Date	09/09/2026
Scope reviewed (Y/N)	Y
Requirements/traceability checked (Y/N)	Y
Repository/governance controls checked (Y/N)	Y
Outcome (ACCEPTED / CONDITIONALLY ACCEPTED / REVISION REQUIRED)	Conditionally accepted

Baseline Approval (Milestone 2)

Project	CivicConnect
Baseline Type	Engineering Baseline (PED)
Version	2.0
Date	30/09/2026
Scope reviewed (Y/N)	N/A

Requirements/traceability checked (Y/N)	N/A
Repository/governance controls checked (Y/N)	N/A
Outcome (ACCEPTED / CONDITIONALLY ACCEPTED / REVISION REQUIRED)	N/A

No baseline approval outcome was recorded for Milestone 2 or Milestone 3. PED v4.0 is submitted as the final baseline for assessment

1. Problem Statement & Business Need

1.1 Problem Statement

A community-focused organisation CivicConnect currently manages their services requests such as facility faults, IT support, maintenance, lost property and security concerns across email, phone, WhatsApp, spreadsheets, and paper. This produces duplicated or lost requests, no visibility for requesters, no reliable reporting for management and no clear ownership for staff.

1.2 Business Need

CivicConnect needs a controlled digital platform that provides a reliable, traceable and usable way to submit, manage, monitor and report on service requests.

1.3 Intended Business Value

CivicConnect business value is a single controlled record of a request's lifecycle. Submit to categorise to assign to act to resolve then report which closes the accountability gap that their current infrastructure has.

2. Stakeholder Analysis

Stakeholder	Role / Description	Needs / Expectations	Influence	Interest	Potential Conflicts	ASR influence
Requester	Community member submitting service requests	Visibility of request, Progress of request, Easy interface, Feedback	Low	High	Wants the request to be handled fast and wants full visibility	ASR-003, as the requester is harmed when requests are lost.
Staff	Actions and resolves requests	Clear ownership, interaction with requests	Medium	High	Admin overhead. Wants fewer notifications Could cause problems with requesters full visibility	ASR-001 and ASR-002, as the query response time and authorisation affect the staff.
Management	Oversight and reporting	Accountability, Reports, Overdue tracking	High	High	Wants detailed reporting which could conflict with staff's "less admin" preference	ASR-001 and ASR-002, as management requires access to all records. The Overdue tracking is also demanding on performance

Stakeholder	Role / Description	Needs / Expectations	Influence	Interest	Potential Conflicts	ASR influence
System owner	Scoping the system	Scope and setting boundaries, Defendable system choices	High	High	Wants breadth of features while team has schedule and cost constraints	ASR-004, ASR-005, and ASR-006. The owner set the cost constraint, and is thus responsible for ASR-006, but retention conflicts with ASR-005 and a three-person team limits ASR-004
Security team	Data handling	How the data is handled, Where the data is stored, How the data is stored,	Medium	High	May impact speed to make sure data is completely secured	ASR-002 and ASR-005, as the data requires authorisation, and raises a cross-cutting concern.
IT team	Service being requested	Alerted of request, Interaction with request,	Medium	Medium	Wants to be thorough with the request and requester wants to have it handled fast	ASR-002, as the IT support is a potentially sensitive category. Routing and restricting a request is thus governed by the IT team.
Lecturer	Oversees the life cycle of the application's creation	As the client substitute, expects a working solution that satisfies the business need. As Assessor, expects traceable evidence to be continuously produced as the project progresses.	High	High	As the substitute for the client, they are the only source of clarification. However, information can be withheld to test the skills, understanding and judgement of the developers. As assessor they need access to the evidence. Time spent on evidence competes with time to create the application.	ASR-004. Maintainability is highly required, as the habits and continuously produced evidence for reviewability is highly required by the Assessor.

3. Scope Baseline

3.1 In Scope

- Requests must be detailed such as category, location, area and short description
- Category must be selectable
- Staff be able to view, search, filter and sort requests and assign, accept, add comments, and update status

- Management must have a dashboard to view requests by status, category, overdue and basic activity reporting
- Role based access
- email notification of a status change

3.2 Out of Scope

- Native mobile app
- Multi-language support
- WhatsApp, SMS and or telephone integration

3.3 Deferred / Future Scope

- Push notifications
- Power BI style reporting to view advance analytics
- Separate Administrator role.

3.4 Defended Exclusion / Deferment

WhatsApp integration is excluded because it depends on a paid third-party API, which conflicts with the cost constraint (CON-003) and adds a reliability dependency. Status notifications are sent outside the customer's 24-hour service window, so they must use utility template messages, which Meta charges per message (Meta, 2026). SMS and telephone integration are excluded for the same cost reason. Email notification of status changes, which is in scope (Section 3.1), meets the same need at no cost. Multi-language support is excluded because most translation services charge per word (Translated, 2025).

3.5 Scope baseline reviews and changes

The scope baseline was reviewed for PED v2.0. No capabilities were added or removed. One clarification was made under change control: CR-011 confirms that email notification of a status change is in scope, which Section 3.4 already anticipated when it named email as the affordable replacement for WhatsApp notification. Push notifications and Power BI style analytics remain in deferred scope. CR-009 reduced the access levels to three roles and moved a separate Administrator role to deferred scope (Section 3.3). Administrative functions are carried by Management for the MVP.

4. Requirements & Acceptance Criteria

4.1 Functional Requirements

The 5-year retention follows the retention periods common in South African law, such as RICA's 3 to 5 years for telecommunication providers (VerifyNow, 2025). As CivicConnect is not a telecommunication provider, the period is a stakeholder decision (CR-019), balanced against POPIA section 14, which says personal information should not be kept longer than necessary (POPIA, 2019).

ID	Requirement	Source / Stakeholder	Priority	Acceptance Criteria
FR-001	Submission: being able to accept a new submission	Requester	High	The form used to submit must be saved into a database.
FR-002	Submission: return of requested data	Requester	High	The requested data must be able to reach the requester.
FR-003	Submission: Validation	Requester	Medium	Form submission must not be allowed if the form does not adhere to submission rules
FR-004	Submission: feedback	Requester	Medium	Upon submission attempt, the user must receive feedback. If it's an error, the problem must be stated. If successful, the user must be notified
FR-005	Sort requests	Requester	Low	A requester can sort their requests by date submitted, date last changed or alphabetical order (changed by CR-018)
FR-006	View Status: Accuracy	Requester	Medium	The status displayed must display the correct status.
FR-007	View Status: Delay	Requester	Medium	From the moment the status is updated on the database, the viewer must be able to see the update if they were to refresh on their end
FR-008	View History: Time data is stored	Requester	Low	A request and its related history, assignment and comment records are retained for 5 years after the last update, then purged automatically (changed by CR-012, v2.0)

ID	Requirement	Source / Stakeholder	Priority	Acceptance Criteria
FR-009	View History: Amount	Requester	Medium	The Requester will see the last 50 requests per page. The Requester can go to other pages to see older requests.
FR-010	Authentication	All users (Staff, management and requesters)	High	Users are assigned exactly one of three roles: Requester, Staff or Management. A user may only reach data permitted to that role (changed by CR-009, v2.0)
FR-011	Authorised requests by staff	Staff	Medium	A Requester sees only their own requests. Staff see unassigned requests and requests assigned to them, not those assigned to other staff. Management sees all requests. Verified by a test per role and endpoint (changed by CR-009, v2.0)
FR-012	Views provide relevant information	Staff	Low	The views should provide enough information in order to make relevant decisions. Examples being: category, status, time submitted, who is assigned to the request.
FR-013	Search, filter and sort	Staff/Management	Low	Search returns every matching request the user is authorised to see. A filter removes non-matching requests. Sort orders results by the chosen column (alphabetical, last updated, etc.). No combination of search, filter and sort causes an error or shows unauthorised data (changed by CR-018)
FR-014	View full details	Staff	Medium	Tying in with the previous requirement, the view must be expandable to access more information on the request. A change history should also be accessible.
FR-015	Assign/Accept responsibility	Staff	High	Staff should be able to assign requests to themselves, offer requests to others, and accept or decline requests offered to them. A staff member may decline up to 3 offers per calendar month. After that the next offer is accepted automatically. A declined offer passes to the staff member with the fewest open requests, or returns to the open pool (changed by CR-032).
FR-016	Request accountability	Staff	Low	Every assignment, offer and acceptance is recorded with the staff member and timestamp and retained for 5 years after the request's last update, then purged automatically (changed by CR-012, v2.0).

ID	Requirement	Source / Stakeholder	Priority	Acceptance Criteria
FR-017	Request update validation	Staff	Low	A status may only change to one of the seeded values open, assigned, in progress, resolved or closed, and only along a transition listed in the transition table (changed by CR-010, v2.0)
FR-018	authorisation of request update	Staff	Medium	Only authorised staff should be allowed to make updates. For example, staff should not be allowed to change the status of a request not assigned to them.
FR-019	Controlled transitions	Staff	Low	Every status change records the staff member and timestamp. The history is retained for 5 years after the request's last update (changed by CR-012, v2.0).
FR-020	Add comments/resolution information	Staff	Medium	Staff can add comments and resolution notes to a request. They are retained for 5 years after the request's last update (changed by CR-012, v2.0).
FR-021	Resolve Requests	Staff	High	When a request has been fulfilled, the assigned staff member can move it to Resolved. It leaves their active list but stays in the system with its full history (changed by CR-018)
FR-022	View all information	Management	Medium	Management can view every request with its full status history, assignments and comments (changed by CR-018)
FR-023	Identify overdue	Management	Low	Requests will have to be resolved within a set amount of time. Early estimates sets it at within 7 days.
FR-024	Access information for performance analysis	Management	Low	Management should have access to the additional information given by the staff members in order to review the resolved requests. Basic analysis features, such as average time to resolution could also be added.

4.2 Non-Functional Requirements

The following NFR's are created making use of the ISO/IEC 25010 (ISO, 2023) standards for software product quality as reference.

ID	Requirement	Category	Source	Priority	Acceptance Criteria
NFR-001	Credential Storage	Security	Master Brief	High	Passwords are stored only as bcrypt hashes with a cost factor of 12. Plain-text passwords are never stored or logged (changed by CR-013)
NFR-002	Authorisation Enforced	Security	Master Brief, Requester	High	Users can only access data that they are allowed to access. For instance, a requester can't access the credentials of other requesters
NFR-003	Status Changes with user and timestamp	Security	Master Brief, Management	Medium	If a status change is made, an accurate timestamp must be made and the last user to change the status must be recorded.
NFR-004	Result return time	Performance	Staff	Medium	For each 100 entries returned, there will be a maximum of 1 second wait time regardless of filters applied (Nielsen, 1993). This does not include Internet lag as we are not responsible for the internet connection.
NFR-005	Availability	Reliability	Management	Medium	The application is available 99% of the time during operating hours, defined as 08:00-18:00, Monday to Friday (Tovarys, 2022).
NFR-006	Data recovery	Reliability	Master Brief	Low	In case of data loss or overwriting, data must be able to be recovered/ rolled back within 48 hours
NFR-007	Ease of use	Interaction capability	Requester	Low	At least 80% of test users submit a request without help on their first attempt (the average task-completion rate across studies is 78%). All functions are at most 3 clicks from the main screen (changed by CR-018) (Jeff, 2011).
NFR-008	Validation failures produce actionable messages.	Interaction capability	Requester	Low	In a usability walkthrough, every validation message names the field concerned and states how to correct it (changed by CR-018) (Jeff, 2011).
NFR-009	Code follows documented standards	Maintainability	Team	Medium	Every pull request merged to main has approvals from the two team members who did not author it, verified from the GitHub history. Code follows the standards in Section 10.2.

4.3 Architecturally Significant Requirements and Quality Drivers

ID	Requirement	Quality attribute	Significance	Measurable response	Decision reference
ASR-001	NFR-004, FR-013	Performance	The speed at which a response must be given is bound, with the "worst-case" query having a limit of 1 second response time per 100 records. This means that optimisation will be required, such as indexing. It also conflicts with the constraint of a free-tier service, as it caps connections and computational capacity.	For every 100 entries, regardless of filter combination, may not add more than 1 second to the search.	D-006 architecture, D-007 stack, D-013 deployment
ASR-002	FR-010, FR-011, FR-013, FR-017, FR-018	Security	Different roles will require different views on the same data. Searches and filters will also cause the security to be a dedicated layer instead of a set of guards on the code, meaning that it will highly affect the querying system.	No query made may return records outside the authorisation level of the specific user, regardless of the combination of filters applied.	D-002 audit history, D-012 authentication and RBAC, D-015 transitions
ASR-003	FR-001, FR-002, NFR-005, NFR-006	Reliability/ availability	A main issue CivicConnect is supposed to solve is requests being lost. Requests must be submitted, or a report must be given. Requests that has been stored must be recoverable when lost.	No submissions may be simply lost. Submission failures must be indicated, successful submissions must be acknowledged. Furthermore, overwritten data must be recoverable within 48 hours. 99% availability during operating hours is also required.	D-008 transactional write, D-011 notification integration, D-013 deployment
ASR-004	NFR-009, Forward Engineering Consideration (FEC)	Maintainability	A three-person team on a fixed schedule with fixed milestones will have trouble with any re-work that may need to be done, and the FEC notes that retrofitting will be difficult and expensive to do.	Core workflow logic stays in service classes, and every change to main is approved by the two non-author team members before merging.	D-006 architecture, D-009 side-effect split, D-010 category strategy

ID	Requirement	Quality attribute	Significance	Measurable response	Decision reference
ASR-005	FR-008, FR-016, FR-019, FR-020	scalability	The 5-year period follows the retention periods common in South African law (RICA requires 3 to 5 years, but only for telecommunication providers), and was adopted as a stakeholder decision (CR-019). POPIA section 14 allows records to be kept only as long as that purpose requires, so they must be deleted when the period ends.	Data cannot be lost or overwritten before the minimum 5 years due to limited storage.	D-008 persistence, D-013 deployment, CR-012 retention
ASR-006	FR-008, FR-016, FR-019, FR-020, NFR-006	Cost	CivicConnect needs to make use of free-tier software and services. This will mean that the 5-year retention rules will need to be able to handle the accumulated data over the 5 years without deleting anything.	Estimation and calculation of a 5-year data volume, and comparing it to the capacity provided by the free-tier services. Automated purges after 5 years.	D-007 stack, D-013 deployment, R-002 and R-013 free-tier limits

5. Constraints

ID	Category	Constraint	Engineering Implication
CON-001	Scope	Exactly 3 registered students must deliver a controlled MVP covering requester, staff and management capabilities within the SEN381 timeline. Once approved, committed scope must be baselined and controlled.	Every additional feature beyond the baseline (e.g. SMS notifications, map-based tagging) creates obligations to specify, secure, test and maintain it. Non-essential capabilities must go to Section 3.3 (Deferred/Future Scope) rather than be absorbed silently, to protect schedule and quality.
CON-002	Schedule	The project must progress through four formal milestones (M1 - M4) within the SEN381 delivery period, each building on the previous approved baseline.	Requirements and early decisions made under this timeline must be evidence-based rather than rushed. Research must be timeboxed (as in Assignment 1) and baselined items may only be re-decided through a formal change record (Master Brief section 14), not informally.
CON-003	Cost / Resources	The team should prefer free or low-cost services where practical but must identify their limitations and the likely operational cost beyond the educational context.	Free-tier hosting/database services typically cap storage, connection counts, request quotas or uptime, for example, Supabase's free tier provides a 500 MB database and pauses projects after one week of inactivity (supabase, 2026).These limits must be documented during M2 technology research so the data model and expected request volume are sized realistically rather than assumed unlimited.
CON-004	Quality	Quality attributes (e.g. traceability of request status, data accuracy, usability for non-technical requesters) must be defined now and later supported by measurable test/acceptance evidence.	NFRs must be written in testable terms at M1 (Section 4.2) - e.g. a measurable response-time or data-integrity condition - rather than as vague aspirations, so M3 testing has something concrete to verify against.

ID	Category	Constraint	Engineering Implication
CON-005	Security	Security is a lifecycle-wide responsibility. CivicConnect handles categories such as security concerns and IT support requests that may contain sensitive personal or safety-related information.	Access control (RBAC) and an audit trail must be treated as foundational requirements at M1 (see Forward Engineering Consideration #1 and #6, Section 8), not features added just before deployment - deferring this risks a costly late redesign of the data model and authentication approach.
CON-006	Team Capability	Only three registered students must analyse, design, build, secure, test, deploy and individually defend the entire system (Master Brief section 8).	The team must weigh unfamiliar-but-powerful technology against realistic learning-curve risk within the schedule constraint above - this interaction is expanded in Section 5.1 and tracked as Risk R-001 (Section 7).
CON-007	Technology environment	Tools must run on the BC Desktop or team laptops so the lecturer can assess the project in the institutional environment. Master Brief section 25 states the institution cannot guarantee that a chosen technology is available, so the team is responsible for checking this (R-014)	This helps the lecturer to assess the project on their end and to see how each part integrates with each other (the master brief also lists this as a requirement section 25)

5.1 Constraint Interaction / Trade-off

Cost/Resources interacts directly with Security. The preference for free-tier hosting and database services (driven by the Cost constraint) is likely to mean weaker default network isolation, connection encryption or backup guarantees than a paid tier would provide. If the team selects a free-tier service purely because it is easiest to set up, without evaluating its security posture, the cost saving could increase security risk exposure for the sensitive request categories described in the Business Need (Section 1). This trade-off must therefore be evaluated explicitly in the M2 weighted technology decision matrix (Master Brief section 18.1) - security ecosystem is one of the required comparison criteria, rather than defaulting to whichever free service is fastest to configure under schedule pressure.

Schedule also interacts with Quality: because M1 - M4 are fixed and non-negotiable, a team that loses time to an unplanned issue (e.g. Risk R-001, technology learning curve) may be tempted to informally cut test coverage to protect the delivery date. The Master Project Brief (section 18) requires that any such trade-off be recorded and approved rather than silently absorbed, which is why Section 9 (Engineering Decision Log) and Section 7 (Risk Register) exist as controlled places to make that trade-off visible instead of hidden.

Milestone 2 outcome: the cost and security trade-off was resolved in D-007 and D-013 rather than left open. The selected stack is free and open source, and the deployment topology was deliberately reduced to one web process, one queue worker and one managed PostgreSQL instance so that the platform fits a free tier without weakening the security posture. Security ecosystem carried 12 of the 100 points in the section 13.3 weighted matrix, so the trade-off was scored rather than assumed. The remaining exposure is the hosting provider itself, which is tracked as R-002 and R-013 and closes with D-013

5.2 Assumptions and Dependencies

ID	Assumption / dependency	Relied on by	If it proves false	Risk / decision	Owner	Status
AS-001	The chosen free-tier host can run a persistent queue worker and Laravel's scheduler (or a cron entry).	D-011, D-013, FR-008	Notifications are queued but never sent. The retention purge never runs.	R-009, OD-01	Michael	Closed. Render does not provide this in the free-tier service. The worker and scheduler has been replaced.
AS-002	PHP 8.3+, Composer, Node.js and PostgreSQL can be installed and run on the BC Desktop or team laptops.	CON-007, D-007	The project cannot be built or assessed in the institutional environment.	R-014	Jared	Closed with contingency: development on own machines. Not verified on a BC Desktop.
AS-003	Five years of request data fits within the free-tier database storage cap.	ASR-005, ASR-006, FR-008	Storage fills before the retention period ends.	R-002, R-013	Michael	Closed. 310 Seeded requests use around 1.08MB, meaning roughly 250 000 requests will fit inside NEON's free tier service.
AS-004	A free email provider allows enough daily sends for status notifications.	D-011, FR-007	Notifications fail or need a paid plan.	OD-02, R-009	Jared	Closed by D-019. Resend's free tier allows 100 emails per day, enough for demonstration.
AS-005	Every requester has an email address and a web browser.	Scope 3.1-3.2, FR-007	Some requesters cannot receive notifications or use the platform.	R-007	Xander	Assumed

ID	Assumption / dependency	Relied on by	If it proves false	Risk / decision	Owner	Status
AS-006	Three roles (Requester, Staff, Management) are enough for the MVP.	CR-009, D-012	Department sub-roles would change the policy layer and data model.	OD-03, R-003	Jared	Assumed
AS-007	Laravel 13, Inertia and the React starter kit stay supported for the project's lifetime.	D-007, 13.3.6	A forced upgrade or migration during construction.	R-001	Xander	Assumed - Laravel 13 security fixes run to Q1 2028

6. Requirements Traceability Matrix (RTM)

6.1 Matrix filled by Milestone 1

Req ID	Stakeholder /Source	Requirement Summary	Acceptance Criteria	Priority
FR-001	Requester	Submission: being able to accept a new submission	The form used to submit must be saved into a database.	High
FR-002	Requester	Requested data is received	The requested data must be able to reach the requester from the database.	High
FR-003	Requester	Input validation	Form submission must not be allowed if the form does not adhere to submission rules	Medium
FR-004	Requester	Requester will receive feedback on submission	Upon submission attempt, the user must receive feedback. If it's an error, the problem must be stated. If successful, the user must be notified	Medium
FR-005	Requester	Requests can be categorised and sorted	On selection, requests must be categorise according to date submitted, date changed, or alphabetical order (Superseded by CR-018)	Low
FR-006	Requester	Accuracy of status	The status displayed must display the correct status.	Medium

Req ID	Stakeholder /Source	Requirement Summary	Acceptance Criteria	Priority
FR-007	Requester	The status, upon update, must be visible to the requester	From the moment the status is updated on the database, the viewer must be able to see the update if they were to refresh on their end	Medium
FR-008	Requester	The data must be stored for a set amount of time	The data must be stored for at least 5 years after the last update before it is automatically deleted (Superseded by CR-012)	Low
FR-009	Requester	The requester must be able to view more than one request at a time	The Requester will see the last 50 requests per page. The Requester can go to other pages to see older requests.	Medium
FR-010	All users of the system	User Authentication	Users can only access data that corresponds with their access levels/roles. These levels are still to be decided (Superseded by CR-009)	High
FR-011	Staff	Authorised requests by staff	When the rules/authority levels are set, test to see that all users can only see what they are authorised to see (Superseded by CR-009)	Medium
FR-012	Staff	Views provide limited but relevant information for quick overview	The views should provide enough information in order to make relevant decisions. Examples being: category, status, time submitted, who is assigned to the request.	Low
FR-013	Staff/ Management	Search, filter and sort features	Search shows all valid data entries, filter removes all relevant data entries, and sort correctly sorts the data according to choice (alphabetical, last updated, etc.). Combinations of sorting, filtering and searching should not cause errors or show unauthorised data. (Superseded by CR-018)	Low
FR-014	Staff	View full details on demand	Tying in with the previous requirement, the view must be expandable to access more information on the request. A change history should also be accessible.	Medium
FR-015	Staff	Assign/Accept responsibility of a request	Staff should be able to assign requests to themselves. They should also be able to offer requests to others, and accept requests offered to them.	High
FR-016	Staff	Accountability of handling request	Assignments should be recorded and stored in the database for review. These recordings should stay for only 5 years before deletion. (Superseded by CR-012)	Low

Req ID	Stakeholder /Source	Requirement Summary	Acceptance Criteria	Priority
FR-017	Staff	Update validation of Requests	Status updates can only be changed to pre-defined options. (Superseded by CR-010)	Low
FR-018	Staff / Management	authorisation of request update	Only authorised staff should be allowed to make updates. For example, staff should not be allowed to change the status of a request not assigned to them.	Medium
FR-019	Staff	Controlled transitions	If updates are made, the staff member and timestamp must be recorded, and stored for 5 years. (Superseded by CR-012)	Low
FR-020	Staff	Add comments/resolution information	Staff should be able to add additional information to requests, and this should also be saved for 5 years in the database. (Superseded by CR-012)	Medium
FR-021	Staff	Resolve Requests	When a request has been fulfilled, staff should be able to remove the request from their assigned requests (Superseded by CR-018)	High
FR-022	Management	View additional information	Management will likely need full access to all information in the database. (Superseded by CR-018)	Medium
FR-023	Management	Identify overdue requests	Requests will have to be resolved within a set amount of time. Early estimates sets it at within 7 days.	Low
FR-024	Management	Access information for performance analysis	Management should have access to the additional information given by the staff members in order to review the resolved requests. Basic analysis features, such as average time to resolution could also be added.	Low

6.2 Matrix filled by Milestone 2

Req ID	ASR	Module / component	Data and persistence impact	Design / interface decision	Technology and decision reference	Implementation evidence	Verification evidence	Status
FR-001	ASR-003	Submission	Inserts a row in service_requests. Commits before the response is returned	Controller validates, then delegates to the submission service	D-006, D-007, D-010	Planned M3: RequestSubmissionService	Planned M3: submission feature test	Approved, not yet implemented
FR-002	ASR-003	RequestQuery	Reads service_requests only	Reads go through the data-access layer, not ad hoc queries	D-006, D-007	Planned M3: requester list page	Planned M3: feature test	Approved, not yet implemented
FR-003	ASR-003	Submission	Constraints enforced by schema and validation rules	Validation contract in the form request, merged with category rules	D-007, D-010	Planned M3: StoreServiceRequest	Planned M3: validation test	Approved, not yet implemented
FR-004	ASR-003	Submission	None, no write of its own	Response always returns a confirmation or a specific field error	D-007, CR-011	Planned M3: submission response handling	Planned M3: feature test	Approved, not yet implemented
FR-005	ASR-001	RequestQuery, Category handling	Sorting applied to indexed columns	Sorting applied in the query, not in the client	D-001, D-010	Planned M3: query scopes	Planned M3: sorting test	Approved, not yet implemented
FR-006	ASR-002	StatusAudit	Current status must agree with the history	One source of truth for status	D-002, D-008, D-015	Planned M3: RequestStatusService	Planned M3: consistency test	Approved, not yet implemented

Req ID	ASR	Module / component	Data and persistence impact	Design / interface decision	Technology and decision reference	Implementation evidence	Verification evidence	Status
FR-007	ASR-003	StatusAudit, Notification	New status visible on the next read	Notification consumes the event. It is not a step inside the update	D-009, D-011, CR-011	Planned M3: RequestStatusChanged event	Planned M3: listener test	Approved, not yet implemented
FR-008	ASR-005, ASR-006	Retention	Records purged 5 years after last update	Scheduled purge command	CR-012, D-013	Planned M3: purge command	Planned M3: retention test	Approved, not yet implemented
FR-009	ASR-001, ASR-002	RequestQuery	Indexed pagination, 50 rows per page	Pagination mandatory on every list view	D-007	Planned M3: paginated queries	Planned M3: pagination test	Approved, not yet implemented
FR-010	ASR-002	Authentication	users.role holds one of three roles	Session authentication, policies applied at the data layer	D-012, CR-009	Planned M3: authentication scaffolding	Planned M3: login and role tests	Approved, not yet implemented
FR-011	ASR-002	Authorisation	Role determines which rows are visible	Enforced in the data layer, not per screen	D-012, CR-009	Planned M3: ServiceRequestPolicy	Planned M3: policy test per role	Approved, not yet implemented
FR-012	ASR-001	RequestQuery	Reads only the fields the summary needs	Summary view separate from the detail read	D-006	Planned M3: list page props	Planned M3: feature test	Approved, not yet implemented
FR-013	ASR-001, ASR-002	RequestQuery, Authorisation	Filters and sorts scoped by authorisation	Sortable columns whitelisted, authorisation	D-012	Planned M3: query builder	Planned M3: combination and authorisation tests	Approved, not yet implemented

Req ID	ASR	Module / component	Data and persistence impact	Design / interface decision	Technology and decision reference	Implementation evidence	Verification evidence	Status
				applied in the query				
FR-014	ASR-002	StatusAudit	Detail read includes the change history	Detail page receives history with the request	D-002	Planned M3: detail page props	Planned M3: feature test	Approved, not yet implemented
FR-015	ASR-004	Assignment	Writes request_assignments rows	Offer and accept workflow rather than direct reassignment	D-006	Planned M3: AssignmentService	Planned M3: assignment test	Approved, not yet implemented
FR-016	ASR-002, ASR-005	Assignment, StatusAudit	Assignment history retained for 5 years after last update	Assignment changes recorded the same way status changes are	D-002, CR-012	Planned M3: assignment history	Planned M3: retention and audit test	Approved, not yet implemented
FR-017	ASR-002	StatusAudit	Status limited to the seeded enumeration	Transition validated in the service and backed by a database constraint	D-015, CR-010	Planned M3: transition table and enum	Planned M3: illegal transition test	Approved, not yet implemented
FR-018	ASR-002	Authorisation, StatusAudit	Authorisation checked before the write commits	Policy check on the service call path	D-012	Planned M3: policy in transition path	Planned M3: unauthorised update test	Approved, not yet implemented

Req ID	ASR	Module / component	Data and persistence impact	Design / interface decision	Technology and decision reference	Implementation evidence	Verification evidence	Status
FR-019	ASR-002, ASR-005	StatusAudit	Append-only history with actor and timestamp	Audit row written inside the same transaction as the status change	D-008, D-009	Planned M3: RequestStatusService transition	Planned M3: StatusTransitionTest	Approved, not yet implemented
FR-020	ASR-005	StatusAudit	request_comments retained 5 years after last update	Adding a comment is an audited action	D-002, CR-012	Planned M3: comment creation	Planned M3: feature test	Approved, not yet implemented
FR-021	ASR-002	Assignment, StatusAudit	Resolution is a status change, not a deletion	Resolve is a transition. The request stays in the system	D-015	Planned M3: resolve action	Planned M3: transition test	Approved, not yet implemented
FR-022	ASR-002	Authorisation, Reporting	Reads the same tables at management scope	Same query path, wider authorisation scope	D-012	Planned M3: management scope	Planned M3: scope test	Approved, not yet implemented
FR-023	ASR-001	Reporting	Overdue derived from timestamps, not stored	Overdue computed on read so the threshold can change	D-003	Planned M3: reporting query	Planned M3: overdue test	Approved, not yet implemented
FR-024	ASR-001	Reporting	Aggregates over the status history	On-demand report rather than a live dashboard	D-003	Planned M3: summary report	Planned M3: reporting test	Approved, not yet implemented
NFR-001	ASR-002	Authentication	Password hashes only, never plain text	bcrypt with cost factor 12	CR-013, D-012	Planned M3: hashing configuration	Planned M3: hash verification test	Approved, not yet

Req ID	ASR	Module / component	Data and persistence impact	Design / interface decision	Technology and decision reference	Implementation evidence	Verification evidence	Status
								implemented
NFR-002	ASR-002	Authorisation, RequestQuery	Authorisation applied on every read	Cross-cutting policy layer rather than per-screen guards	D-012	Planned M3: policy layer	Planned M3: policy test matrix	Approved, not yet implemented
NFR-003	ASR-002	StatusAudit	Actor and timestamp recorded per change	Recorded inside the transaction	D-008	Planned M3: history writes	Planned M3: audit field test	Approved, not yet implemented
NFR-004	ASR-001	RequestQuery	Indexes sized against a 5-year data volume	Pagination plus indexes on status, due date and assignee	D-007	Planned M3: migrations with indexes	Planned M3: timed query check	Approved, not yet implemented
NFR-005	ASR-003	Deployment	Availability depends on the chosen host	Single instance accepted as a point of failure at MVP	D-013	Planned M3: deployment configuration	Planned M4: availability observation	Approved, not yet implemented
NFR-006	ASR-003, ASR-005	RequestQuery, Deployment	Backups consume the storage allowance	Host backup or a scheduled database dump	D-013	Planned M3: backup configuration	Planned M3: restore rehearsal	Approved, not yet implemented
NFR-007	Not architecturally significant	UI	None	Every screen within three clicks of the landing view	D-006	Planned M3: navigation structure	Planned M3: click-path walkthrough	Approved, not yet implemented

Req ID	ASR	Module / component	Data and persistence impact	Design / interface decision	Technology and decision reference	Implementation evidence	Verification evidence	Status
NFR-008	ASR-004	Submission, UI	None	Field-level validation messages returned with the response	D-007	Planned M3: error handling	Planned M3: validation message test	Approved, not yet implemented
NFR-009	ASR-004	Repository governance	None	Two-reviewer approval before merge	Section 10, D-014	In progress: pull request history	Planned M3: review evidence in PRs	In development

6.3 Matrix filled by Milestone 3

Req ID	Acceptance criteria	Implementation/ component	Test cases	Expected results	Technology and decision reference	Implementation evidence
FR-001	The submitted form is saved in the database	StoreServiceRequest, ServiceRequestController::store saves the request and its first history row (open) in one transaction.	Automated tests: SubmitRequestTest: "a signed-in user can submit a request", "guests cannot submit a request". TC-E2E-01 (submit step).	The request is stored with the requester, status open and one history row. A guest is sent to the login page.	Laravel 13, PostgreSQL. D-002, D-008.	TC-E2E-01 passes the submit step.
FR-002	The requested data reaches the requester.	requests/index (own list) and requests/show (detail with history and comments).	RequestVisibilityTest: "the list shows each role only what it may see" (requestor case), "the detail page is only open to users who may see the request" (own = 200).	A requester sees their own request in the list and can open its detail page.	Inertia + React pages. D-006, D-012.	Manual inspection of the application. Signed in as a requestor, staff, and manager.

Req ID	Acceptance criteria	Implementation/ component	Test cases	Expected results	Technology and decision reference	Implementation evidence
FR-003	If the form does not adhere to the submission rules, it must be rejected	Schema prototype and the planned form validation.	Schema check test	A valid request is stored. Invalid inputs causes the request to be rejected.	Schema_check_tests .sql file for tests, schema_check_tests _output.docx contains outputs.	No defects found
FR-004	On submission the user gets feedback: the problem if it fails, a confirmation if it succeeds.	Custom messages in StoreServiceRequest.	Automated tests TC-E2E-01 ("Request submitted" message).	A success message with the CC- reference after saving. Field-level messages after a failure.	Inertia flash props. CR-011 (email part tracked under FR-007).	TC-E2E-01 shows the message.
FR-006	The displayed status is the correct status.	Status column on service_requests, StatusBadge component. History rows written with each change.	Automated test: status-badge.test.tsx. TakeRequestTest: "staff can take an open, unassigned request"	The badge shows the stored status	D-002, D-008, D-015.	Success on automated tests. Partially implemented, as it only Take changes status so far.
FR-007	After a status update, a refresh shows the new status (and the requester is emailed, CR-011).	Every page load reads the database (no caching). Email: Resend transport configured (D-019). NotifyRequesterOfStatusChange listener on RequestStatusChanged (D-011), merged in PR #24.	Email: manual test 15/10/2026, redirected by MAIL_DEMO_TO and delivered.	The new status shows on the next read. A test email arrives in the demo inbox.	Laravel Mail, Resend. D-011, D-019.	Email delivers to correct email address.
FR-008	Records are retained for 5 years after last update, then automatically purged	On Delete restriction on foreign keys	7 Schema check test	Users, categories, requests and transitions still referenced by other rows cannot be deleted or renamed.	Schema_check_tests .sql file for tests, schema_check_tests _output.docx contains outputs.	Implemented. Records are protected from premature deletion , and a scheduled

Req ID	Acceptance criteria	Implementation/ component	Test cases	Expected results	Technology and decision reference	Implementation evidence
						deletion is added.
FR-009	The requester sees 50 requests per page and can page back to older ones.	ServiceRequestController::index uses paginate(50), Pagination component.	No automated test. Personally inspected the front end.	Page 1 shows at most 50 rows with links to further pages.	D-006.	Personally inspected the front end. Implemented
FR-010	Users are only assigned one role, and must have one role at minimum	Checks on role options. Role defaults to requestor.	2 Schema check tests	A role outside the decided options are rejected, and a new user receives “requestor” as default	Schema_check_tests .sql file for tests, schema_check_tests _output.docx contains outputs.	No defects.
FR-011	Requesters see only their own requests. Staff see unassigned ones and their own assignments, management sees all.	ServiceRequest::scopeVisibleTo() for lists, ServiceRequestPolicy::view() for detail pages.	RequestVisibilityTest: list per role (3 cases) and detail page (6 cases). CommentTest: "a user who cannot see a request cannot comment on it".	Each role gets exactly its expected set. A forbidden detail page returns 403.	Decision table (role x assignment state). D-012, ASR-002.	All features passed the automated tests. implemented.
FR-012	Views show enough to decide: category, status, time submitted, assignee.	Request list columns: reference, category, location, status (with Overdue badge), assignee, last updated.	No automated test, checked in TC-E2E runs.	All listed fields appear on every row.	D-006.	Implemented, verified manually.
FR-013	Search, filter and sort show only authorised matches without errors.	Search (q), status, category and date filters, "assigned to me", all applied on top of scopeVisibleTo(). Sorting not built.	filter-bar.test.tsx (3). workflow.test.ts: "cleanFilters" (2).	Only filled-in filters are sent, results stay inside the user's visible set.	D-012, ASR-002.	All automated tests pass. Partially implemented: sorting not built.

Req ID	Acceptance criteria	Implementation/ component	Test cases	Expected results	Technology and decision reference	Implementation evidence
FR-014	A request can be expanded to full details, including its change history.	requests/show with history and comments.	RequestVisibilityTest (detail access). TakeRequestTest: "the detail page offers Take only when it is allowed".	Authorised users see details and history, others get 403.	D-002.	All automated tests passed. Implemented
FR-015	Staff can assign requests to themselves, offer requests to others and accept offered requests.	State checks on the request assignments, and self assignments	5 Schema check tests.	Offers, self assignments and acceptances are stored. Invalid stated are rejected.	Schema_check_tests .sql file for tests, schema_check_tests _output.docx contains outputs.	No defects.
FR-016	All assignments, offers and acceptances are recorded with a timestamp.	Offers via foreign keys. Does not allow NULL values, Checks responds after creation. On Delete restrictions	3 Schema check tests	Responses dated before its offer and offers/assignments without a valid staff member are rejected. Staff members with assignments cannot be deleted.	Schema_check_tests .sql file for tests, schema_check_tests _output.docx contains outputs.	No defects, Purge after 5 years not yet tested.
FR-017	Status changes are only allowed to change to the set values, and can only transition along a transaction pipeline.	Checks on the status, the new status and old status. Five listed transitions.	6 Schema check tests	Statuses outside the list are rejected. Statuses not adhering to the pipeline are also rejected.	Schema_check_tests .sql file for tests, schema_check_tests _output.docx contains outputs.	No defects.
FR-018	Only authorised staff can update a request.	ServiceRequestPolicy::take.	TakeRequestTest: "only staff may take a request", "a request that is already assigned cannot be taken".	Non-staff and other staff get 403, nothing changes.	D-012.	all automated tests pass. Partially implemented:

Req ID	Acceptance criteria	Implementation/ component	Test cases	Expected results	Technology and decision reference	Implementation evidence
						status updates not built.
FR-019	Status changes records the timestamp and the author. Retained for 5 years.	Updates the request status history when a change is made.	5 Schema check tests	First entry from no status to open is allowed. Other starting options are rejected. Missing and invalid inputs are also rejected.	Schema_check_tests .sql file for tests, schema_check_tests _output.docx contains outputs.	No defects.
FR-020	Staff can add comments and resolution notes to a request. Comments/notes are held for 5 years.	Checks cannot be null, and author links	4 Schema check tests	Only valid comments are stored. Empty comments and comments made by unauthorised authors are reject.	Schema_check_tests .sql file for tests, schema_check_tests _output.docx contains outputs.	No defects. This should also allow additional security if a requestor gains access to the comments.
FR-021	Assigned staff can change a fulfilled request to Resolved. The request remains inside the history logs.	Checks that requests has a timestamp saved when moved to Resolved. On Delete Restrictions.	6 Schema check tests	Resolving and closing of requests are accepted if it has a timestamp. Changes to resolved or closed status are rejected without a timestamp.	.sql file for tests, schema_check_tests _output.docx contains outputs.	No defects.
FR-022	Management can view every request with full history, assignments and comments.	scopeVisibleTo() and the policy allow management everything.	RequestVisibilityTest: management list (all 4) and detail of another staff member's request (200).	Management sees every request and its detail page.	D-012.	All automated test passed. Implemented.
FR-023	Requests not resolved within 7 days are flagged as overdue.	ServiceRequest::OVERDUE_DAYS = 7, overdue scope, Overdue badge, dashboard count.	DashboardTest: "counts requests by status, category and overdue". status-badge.test.tsx:	A request created 10 days ago counts as overdue, newer ones do not.	Dashboard. FR-023 estimate.	all Automated tests pass. Implemented.

Req ID	Acceptance criteria	Implementation/ component	Test cases	Expected results	Technology and decision reference	Implementation evidence
			"shows an Overdue badge only when overdue".			
FR-024	Management can review resolved requests. Basic analysis such as average time to resolution.	DashboardController: counts by status and category, overdue count, category and date filters.	DashboardTest (category filter case).	Totals change with the category filter.	D-003.	All automated tests pass. Partially implemented: average time to resolution not built.
NFR-001	Passwords are stored only as bcrypt hashes, cost factor 12.	Laravel bcrypt, BCRYPT_ROUNDS=12 in .env.example (tests use 4 for speed).	PasswordUpdateTest. seeded users hashed with bcrypt.	No plain-text password is stored.	CR-013.	Implemented, configuration evidence.
NFR-002						
NFR-003	Accurate timestamps are made and the author is saved if a status change is made.	History logs are linked to an author via foreign key. Logs does not allow NULL values.	1 Schema check test	History logs without a valid user are rejected.	.sql file for tests, schema_check_tests _output.docx contains outputs.	No defects.

6.4 Matrix filled by Milestone 4 / final submission.

This new table closes the traceability chain for the final release v3.0-rc1 (commit 288849f). It completes the rows missing from 6.3 (FR-005, NFR-002, NFR-004 to NFR-009) and replaces 6.3's M3 statuses, which were recorded before sorting, status updates and the average resolution time were finished. 6.1 to 6.3 are kept unchanged as history.

Req ID	Stakeholder	Decision	Final implementation	Final evidence	Open defect / limitation	Final status
FR-001	Requester	D-006, D-007, D-008	ServiceRequestController::store, StoreServiceRequest	SubmitRequestTest, TC-E2E-01	None	Met
FR-002	Requester	D-006, D-012	requests/index and requests/show	RequestVisibilityTest (requester cases)	None	Met
FR-003	Requester	D-007, D-010	StoreServiceRequest rules + category rules, schema CHECKs	TE-01, TE-02, schema checks	None	Met
FR-004	Requester	D-007, CR-011	Success flash with CC- reference, field errors	TC-E2E-01, TE-02	None	Met
FR-005	Requester	D-001, D-010	ServiceRequestController::applySort (whitelisted fields)	RequestSortTest	None	Met
FR-006	Requester	D-002, D-008, D-015	Status column + history, StatusBadge	StatusTransitionTest, status-badge.test.tsx	None	Met
FR-007	Requester	D-009, D-011, D-019	No caching on reads, RequestStatusChanged event + mail listener	StatusNotificationTest, D-019 manual email test	Email reaches one demo inbox (L-01)	Met
FR-008	Requester	CR-012, D-013	ServiceRequest Prunable, daily schedule in routes/console.php	RetentionPruneTest	Scheduler does not run on Render free tier (TD-08)	Partially met
FR-009	Requester	D-007	paginate(50) + Pagination component	Manual check	No automated test	Met
FR-010	All users	D-012, CR-009	users.role CHECK, default requestor, Fortify login	Schema checks, AuthenticationTest	Staff and management roles assigned in the database only (D-022)	Met
FR-011	Staff	D-012, CR-009	scopeVisibleTo(),ServiceRequestPolicy::view	RequestVisibilityTest, TE-03, TC-E2E-02	Rule held in two places (TD-01)	Met
FR-012	Staff	D-006	List columns: reference, category, location, status, overdue, assignee, updated	TC-E2E runs, manual	None	Met

Req ID	Stakeholder	Decision	Final implementation	Final evidence	Open defect / limitation	Final status
FR-013	Staff/Mgmt	D-012	Search, filters, sort on top of scopeVisibleTo()	RequestListFilterTest, RequestSortTest, filter-bar.test.tsx	No status/updated_at index (TD-04)	Met
FR-014	Staff	D-002	requests/show with history, comments, assignments	RequestVisibilityTest (detail)	None	Met
FR-015	Staff	D-006, D-020, CR-032	Take, OfferService (offer, accept, decline, pass-on)	TakeRequestTest, OfferTest (15), TE-04	Row locks only tested on PostgreSQL (TD-03)	Met
FR-016	Staff	D-002, CR-012	request_assignments rows with actor and timestamps	OfferTest, schema checks	5-year purge does not run on staging (TD-08)	Partially met
FR-017	Staff	D-015, CR-010	Transition table + composite FK, RequestStatusService	StatusTransitionTest, schema checks	Two places to edit for a new status (TD-02)	Met
FR-018	Staff	D-012	ServiceRequestPolicy::changeStatus / take	StatusTransitionTest, TakeRequestTest	None	Met
FR-019	Staff	D-008, D-009	History row written in the same transaction	StatusTransitionTest, schema checks	None	Met
FR-020	Staff	D-002, CR-012	RequestCommentController, resolution note required on resolve	CommentTest	None	Met
FR-021	Staff	D-015	Resolve is a transition, "Active only" filter	StatusTransitionTest	None	Met
FR-022	Management	D-012	Management scope sees all requests and details	RequestVisibilityTest (management)	None	Met
FR-023	Management	D-003	overdue scope (7 days), badge, dashboard count	DashboardTest. TE-05	None	Met
FR-024	Management	D-003	DashboardController counts + average resolution time	DashboardTest (average cases)	None	Met

Req ID	Stakeholder	Decision	Final implementation	Final evidence	Open defect / limitation	Final status
NFR-001	Master Brief	CR-013, D-012	bcrypt, BCRYPT_ROUNDS=12	PasswordUpdateTest. config	None	Met
NFR-002	Master Brief	D-012	Policies + visibility scope on every read	RequestVisibilityTest. 403 tests	None	Met
NFR-003	Management	D-008	changed_by + changed_at on each history row	Schema check. StatusTransitionTest	None	Met
NFR-004	Staff	D-007, OD-07	Paginated, filtered queries	tests/perf/request-list.mjs: max 193 ms (14.8)	Local only. Neon latency not measured	Met
NFR-005	Management	D-013, D-018	Single Render free instance	None	Free tier sleeps after 15 min (L-03)	Not met
NFR-006	Master Brief	D-013, D-016	Neon 6-hour history only	None	No pg_dump or restore rehearsal (L-04)	Not met
NFR-007	Requester	D-006	Sidebar navigation	Walkthrough on 16/10/2026	80% user test not run (L-07)	Partially met
NFR-008	Requester	D-007	Field-level messages in StoreServiceRequest	Walkthrough on 16/10/2026	None	Met
NFR-009	Team	D-014, Section 10	Main Protection ruleset (2 approvals)	GitHub ruleset. PR history	Review mostly outside PRs.	Partially met

7. Risk Register

7.1 Index

ID	Risk	Prob.	Impact	Priority	Owner	Status
R-001	Team lacks production experience with the selected stack	Med	High	High	Xander	Closed
R-002	Free-tier hosting and database limits exceeded or discovered late	Med	Med	Med	Michael	Accepted
R-003	Inadequate role-based access control exposes request information	Low	High	High	Jared	Closed
R-004	Uncontrolled scope creep	Med	Med	Med	Xander	Closed
R-005	Two-reviewer approval creates a review bottleneck	Med	Med	Med	Michael	Occurred
R-006	AI-assisted work accepted without sufficient verification	Med	High	High	Jared	Mitigated
R-007	Duplicate, incomplete or miscategorised requests	Med	Med	Med	Xander	Accepted
R-008	Concurrent status edits lose an update	Med	High	High	Xander	Mitigated
R-009	No background worker on the chosen free-tier host	Med	Med	Med	Michael	Occurred, accepted
R-010	Session and CSRF misconfiguration	Low	Med	Med	Jared	Closed
R-011	Status changed outside the service layer, bypassing the audit trail	Low	High	High	Xander	Mitigated
R-012	Whole team learning PHP at the same time	Med	Med	Med	Michael	Closed
R-013	Free-tier database pauses or fills during assessment	Med	Med	Med	Michael	Accepted

ID	Risk	Prob.	Impact	Priority	Owner	Status
R-014	Toolchain unavailable on the BC Desktop	Med	Med	Med	Jared	Accepted (contingency used)
R-015	Stack unvalidated because construction was deferred	Med	High	High	Xander	Closed
R-016	Inertia coupling limits future non-browser clients	Low	Low	Low	Jared	Accepted

R-001: Team lacks production experience with the selected stack

Field	Record
Description	Team lacks prior production experience with the technology stack eventually selected in M2, causing slower-than-planned implementation in M3.
Cause	Limited prior exposure beyond coursework-level use of the relevant framework(s).
Probability	Med
Impact	High
Priority	High
Mitigation	Select the M2 stack using the required weighted decision matrix, explicitly weighting team capability/learning curve. Build a small proof-of-concept before committing.
Contingency	Fall back to the more familiar of the two shortlisted M2 stacks if velocity targets are missed by the M3 checkpoint.
Owner	Xander
Status	Mitigating

Field	Record
Milestone 2 review	Stack selected on evidence in D-007 using the section 13.3 matrix, with team capability and framework learning weighted 20 of 100 combined. Named fallback is now bundle B-003, the same Laravel backend with Blade views, which replaces only the presentation layer. The proof-of-concept part of the mitigation is not yet done and is carried as R-015.

R-002: Free-tier hosting and database limits exceeded or discovered late

Field	Record
Description	Free-tier hosting/database limits (storage, connections, request quota) are exceeded or discovered too late, constraining deployment or demo reliability.
Cause	Cost constraint (Section 5) drives selection of free-tier services without upfront investigation of their limits.
Probability	Med
Impact	Med
Priority	Med
Mitigation	Document free-tier limits for shortlisted platforms during M2 research. Size the data model and expected request volume against those limits before committing.
Contingency	Identify a low-cost upgrade path in advance so the team can react quickly if limits are approached before M4.
Owner	Michael
Status	Mitigating
Milestone 2 review	D-013 fixes the topology at one web process, one queue worker and one managed PostgreSQL instance, which is the smallest footprint that satisfies the design. Section 13.7.4 records the limits that interact with the 5-year retention rule. The specific provider is still open, so this risk stays open until D-013 is closed.

Field	Record
Milestone 3 Review	Reviewed on 10/10/2026. Database provider that is chosen is the free plan of NEON, based in the London region. Neon provides 1GB storage per project, and 100 computing hours per project per billing period, and suspends after 5 minutes without activity (NEON, 2026).

R-003: Inadequate role-based access control exposes request information

Field	Record
Description	Inadequate role-based access control exposes request information
Cause	RBAC is treated as an implementation detail rather than a first-class requirement at baseline.
Probability	Low
Impact	High
Priority	High
Mitigation	Define RBAC roles and access rules as explicit NFRs and acceptance criteria at M1 (Section 4.2). Include access-control test cases in M3 security evidence.
Contingency	Conduct a focused access-control review before staging deployment. Restrict production data exposure until verified.
Owner	Jared
Status	Mitigating
Milestone 2 review	D-012 places authorisation in ServiceRequestPolicy at the data-access layer rather than per screen, so the same rule applies to every query path. The policy test set is specified but not yet written, so the risk remains open until M3 evidence exists. The fourth ‘Admin’ level was resolved by CR-009: three roles for the MVP, with a separate Administrator role deferred.

R-004: Uncontrolled scope creep

Field	Record
Description	Uncontrolled scope creep (for example adding SMS notifications or a mobile app beyond the approved baseline) consumes time needed for core-capability quality.
Cause	Desire to appear comprehensive or impressive at milestone presentations.
Probability	Med
Impact	Med
Priority	Med
Mitigation	Enforce the scope baseline (Section 3). Any proposed addition must pass through the Change Request/Impact Analysis template (Master Brief Appendix E) before implementation.
Contingency	Revert to baseline scope and formally defer the addition to future scope if schedule slippage is detected at a checkpoint.
Owner	Xander
Status	Open
Milestone 2 review	The control worked as intended: the only scope movement in M2, confirming that status-change email is in scope, went through the change record rather than being absorbed silently. No features were added beyond the M1 baseline.

R-005: Two-reviewer approval creates a review bottleneck

Field	Record
Description	With only three team members, the mandatory two-reviewer PR approval effectively requires unanimous team review of every substantive change, creating a review bottleneck or an incentive to rubber-stamp under time pressure.
Cause	Small team size relative to the two-approvals-from-non-authors governance control (Master Brief section 9).
Probability	Med

Field	Record
Impact	Med
Priority	Med
Mitigation	Agree review-turnaround expectations in the Team Working Agreement (Section 10.2). Stagger substantive PRs rather than batching them near deadlines. Reviewers respond to a review request within 24 hours on weekdays. If a reviewer is unavailable for longer, the author records the delay on the pull request rather than merging without the second approval.
Contingency	If a bottleneck occurs, prioritise reviewing PRs that block others' work and record the delay rather than silently skipping meaningful review.
Owner	Michael
Status	Open
Milestone 2 review	Still open. The risk grows in M3 when code review replaces document review, because all three members will be reviewing PHP none of them has written before. Record actual PR turnaround times from M3 so the exposure can be measured rather than estimated.

R-006: AI-assisted work accepted without sufficient verification

Field	Record
Description	AI-generated code, tests or requirement wording is accepted into the controlled baseline without sufficient independent verification, introducing a defect, incorrect assumption or security gap.
Cause	Schedule pressure encourages faster acceptance of AI output. Verification takes time.
Probability	Med
Impact	High
Priority	High

Field	Record
Mitigation	Apply the AI Usage Register (Section 11) to every material AI contribution. The responsible student must test, cite or otherwise verify the specific claim before it enters the baseline.
Contingency	Treat any unverifiable AI-assisted claim as rejected by default rather than including it to save time.
Owner	Jared
Status	Open
Milestone 2 review	Register maintained through M2. The exposure changes in M3: verifying a factual claim in a document is cheaper than verifying generated PHP in a language none of the team has used, so AI-assisted code must be read line by line and covered by a test before merge.

R-007: Duplicate, incomplete or miscategorised requests

Field	Record
Description	Requesters submit duplicate, incomplete or miscategorised service requests, reducing staff's ability to prioritise and increasing the perceived unreliability of the platform.
Cause	This is one of the core problems described in the CivicConnect business need. Unconstrained submission would reproduce it.
Probability	Med
Impact	Med
Priority	Med
Mitigation	Design mandatory fields and a controlled category list into the functional requirements (Section 4.1) rather than leaving submission unconstrained.
Contingency	Allow staff to merge/relabel duplicate or miscategorised requests as a controlled, audited action.

Field	Record
Owner	Xander
Status	Mitigating
Milestone 2 review	Mitigation strengthened: category handling now goes through a dedicated CategoryHandler class per category, resolved from a CategoryRegistry keyed on the category code (D-010, Section 9). A request reclassified out of 'Other' is routed through its own handler rather than a conditional branch, so a reclassification cannot silently default to generic handling.

R-008: Concurrent status edits lose an update

Field	Record
Description	Two staff members open the same request and update its status at nearly the same time. The second write silently overwrites the first, so an action is lost and the audit trail records a state nobody intended.
Cause	Requests are visible to multiple staff members, and without a concurrency control the last write wins.
Probability	Med
Impact	High
Priority	High
Mitigation	D-008 adds an optimistic version column checked inside the transaction. A stale version raises StaleRequestException rather than overwriting. The user is returned to the request with its current state and asked to retry.
Contingency	If conflicts prove frequent in M3 testing, narrow the editable window by refreshing request state when the detail screen is opened, before considering heavier locking.
Owner	Xander
Status	Mitigating

Field	Record
Milestone 2 review	Raised at M2 from Assignment 2 Task 2 research. Design-level mitigation is in place. Verification is a Milestone 3 feature test asserting that a stale version is rejected.

R-009: No background worker on the chosen free-tier host

Field	Record
Description	The selected free-tier host does not support a persistent background worker process, so queued status notifications are accepted by the application but never delivered.
Cause	D-011 makes notification delivery a queued listener, which requires a worker process that many free tiers do not provide.
Probability	Med
Impact	Med
Priority	Med
Mitigation	Make worker support a selection criterion when D-013 closes. Confirm it before committing to a provider.
Contingency	Run the queue from a scheduled task using queue:work with a stop-when-empty flag, accepting delayed delivery, or send the notification synchronously after commit as a temporary measure.
Owner	Michael
Status	Accepted
Milestone 2 review	Raised at M2 as a direct consequence of D-011 and D-013. Remains open until the hosting provider is chosen in section 13.8.
Milestone 3 review	Reviewed 15/10/2026. Render's free tier has no background workers (D-018). Mitigated with the risk's own contingency: notifications are sent synchronously after commit, and the purge moves to a GitHub Actions schedule. Residual risk: a slower response while an email is sent, and no automatic retry. (The GitHub Actions schedule was planned but not set up. See the Milestone 4 final review and TD-08.)

R-010: Session and CSRF misconfiguration

Field	Record
Description	Session cookie or CSRF settings are wrong for a deployed environment, so users cannot log in, or protection against cross-site request forgery is weakened.
Cause	Session lifetime, cookie domain, secure flag and CSRF token handling differ between local development and a hosted environment.
Probability	Low
Impact	Med
Priority	Med
Mitigation	Keep all environment-specific values in environment variables (section 13.7.2). Add a feature test covering login and an authenticated Inertia request so a broken configuration fails in CI rather than in the demo.
Contingency	Roll back to the last known-good environment configuration and verify against the CI test before redeploying.
Owner	Jared
Status	Mitigating
Milestone 2 review	Revised at M2. The original form of this risk covered CORS and cross-origin token handling for a separate SPA. D-006 and D-012 removed that boundary entirely, so the remaining exposure is ordinary session and CSRF configuration, which is smaller. Downgraded from High to Medium priority.

R-011: Status changed outside the service layer, bypassing the audit trail

Field	Record
Description	A controller, seeder, console command or future feature updates a request's status directly through the model, skipping RequestStatusService, so no history row is written and the audit trail has a gap that cannot be reconstructed.

Field	Record
Cause	Nothing in the framework prevents a direct model update. The rule that only the service may change status is a convention.
Probability	Low
Impact	High
Priority	High
Mitigation	State the rule explicitly in the code standards, enforce it in review, and add a Milestone 3 test asserting that the count of history rows equals the count of status transitions for a request.
Contingency	If a gap is found, record it as a defect, backfill what can be reconstructed from application logs, and state plainly in the PED what cannot be recovered.
Owner	Xander
Status	Mitigating
Milestone 2 review	Raised at M2 from D-002, D-008 and Forward Engineering Consideration #3. Audit gaps cannot be backfilled after the fact, which is why the probability is low but the impact is high.

R-012: Whole team learning PHP at the same time

Field	Record
Description	All three members are learning PHP and Laravel simultaneously, with no member able to act as an internal reference, so early mistakes are less likely to be caught in review and rework is more likely.
Cause	D-007 selects a stack in a language none of the team has used, accepted deliberately as a trade-off.
Probability	Med
Impact	Med

Field	Record
Priority	Med
Mitigation	Pair on the first vertical slice rather than splitting unfamiliar work three ways. Follow the official Laravel documentation and the official starter kit structure rather than mixed tutorials. Keep the first PRs small so review is meaningful.
Contingency	Invoke the D-007 fallback to bundle B-003 if the first slice takes materially longer than planned.
Owner	Michael
Status	Mitigating
Milestone 2 review	Raised at M2 as a distinct exposure from R-001: R-001 is about stack experience generally, this is about having no internal expert to review against.

R-013: Free-tier database pauses or fills during assessment

Field	Record
Description	A free-tier PostgreSQL instance pauses after inactivity or approaches its storage cap, making the demonstration unreliable or blocking writes during assessment.
Cause	Free tiers commonly suspend idle databases and cap storage, which interacts with the 5-year retention requirement in FR-008.
Probability	Med
Impact	Med
Priority	Med
Mitigation	Record the chosen provider's actual limits when D-013 closes. Seed a small realistic dataset rather than bulk test data. Warm the connection before presenting.
Contingency	Keep a local PostgreSQL environment ready as a demonstration fallback and state plainly that it is not the hosted environment.

Field	Record
Owner	Michael
Status	Accepted
Milestone 2 review	Raised at M2, related to R-002 but specific to demonstration reliability rather than to sizing.
Milestone 3 review	Reviewed on 10/10/2026. NEON's free plan pauses idle databases after 5 minutes, and resumes on the next connection. This means in the demonstration there is a risk of slower first requests. A potential mitigation strategy is to "warm up" the connection before presentation. The NEON database has been staged successfully and verified with 53/53 successful tests. Render's free web service also sleeps after 15 minutes without traffic, so the app must be given time to "warm up" as well.

R-014: Toolchain unavailable on the BC Desktop

Field	Record
Description	PHP, Composer, Node or PostgreSQL cannot be installed or run on the BC Desktop platform, so the project cannot be built or assessed in the institutional environment.
Cause	CON-007 and Master Brief section 25: the institution does not guarantee that a chosen technology is available or supported on its platform.
Probability	Med
Impact	Med
Priority	Med
Mitigation	Install and run the full toolchain on a BC Desktop machine and record the result in section 13.3 before construction begins.
Contingency	Develop on team laptops and document the limitation, or invoke the D-007 fallback if the difference blocks assessment.
Owner	Jared

Field	Record
Status	Open
Milestone 2 review	Raised at M2. Until the installation is verified, the dev-environment row of the section 13.3 matrix is an estimate rather than evidence. This is a few hours of work and should be closed early in M3.

R-015: Stack unvalidated because construction was deferred

Field	Record
Description	With no proof of concept built during M2, the assumptions behind D-007, D-008 and D-013 are untested, so a problem with the stack, the transaction design or the hosting topology would surface only in M3 under delivery pressure.
Cause	D-014 defers construction to M3, concentrating learning and delivery in the same window.
Probability	Med
Impact	High
Priority	High
Mitigation	Timebox a spike at the start of M3: create the Laravel project, connect it to PostgreSQL, run one migration and render one Inertia page, before committing to the full build order.
Contingency	If the spike reveals a blocking problem, invoke the D-007 fallback to B-003 and record the change through the change-control process.
Owner	Xander
Status	Open
Milestone 2 review	Raised at M2 alongside D-014. This is the direct cost of deferring construction and should be stated plainly in the defence rather than left implicit.

R-016: Inertia coupling limits future non-browser clients

Field	Record
Description	Because D-006 exchanges page props rather than a published API, a future mobile client or third-party integration would require a JSON API to be added before it could consume CivicConnect data.
Cause	Inertia couples the presentation layer to the application, which is the trade-off accepted in D-006 and D-007.
Probability	Low
Impact	Low
Priority	Low
Mitigation	Keep business rules in services rather than controllers so that adding routes/api.php later is additive rather than a rewrite.
Contingency	Add a JSON API alongside Inertia over the same services if a second client is ever required.
Owner	Jared
Status	Accepted
Milestone 2 review	Accepted at M2. No stakeholder requirement in the approved scope calls for a non-browser client, and the scope baseline excludes a native mobile app.

Finalised risk register

This review concludes the register for release v3.0-rc1. The Status rows in the individual records reflect the M2 and M3 reviews.

ID	Risk	New status	Milestone 4 final review (18/10/2026)
R-001	Lack of production experience	Closed	The stack was built, tested and deployed to staging (Section 14).

ID	Risk	New status	Milestone 4 final review (18/10/2026)
R-002	Free-tier limits	Accepted	Limits recorded in 13.7.4. Usage stays within them for the MVP.
R-003	RBAC exposes data	Closed	RequestVisibilityTest guards the rule. DF-04 fixed. Residual: TD-01.
R-004	Scope creep	Closed	Scope frozen at release v3.0-rc1. Every change went through the change record.
R-005	Review bottleneck	Occurred	Reviews moved to team discussion. Consequence is NFR-009 partially met.
R-006	Unverified AI work	Mitigated	AI register records verification. CI test suites check AI-assisted code.
R-007	Duplicate or miscategorised requests	Accepted	Controlled categories (D-001) reduce miscategorising. No duplicate detection.
R-008	Concurrent status edits	Mitigated	Version check, conditional Take, row locks. Residual: locks untested in CI (TD-03).
R-009	No background worker	Occurred, accepted	Mail sent synchronously. Purge does not run (TD-08). Blocks production (17.4).
R-010	Session/CSRF misconfiguration	Closed	Laravel session auth and CSRF defaults. No defects raised.
R-011	Status changed outside the service	Mitigated	All transitions go through RequestStatusService. Database CHECKs back it up.
R-012	Team learning PHP	Closed	Product delivered. Risk no longer active.
R-013	Database pauses during assessment	Accepted	Warm up the app and database before the demonstration.

ID	Risk	New status	Milestone 4 final review (18/10/2026)
R-014	Toolchain unavailable on BC Desktop	Accepted (contingency used)	Developed on the team's own machines (the contingency). The toolchain was not verified on a BC Desktop.
R-015	Stack unvalidated (construction deferred)	Closed	Construction completed in M3. Stack validated by tests and staging.
R-016	Inertia coupling	Accepted	No non-browser client needed (OD-09). Unchanged.

8. Forward Engineering Considerations

	Concern	Why It Matters Now	Decision/Activity It Influences	Information Still Missing	Risk of Ignoring It	Milestone 2 status	Resolved by / next step	Final status (M4)
1	Authentication & role-based access control (Requester / Staff / Management)	The three-role model is already implied by the minimum business capabilities (Master Brief section 3) and shapes which requirements, screens and data fields exist at all.	M2 authentication/architecture design, M2 data model, M3 security testing, M4 security evidence.	Whether staff need finer sub-roles (e.g. by department), and whether management needs read-only or configurable access.	Retrofitting access control after construction typically forces rework of the data model, API contracts and UI, with a higher chance of an overlooked authorisation gap.	Resolved for the three-role model. Department sub-roles carried forward	D-012 and CR-009. Sub-roles reconsidered only if a stakeholder need appears	Closed: three roles (D-012), sub-roles not needed (OD-03).
2	Controlled request category &	Several FRs (submission, filtering, reporting) and the quality/NFR	M1 requirement wording, M2 data model, M4	The final list of categories/status values, and whether it must	Free-text categorisation reproduces the exact fragmentation problem	Resolved	CR-010 fixes the status set. D-001 and D-010 fix the	Closed (CR-010, D-001, D-015).

	Concern	Why It Matters Now	Decision/Activity It Influences	Information Still Missing	Risk of Ignoring It	Milestone 2 status	Resolved by / next step	Final status (M4)
	status taxonomy	constraint depend on a fixed, agreed vocabulary rather than free text.	management reporting.	be extensible later without a code change.	CivicConnect is meant to solve. Late taxonomy changes ripple through requirements, data and reports.		category set and handling	
3	Request audit trail / status-change accountability	The business need explicitly names 'weak accountability for changes to request status' as a current problem. The data model must capture who changed what and when from the outset.	M2 data/persistence design, M3 traceability evidence, M4 stakeholder validation of accountability.	Retention period for audit history and whether staff need full history or only current state.	Adding an audit trail after core tables are built typically requires a migration touching every write path. Audit gaps found late cannot be backfilled.	Designed, not yet implemented	D-008 and D-009. Retention of audit history settled by CR-012. Enforcement verified in M3 (R-011)	Implemented: append-only history in the same transaction (D-008, StatusTransition Test).
4	Deployment platform & environment strategy (dev/test/staging/production)	The cost constraint (free/low-cost preference) and the later need for staging-to-production parity (Master Brief section 17) both depend on understanding platform options and limits early.	M2 technology decision matrix, M3 staging deployment, M4 production release.	Concrete free-tier limits, and whether the eventual stack's hosting options support proper environment separation.	Discovering a platform incompatibility at M3/M4 could force an emergency migration under deadline pressure with no time to test properly.	Partly resolved: topology fixed, provider deferred	D-013. Provider selection closes in Section 13.8 before the M3 staging deployment	Closed: Neon (D-016) and Render (D-018).
5	Testability of core workflows (submission	If requirements and design don't anticipate automated testing (Master Brief	M1 acceptance-criteria wording, M2 design decisions (e.g. avoiding tight	Which workflows are highest-risk and therefore deserve the	Untestable design decisions made in M2 (e.g. logic embedded directly in UI code) are	In progress	D-006 keeps business rules in services rather than the UI,	Implemented: business rules in services. Covered by

	Concern	Why It Matters Now	Decision/Activity It Influences	Information Still Missing	Risk of Ignoring It	Milestone 2 status	Resolved by / next step	Final status (M4)
	→ assignment → resolution)	section 15), M3's automated test evidence will be retrofitted and weaker.	coupling), M3 test strategy.	earliest test coverage.	expensive to unpick once construction is underway.		which is what makes them testable. Test set defined in Section 6.2, written in M3	Pest, Vitest and E2E (Section 14).
6	Sensitive-data handling for security/IT-support request categories	Some categories (security concerns, IT support) may contain sensitive information, which interacts directly with RBAC (Concern 1) and drives specific NFRs.	M1 NFRs (Section 4.2), M2 architecture (e.g. encryption at rest/in transit), M3 security review.	Whether any category needs restricted visibility even from general staff, not only from requesters.	Sensitive information could be exposed to unauthorised staff, leaving a residual security risk the team cannot honestly claim to understand at M4 (Master Brief section 23).	Partly resolved	D-012 applies authorisation at the data layer. Encryption at rest depends on the provider chosen in D-013	Partly closed: authorisation at the data layer. Encryption at rest relies on the provider default (OD-05).
7	Application evidence and the start of construction	D-014 defers construction to Milestone 3, so the baseline is not yet backed by code and the end-to-end trace stops at the technology decision	M3 build order, Section 6.2 status values, criterion F evidence	How long the first vertical slice takes with a stack new to the team	Learning and delivery are compressed into the same window, and an unworkable assumption would surface late (R-015)	Open - construction deferred to Milestone 3 (D-014)	R-015 spike at the start of M3, then the first vertical slice covering FR-001, FR-017 and FR-019	Closed: construction completed in M3 (Section 14).
8	Staging and production parity	Master Brief section 17 expects staging to resemble production, and the queue worker required by D-011 must exist in both	D-013 provider selection, M3 staging deployment, M4 release	Whether the chosen free tier supports a persistent worker and environment separation	A platform that cannot run a worker would break notification delivery only after deployment (R-009)	Closed. The database will be hosted using NEON, the application will be hosted via Render.	OD-01 and OD-02, confirm worker and scheduler support before the M3 staging deployment (R-009, AS-001)	Accepted with limitation: no worker or scheduler on Render (R-009, TD-05, TD-08).

9. Engineering Decision Log

9.1 Index

ID	Decision	Status
D-001	Controlled request category vocabulary	Baselined M1, Later Consequence updated at M2
D-002	Append-only status history	Baselined M1, Later Consequence updated at M2
D-003	On-demand management reporting at MVP	Baselined M1, Later Consequence updated at M2
D-004 (Deferred, now closed)	Technology-stack selection	Closed at M2, resolved by D-007
D-005 (Deferred, now closed)	Deployment and hosting platform selection	Closed at M2, superseded by D-013
D-006	Architecture style	New at M2, option C (SPA) superseded by option B (Inertia) through CR-017
D-007	Technology stack selection	New at M2, closes D-004
D-008	Status change and audit write in a single transaction	New at M2
D-009	Splitting status-change side effects	New at M2, adapts the Assignment 2 recommendation
D-010	Category-specific handling	New at M2
D-011	Status-to-notification integration mechanism	New at M2
D-012	Authentication and role-based access control	New at M2
D-013	Deployment direction and environment strategy	New at M2, supersedes D-005. Closed by D-016. Hosting of the application is deferred to D-017.
D-014	Construction deferred to Milestone 3	New at M2
D-015	Status taxonomy and controlled transitions	New at M2, closes Forward Engineering Consideration #2
D-016	Hosting service for the database.	New at M3.

ID	Decision	Status
D-017	Hosting service for the application	New at M3
D-018	Chosen Hosting service for Application	New at M3, supersedes D-017
D-019	Emailing service	New at M3. Closes OD-02.
D-020	Offer and decline rule	New at M3. Confirmed at M4 (CR-032).
D-021	Final release decision	New at M4.
D-022	Role assignment	New at M4. Closes OD-10

D-001: Controlled request category vocabulary

Field	Record
Status	Baselined M1, Later Consequence updated at M2
Context	CivicConnect requires a shared vocabulary for request types to enable consistent submission, staff triage and management reporting.
Constraints	Must stay simple for non-technical requesters. Must cover the categories named in the business need (facility faults, damaged equipment, security concerns, IT support, maintenance, lost property), plus 'Other'.
Alternatives	(a) Free-text field, (b) Fixed controlled list, (c) Fixed list with an 'Other' option requiring staff reclassification.
Decision	Adopt (c): a fixed controlled list with a staff-reclassifiable 'Other' category.
Rationale	Balances requester ease-of-use with the data-quality and reporting needs in the business need, without an over-rigid list that blocks legitimate edge cases.
Trade-offs	Slightly more implementation complexity than free text. Requires a staff workflow for reclassifying 'Other' items.
Risks	If the list is poorly chosen, most requests may default to 'Other', undermining the benefit (linked to Risk R-007).
Evidence	Master Brief section 2 (fragmentation problem), Stakeholder Analysis, Section 2.

Field	Record
Later consequence	M2: realised as the categories table (section 13.2.1) and the per-category handlers of D-010. Reclassification of 'Other' is an audited staff action.

D-002: Append-only status history

Field	Record
Status	Baselined M1, Later Consequence updated at M2
Context	Every service request must support staff and management accountability for status changes.
Constraints	It must remain technology-agnostic at M1. It must not assume a specific database engine.
Alternatives	(a) Store only current status. (b) Store current status plus a full append-only status-change history.
Decision	Adopt (b): every status transition is recorded as an immutable history entry (who, what, when).
Rationale	Directly addresses the 'weak accountability' problem in the business need and is required groundwork for the M3/M4 traceability and quality-evidence standards.
Trade-offs	Slightly larger data model and additional write operations per status change.
Risks	If not enforced consistently across every code path that changes status, gaps could appear in the audit trail (linked to Forward Engineering Consideration #3).
Evidence	Master Brief section 2 problem statement, and section 11 traceability standard.
Later consequence	M2: realised as request_status_history (section 13.2.1), written inside the D-008 transaction. The enforcement rule is set by D-009.

D-003: On-demand management reporting at MVP

Field	Record
Status	Baselined M1, Later Consequence updated at M2

Field	Record
Context	Whether management reporting requires a real-time dashboard or a periodic/on-demand view at MVP stage.
Constraints	Must fit within the schedule constraint (Section 5) and the three-person team capability constraint.
Alternatives	(a) Real-time dashboard, (b) Periodic/on-demand report view.
Decision	Adopt (b) for MVP scope. Defer (a) to future scope.
Rationale	Reduces architectural complexity and risk within the schedule constraint while still satisfying the minimum management capability of identifying open/overdue/resolved requests (Master Brief section 3).
Trade-offs	Less immediate visibility for management than a live dashboard would offer.
Risks	If stakeholders expected real-time visibility, this could become a change request later (Master Brief Appendix E).
Evidence	Scope Baseline, Section 3.3.
Later consequence	M2: realised as an on-demand reporting read path (Reporting component, section 13.1.1). No live dashboard has been introduced.

D-004 (Deferred, now closed): Technology-stack selection

Field	Record
Status	Closed at M2, resolved by D-007
Context	Technology-stack selection for CivicConnect.
Constraints	Not applicable at the time of the M1 entry.
Alternatives	Not applicable at the time of the M1 entry. M2: the nine bundles compared in section 13.3.1.
Decision	DEFERRED to Milestone 2. M2: resolved by D-007.
Rationale	M1 explicitly excludes final technology-stack selection (Milestone 1 Brief section 5). A defensible choice requires the weighted decision matrix and evidence gathering scoped to M2 (Master Brief section 18.1), not yet performed at M1.
Trade-offs	Not applicable at the time of the M1 entry.

Field	Record
Risks	Delaying keeps options open but leaves less M2 time for the required comparison (linked to Risk R-001). Research must begin immediately after M1 submission.
Evidence	Not applicable at the time of the M1 entry. M2: section 13.3 matrices and D-007.
Later consequence	Superseded by D-007 on the date recorded in Document Control.

D-005 (Deferred, now closed): Deployment and hosting platform selection

Field	Record
Status	Closed at M2, superseded by D-013
Context	Deployment/hosting platform selection.
Constraints	Not applicable at the time of the M1 entry.
Alternatives	Not applicable at the time of the M1 entry. M2: the options compared in D-013.
Decision	DEFERRED to Milestone 2, pending D-004. M2: superseded by D-013, which fixes the deployment topology and defers only the provider.
Rationale	Hosting compatibility depends on the technology stack chosen in D-004. Deciding hosting first would risk prematurely constraining that choice.
Trade-offs	Not applicable at the time of the M1 entry.
Risks	Free-tier limits (Risk R-002) must still be scoped generally now (Forward Engineering Consideration #4) even though the specific platform is deferred.
Evidence	Not applicable at the time of the M1 entry. M2: section 13.7 and D-013.
Later consequence	Provider selection remains open and is tracked in section 13.8.

D-006: Architecture style

Field	Record
Status	New at M2, revised to the Inertia presentation layer
Context	CivicConnect needs an architecture that enforces auditable status changes, and that applies authorisation consistently across queries. All this must be done within the free-tier service constraint.
Constraints	The cost constraint, as the services must be free. It needs browser-based access and must be buildable and maintainable by a 3-person team.
Alternatives	A server-rendered application without APIs (option A), a layered modular monolith with an Inertia-rendered React presentation layer (option B), a layered modular monolith with a separate React SPA over a Laravel API (option C), or distributed services (option D).
Decision	Option B, a layered modular monolith application using React and TypeScript rendered through Inertia over Laravel. There is no internal network boundary and no separate JSON API at this stage.
Rationale	Detailed justification and reasoning is in section 13.1.3. In summary: option D turns commits into distributed transactions and would exceed the free-tier constraint. Option C satisfies the requirements but adds a network boundary, CORS handling, a second deployable and duplicated validation for a client the project does not need to serve independently, which is the cost Assignment 2 Task 3 warned against. Option A is the simplest but leaves the team's React and TypeScript skills unused. Option B keeps one deployable and server-side authorisation while still using React for presentation.
Trade-offs	The front-end and back-end deploy together and cannot be released independently. Page props are an internal contract, not a reusable API, so a future mobile or third-party client would require a JSON API to be added alongside Inertia.
Risks	R-001 and R-005, the boundaries may erode under the pressure of the project's schedule. Presentation logic could leak into controllers if the component responsibilities in section 13.1.1 are not enforced in review.
Evidence	Section 8 considerations 3 and 5, D-002, FR-013, FR-019, section 5's cost constraint, and the business needs in section 1.1. Assignment 2 Task 3, sections 4.2 and 4.3.
Later consequence	Influences the repository structure, component names, deployment and the layering, the D-012 authentication approach, and the interface documentation in section 13.6. Because the business logic sits in services, adding a JSON API later is additive rather than a rewrite.
Superseded version	20/09/2026: option C, a React SPA over a Laravel JSON API, was the first selection. It was replaced by option B through CR-017 after the section 13.3 matrix ranked B-009 highest and the SPA's network boundary, CORS handling and second deployable were found to add cost with no requirement that needed them. The SPA analysis is kept as option C in section 13.1.2.

D-007: Technology stack selection

Field	Record
Status	New at M2, closes D-004
Context	The technology stack had to be selected on evidence before construction begins. Section 13.3 scored nine common web bundles against ten weighted criteria drawn from Master Brief section 18.1.
Constraints	Free or low-cost services (CON-003). A three-person team with PHP new to all members and React/TypeScript known to two of the three (CON-006). Tools must run on the BC Desktop or team laptops (CON-007). The fixed milestone schedule (CON-002).
Alternatives	Nine bundles, B-001 to B-009 (section 13.3.1). MERN was excluded before scoring because MongoDB is not relational. The PERN variant was scored as B-001. Leading scores: B-009 Laravel with Inertia 85%, B-003 Laravel with Blade 83%, B-005 Django 82%, B-006 ASP.NET Core with Razor 80%, B-004 Laravel API with a separate SPA 74%.
Decision	Adopt bundle B-009: Laravel (PHP) with Inertia serving React and TypeScript components, over PostgreSQL. This is the highest-scoring bundle in the section 13.3.3 matrix and is consistent with the architecture selected in D-006.
Rationale	B-009 scores highest on the criteria carrying the most weight. It deploys as a single unit, which gives the team a working deployment quickly, and CivicConnect has no requirement to scale the presentation layer independently of the application. It keeps session-based security and framework validation, and it uses the React and TypeScript that two members already work in while the team takes on Laravel. PostgreSQL wins its own sub-matrix at 96% on CHECK constraints, composite foreign keys and transactional DDL, which the D-008 and D-015 integrity design depends on. Tiebreakers between the four candidates within five percentage points: (1) reuse of the React and TypeScript skills two members already have (2) a single deployable with session authentication (3) an officially supported Laravel starter kit for React, Inertia and TypeScript (4) PostgreSQL integrity support. B-009 is the only candidate that meets all four
Trade-offs	PHP is new to all three members, so the framework learning curve is real and is the weakest dimension of every Laravel bundle. Inertia couples the front-end and back-end: page props are not a reusable API contract.
Risks	R-001 learning curve, R-014 toolchain availability on the BC Desktop, not yet verified, R-015 no proof of concept before M3.
Evidence	Section 13.3.3 weighted matrix, section 13.3.4 database sub-matrix, section 13.3.5 result and sensitivity check, Assignment 2 Task 3, the laravel/react-starter-kit repository (Laravel, React, Inertia, TypeScript) as evidence that this combination is an officially supported path.

Field	Record
Later consequence	If velocity targets are missed at the M3 checkpoint, the fallback recorded against R-001 is B-003, the same Laravel backend with Blade views, which replaces only the presentation layer. Target versions and compatibility assumptions are recorded in section 13.3.6 and confirmed against composer.lock and package.json when the project is created.

D-008: Status change and audit write in a single transaction

Field	Record
Status	New at M2
Context	D-002 (Milestone 1) committed CivicConnect to recording every status change as an immutable history entry, but left the transactional and concurrency detail open. That detail has to be settled before the persistence layer is built.
Constraints	Free-tier relational database (Section 5). FR-018 (only the assigned staff member may update a request). NFR-003 (user and timestamp on every status change). Three-person team and academic schedule.
Alternatives	(a) Single relational transaction, application-managed, with optimistic concurrency via a version column and a service-layer validation check backed by a database constraint. (b) Outbox/event-sourced write, where the status change and a separate outbox event are written atomically and a worker process later handles downstream effects.
Decision	Adopt (a). RequestStatusService::transition() updates service_requests.status guarded by an optimistic version check (a stale version raises StaleRequestException, which the controller turns into a redirect back with an error message) and inserts the matching request_status_history row, both inside one DB::transaction call.
Rationale	Option (a) delivers exactly the atomicity D-002 requires without introducing a worker process, message queue or eventual consistency the team does not yet have a justified need for. It maps onto Laravel's DB::transaction(). Laravel has no built-in optimistic locking, so the version check is written explicitly: the update includes WHERE id = ? AND version = ?, increments version, and if no row is affected RequestStatusService throws StaleRequestException.
Trade-offs	The transaction briefly holds a database connection open. Calling code has to catch a version conflict and retry or surface it to the user rather than assuming every write succeeds.
Risks	The database CHECK constraint on status and the database transition table (request_status_transitions) have to be kept in sync by hand until an automated consistency check exists (Forward Engineering Consideration #3). Concurrent staff edits are tracked as R-008.
Evidence	Assignment 2, Task 2, Sections 3.2 to 3.4.

Field	Record
Later consequence	Implemented in RequestStatusService: the status change and its history row are written in one transaction with a version check (StaleRequestException). Take uses a conditional update instead (CC-3). Verified by StatusTransitionTest and the schema checks, R-011 mitigated

D-009: Splitting status-change side effects

Field	Record
Status	New at M2, adapts the Assignment 2 recommendation
Context	FR-016 and FR-019 require an audit entry recording who changed a request's status and when. FR-007 requires the requester to see the update reflected on their side. Forward Engineering Consideration #3 flags that the audit trail has to be designed in from the outset rather than retrofitted.
Constraints	The audit write must stay inside the D-008 transaction so it can never be silently skipped. A notification failure must not roll back the status change. Laravel's event/listener mechanism changes what building this reaction costs compared with the Assignment 2 research.
Alternatives	(a) A general-purpose Observer applied directly at the ServiceRequest entity, as Assignment 2 Task 1 recommended, with the audit write as one of the registered observers. (b) Split the two reactions: audit write synchronous inside the D-008 transaction, every other reaction a queued listener on a RequestStatusChanged event dispatched only after commit. (c) A general-purpose domain-event/pub-sub dispatcher.
Decision	Adopt (b). The audit write is written directly inside RequestStatusService::transition() as part of D-008. NotifyRequesterOfStatusChange is a queued listener (implements ShouldQueue) on the RequestStatusChanged event, dispatched once the transaction commits. RequestStatusChanged implements ShouldDispatchAfterCommit, so Laravel dispatches it only after the D-008 transaction commits. If the transaction rolls back, the event is never dispatched
Rationale	Assignment 2 Task 1 recommended a uniform Observer over a full dispatcher because a dispatcher looked like more infrastructure than a single entity raising events needed. That reasoning changes once Laravel is the confirmed stack: the event/listener mechanism already exists. What Assignment 2 Task 1 did not consider is Assignment 2 Task 2's constraint that the audit write must be inside the same transaction as the status update, so a uniform after-commit Observer would not fit. Splitting the reactions keeps D-008's atomicity guarantee intact.
Trade-offs	The status-change reaction logic is no longer handled by one uniform mechanism, which is one more thing to explain in the defence. A misconfigured listener still silently skips its reaction.

Field	Record
Risks	If a future reaction is added as a queued listener without checking whether it also needs to be inside the transaction, it could reintroduce a consistency gap.
Evidence	Assignment 2, Task 1, Section 2.1 (Design Problem 1). Assignment 2, Task 2, Sections 3.2 to 3.4.
Later consequence	Implemented as the RequestStatusChanged event and NotifyRequesterOfStatusChange listener. On Render it runs synchronously, and a mail failure is logged instead of undoing the status change. StatusNotificationTest. Consequence: no retry (TD-05

D-010: Category-specific handling

Field	Record
Status	New at M2
Context	D-001 fixed the category vocabulary itself, but not how category-specific validation and routing behaviour is implemented. Writing that behaviour as one long conditional means every new category requires editing and retesting the same method.
Constraints	Adding a category should not require editing already-tested code. CivicConnect's categories differ in behaviour, not only in the fields they require.
Alternatives	(a) Strategy pattern, one handler class per category behind a shared interface. (b) Factory Method producing category-specific request objects. (c) A configuration-only lookup of required fields.
Decision	Adopt (a), the Strategy pattern, as Assignment 2 recommended. Each category implements a shared CategoryHandler interface (rules(), defaultAssignee()). CategoryRegistry, bound in AppServiceProvider, resolves the handler for a given category code, and an unrecognised code falls back to OtherHandler. StoreServiceRequest merges the resolved handler's rules into its own FormRequest validation.
Rationale	The configuration-only approach was set aside because the categories differ in behaviour, not only in required fields. Factory Method was set aside because the categories differ in behaviour, not in data shape. A subclass per category would force a class hierarchy onto the persisted request and complicate the data model (Assignment 2, Task 1). With Strategy, adding a category means one new handler class, one seeded category row and one line in the CategoryRegistry mapping. Existing handlers and StoreServiceRequest are not edited.
Trade-offs	Requires a small registry/lookup mechanism. If the category list stays very small with simple, near-identical rules, this is more structure than the problem needs.
Risks	If the handler set drifts from the seeded category list, a request could silently resolve to OtherHandler (linked to Risk R-007).

Field	Record
Evidence	Assignment 2, Task 1, Design Problem 2. Section 13.4.2.
Later consequence	Implemented as CategoryHandler, CategoryRegistry and 7 handlers. Lost Property and Security Concern add their own behaviour. CategoryHandlingTest. A new category needs one handler class and a registry entry.

D-011: Status-to-notification integration mechanism

Field	Record
Status	New at M2
Context	The status-update component and the notification component must exchange information without a notification failure affecting the status change (Assignment 2 Task 3).
Constraints	A notification failure may not roll back a status change. No network boundary without justification. Free-tier hosting with no Redis instance available.
Alternatives	(a) Synchronous in-process call. (b) Asynchronous in-process event handled by a queued listener. (c) A separate notification service called over HTTP/REST.
Decision	Adopt (b), as Assignment 2 recommended: a queued listener on the RequestStatusChanged event, using the database queue driver so the queue lives in PostgreSQL.
Rationale	Option (a) lets a mail-provider failure roll back the transaction, the exact failure Task 3 identified. Option (c) adds a second deployable, a security boundary and a versioned contract for benefits, such as independent scaling and third-party consumers, that this project does not need. Option (b) satisfies both constraints inside the single deployable chosen in D-006.
Trade-offs	Requires a queue worker process in every deployed environment. Delivery is eventual rather than immediate.
Risks	R-009: a free-tier host without background-worker support would accept notifications that are never sent.
Evidence	Assignment 2, Task 3, Sections 4.2 and 4.3. Section 13.7.3.
Later consequence	Makes a persistent worker process a hosting requirement, which constrains D-013. The email provider is recorded in section 13.7 once a free-tier provider is confirmed.

D-012: Authentication and role-based access control

Field	Record
Status	New at M2
Context	FR-010, FR-011 and FR-018 require role-based access. NFR-002 requires authorisation on every read. Forward Engineering Consideration #1 warns that retrofitting access control forces rework of the data model and UI.
Constraints	Sensitive request categories (Forward Engineering Consideration #6). Credentials hashed at cost factor 12 (NFR-001). A single deployable with server-side rendering through Inertia (D-006).
Alternatives	(a) Laravel's session-based authentication with CSRF protection and framework policies. (b) API tokens held in browser storage. (c) An external identity provider.
Decision	Adopt (a): session cookie authentication with CSRF protection, and authorisation enforced by ServiceRequestPolicy at the data-access layer rather than per screen. Roles are stored on users.role.
Rationale	Because D-006 keeps one deployable and Inertia requests are ordinary authenticated requests, no cross-origin token exchange is needed, which removes the CORS and stateful-domain configuration a separate SPA would have required. Framework policies apply the same rule to every query path, which is what NFR-002 and ASR-002 require. An external provider adds cost and an external dependency for no stated requirement.
Trade-offs	Session authentication assumes browser clients. A future non-browser client would need token authentication added alongside.
Risks	The role model must match the data model, CR-009 fixed it at three roles. Policy coverage must be verified by test rather than assumed.
Evidence	FR-010, FR-011, FR-018, NFR-001, NFR-002. Section 13.5.1.
Later consequence	Determines the users.role column, the policy test set for M3, and the route-level role guards, which remain a usability convenience only.

D-013: Deployment direction and environment strategy

Field	Record
Status	New at M2, supersedes D-005. Closed by decision 16.
Context	A deployment direction is needed so the architecture and technology choices can be checked for plausibility, but provider selection depends on evidence the team does not yet have.

Field	Record
Constraints	Free or low-cost services (CON-003). One web process, one queue worker and one PostgreSQL instance (ASR-006). Environment separation (Master Brief section 17). A persistent worker is required by D-011.
Alternatives	(a) Commit to a named provider now. (b) Fix the topology and environment strategy and defer the provider. (c) Defer the whole question to M3.
Decision	Adopt (b). The topology is one Laravel deployable serving Inertia pages and compiled assets, one queue worker sharing that codebase, and a managed PostgreSQL instance. Development runs on Docker Compose and test runs in CI against a PostgreSQL service container. The specific provider is deferred.
Rationale	Because D-006 produces a single deployable, no separate static host is required, which widens the range of usable free tiers and shortens the deployment path. Fixing the topology is enough to size free-tier limits and validate the architecture, whereas naming a provider before verifying worker support and backups would be an unevidenced claim.
Trade-offs	Environment-specific configuration and secrets handling must be designed before the provider is known.
Risks	R-009 worker support on free tiers. R-013 idle pausing and storage caps. NFR-006 recovery depends on the provider's backup facility.
Evidence	Sections 13.7.1 to 13.7.5. Master Brief section 17. Free-tier limits recorded in section 13.7.4.
Later consequence	Provider selection remains open in section 13.8 and must be closed before the M3 staging deployment.

D-014: Construction deferred to Milestone 3

Field	Record
Status	New at M2
Context	Milestone 2 is a design baseline for this team, and construction is planned to begin in Milestone 3.
Constraints	The fixed M2 window (CON-002). PHP is new to all three members (CON-006). Milestone 2 brief section 7 expects development to have begun.
Alternatives	(a) Build a thin vertical slice during M2. (b) A throwaway prototype only. (c) Defer construction to M3 with the repository, CI and backlog prepared during M2.

Field	Record
Decision	Adopt (c): the design baseline, backlog and repository governance (protected main and two-reviewer approval) are prepared during M2. The repository folder structure, README, pull request template and CI workflow are created at the start of M3 together with the Laravel project, and application code begins in M3.
Rationale	Concentrating construction in M3 keeps the M2 evidence coherent and avoids partly-built code that contradicts the baseline. The team accepts that the evidence for the development criterion is correspondingly limited.
Trade-offs	Implementation and verification columns in the RTM read 'Planned - M3', and the end-to-end trace is complete only as far as the technology decision.
Risks	R-015: without a proof of concept, the stack and toolchain risks (R-001, R-014) remain unvalidated until M3.
Evidence	Milestone 2 brief sections 5.7, 7 and 12. RTM section 6.2 status values. Forward Engineering Consideration #7.
Later consequence	Sets the M3 build order: the first deliverable is a vertical slice covering FR-001, FR-017 and FR-019.

D-015: Status taxonomy and controlled transitions

Field	Record
Status	New at M2, closes Forward Engineering Consideration #2
Context	Forward Engineering Consideration #2 left the status vocabulary open, but FR-006, FR-017, FR-019 and the D-008 transaction all depend on a fixed set of states and legal transitions.
Constraints	Status updates limited to pre-defined options (FR-017). Transitions must be auditable (D-002) and enforceable in the database (D-008).
Alternatives	(a) Free-form status text. (b) A fixed enumeration checked in application code only. (c) A fixed enumeration plus a seeded transition table referenced by the history rows.
Decision	Adopt (c): the statuses are open, assigned, in_progress, resolved and closed, with legal transitions seeded in request_status_transitions and referenced by a composite foreign key from request_status_history.
Rationale	The service layer can return a clear message for an illegal transition while the database refuses it as a backstop, which is the layered validation Assignment 2 Task 2 recommended. A code-only enumeration leaves the database able to store states the business rules forbid.
Trade-offs	Adding or renaming a status requires a migration and a seeded row rather than a code change alone.

Field	Record
Risks	If the CHECK constraint and the transition table drift apart, a legal transition could be rejected or an illegal one accepted (Forward Engineering Consideration #3).
Evidence	Assignment 2, Task 2, Section 3.2. Forward Engineering Consideration #2. Change record CR-010.
Later consequence	Fixes the status options offered in the UI, the transition test set for M3, and the definitions of open and overdue used in reporting.

D-016: Database Provider selection

Field	Record
Status	New at M3
Context	A database provider was required to host the database.
Constraints	The database provider must be a free-tier service
Alternatives	Supabase
Decision	Make use of the provider NEON to host the database for staging and production.
Rationale	The free plan provides 1GB storage, hosts PostgreSQL 18 which we are using, and allows different branches for staging and production
Trade-offs	Suspends after 5 minutes of being idle, and a point-in-time restore on the free plan remains unclear.
Risks	R-002, R-013
Evidence	53 tests run and passed on the NEON staging database.
Later consequence	AWS London being the nearest causes a latency of 150 to 200 ms, meaning NFR-004 will be at risk. The free tier does have the limits of 100 computing hours and 1 GB storage per project., reducing scalability.

D-017: Application deployment environment

Field	Record
Status	New at M3
Context	The hosting provider for the application has been deferred. The database hosting provider has been decided on and tested.
Constraints	Free or low-cost services (CON-003). The M3 schedule (CON-002). The host must support a persistent queue worker and the scheduler (D-011, AS-001, R-009)
Alternatives	(a) Choose a host now and deploy the stating app during milestone 3. (b) Run the application locally for milestone 3, but connect to the staging database on Neon. (c) Run everything locally.
Decision	Adopt (b)
Rationale	The staging database has already been set up, but there is no decision yet for where the application will be hosted. Milestone 3 aim currently for a MVP. Decision (a) would divert attention away from the construction of the application itself, risking functionality. Decision (c) would leave the hosted database untested.
Trade-offs	No public staging URL for milestone 3, and hosting specific problems can occur during Milestone 4.
Risks	R-009 (worker support), R-010 (session/APP_URL configuration), R-013. AS-001 stays unverified until M4.
Evidence	Neon staging results as of section 13.2.6. Milestone 3 brief allows for a simulated staging environment.
Later consequence	OD-01's application-host part stays open until M4. OD-02 (email/SMS) is decided together with the host.

D-018: Chosen Application deployment environment

Field	Record
Status	New at M3, Supersedes D-017
Context	D-017 was originally deferred to milestone 4. However, due to the postponement of the Milestone 3 submission and milestone 4 not being extended as well, the hosting service for the application has been moved back to milestone 3.
Constraints	Free services (CON-003). The M3 schedule (CON-002). The app must reach the Neon database over SSL (D-016). D-011 expects a queue worker and a scheduler (AS-001, R-009)
Alternatives	(a) Render free web service. (b) AWS Free plan in London. (c) Koyeb free web service. (d) Oracle Cloud Always Free server.

Field	Record
Decision	Chosen option (a). The host for the application will be Render's free tier, built from a Docker image in the Frankfurt region. The hosted application shall connect to the database hosted on NEON.
Rationale	Render has a free-tier option and gives the team a public staging URL with the least setup. (b) runs a worker and a scheduler properly, but does not have free-tier service. (c) requires card information to make use of the service, and has a smaller CPU capacity. (d)needs the highest level of server administration among the alternatives while also having known capacity issues.
Trade-offs	The app sleeps after 15 minutes without traffic and takes around a minute to warm up again. There are no free background workers, meaning the email delivery will be done during the web request. It has a limited RAM capacity at 512MB, and the file system is reset on each deployment.
Risks	R-009 (worker support): mitigated by synchronous mail and GitHub Actions. R-010 (APP_URL/session). R-013. AS-001 is answered as not met on this host, and the mitigation replaces it. The GitHub Actions purge was planned but not set up (TD-08)." he GitHub Actions purge was planned but not set up (TD-08).
Evidence	Flavio Copes (Copes, 2026) compared 11 hosting providers' free tier services. Deployed 16/10/2026 to https://civicconnect-staging-kski.onrender.com. Health check /up returned 200, migrations reported nothing to migrate, and a status-change email was delivered.
Later consequence	Closes the application-host part of OD-01. If the project needs queued notifications or a host that is always on, the plan changes to Render's paid tier or option (b). OD-02 (email) has to use a provider that works with synchronous sending.

D-019: Emailing service

Field	Record
Status	New at M3. Closes OD-02.
Context	FR-007 and CR-011 require an email when a request's status changes.
Constraints	D-018 hosts the app on Render's free tier, which has blocked outbound SMTP ports since September 2025 (Render, 2025), making ordinary SMTP sending no longer possible
Alternatives	(a) Resend, through its HTTPS API. (b) Brevo, through its HTTPS API. (c) SMTP through a Gmail or Outlook mailbox. (d) Mailgun or Mailtrap.
Decision	Chosen (a). Laravel's built-in resend mail driver is used, sending emails from onboarding@resend.dev. On staging a code line was added (MAIL_DEMO_TO) in order to redirect all emails to a demo inbox, as the seeded users does not actually exist.

Field	Record
Rationale	Resend has a built-in Laravel driver, so no custom code is needed. It uses HTTPS, so Render's SMTP block does not affect it. Its sender address is authenticated by Resend, which lowers the junk-mail risk compared with Brevo sending as a campus address. (c) is blocked by Render. (d) needs a verified domain. The call is outbound and server-to-server, and the key is held in environment variables, so the browser never sees it. This is consistent with D-006 and D-012, because CivicConnect still exposes no API of its own.
Trade-offs	Without a verified domain, Resend only delivers to the account owner's address, so the demo can only email one inbox. Free limits are 100 emails per day and 3,000 per month.
Risks	R-009 (sending during the web request adds delay, and there is no automatic retry). AS-004 is answered: the free limit is enough for the demonstration.
Evidence	15/10/2026: a test email from the local application, addressed to the seeded user thandi.nkosi@example.com, was redirected by MAIL_DEMO_TO and delivered to the demo inbox.
Later consequence	Production will require a verified domain, and the code snipet (MAIL_DEMO_TO) must be removed

D-020: offer/decline rule

Field	Record
Status	New at M3. Answers the open point in FR-015 on what happens after a declined offer.
Context	FR-015 lets staff and management offer a request to a staff member, who accepts or declines it. FR-015 did not say what happens after a decline, so a request could keep waiting for someone to accept it.
Constraints	Render's free tier runs no worker or scheduler (D-018), so the rule must work at the moment someone answers, without timed jobs. Every offer, decline and acceptance must be recorded (FR-016).
Alternatives	(a) Allow 3 declines per calendar month. After that the next offer is accepted automatically. (b) Keep the offer pending but remove Decline once the limit is reached. (c) Unlimited declines, the request returns to the open pool after each decline. (d) Offers expire after a set time.
Decision	Chosen (a). A staff member can decline up to 3 offers per calendar month. After that their next offer is accepted automatically. A declined offer moves to the staff member with the fewest open requests (then lowest id), skipping anyone who already declined this request and the person who made the offer. If nobody is left, the request returns to the open pool. Take is blocked while an offer is pending. Staff and management can make offers, requestors cannot.
Rationale	An offer can never get stuck: it is accepted, passed to the next person, or returned to the open pool. No scheduled job is needed, which option (d) would require and Render's free tier cannot run (D-018). Passing the offer to the person with the fewest open requests spreads the work.
Trade-offs	An automatic accept assigns work the staff member never actively accepted. The count resets each calendar month, so 3 declines on 31 October and 3 more on 1 November are possible. The next person is chosen by workload, not by skill or category.
Risks	R-008 (two staff acting on the same request): the request and offer rows are locked while an answer is saved, and each decline locks the staff member's row so two declines at once cannot pass the limit. SQLite ignores these locks, so this is only protected on PostgreSQL (TD-03).
Evidence	Implemented in OfferService and RequestAssignmentController, merged in PR #24 (ee33a95) and to main in PR #25 (288849f). tests/Feature/OfferTest.php (15 tests) covers the decline limit at 2 and 3 declines, automatic accept, declines from last month not counting, passing the offer to the next person, the return to the open pool, Take blocked during an offer, and that only the offered person can answer.
Later consequence	FR-015's acceptance criteria are extended through CR-032. A later version could choose the next person by category or skill instead of workload.

D-021: Final release decision

Field	Record
Status	New at M4.
Context	The Final brief requires a declared final deployment state: production, simulated production, or staging under a justified no-go.
Constraints	Free-tier only (CON-003). Render free tier has no worker or scheduler (D-018). Resend delivers to one inbox without a verified domain (D-019).
Alternatives	(a) Release to real production. (b) Simulated production on a second Render service and the Neon production branch. (c) Keep staging as the final deployment state and accept the MVP conditionally.
Decision	Chosen (c). Release v4.0 is accepted as a working MVP on staging. Real production is a no-go until the conditions in Section 17.4 are met.
Rationale	The core lifecycle works and is tested end to end. But real users would not receive status emails (L-01), personal data would never be purged (L-02, POPIA s14), and backups could not restore lost data within 48 hours (L-04). Option (b) would look like production while carrying the same defects, so it would not change the decision.
Trade-offs	The product is not used by real users at the deadline. In exchange, the team does not release functionality it knows is unsafe or incomplete.
Risks	R-009 (occurred), R-013, TD-05, TD-08.
Evidence	Section 14.10 (CONDITIONALLY READY), Section 17.4, TC-E2E-01 to 03 on staging.
Later consequence	Production needs a verified email domain, a scheduler or worker host, and backups.

D-022: Role Assignment

Field	Record
Status	New at M4. Closes OD-10.
Context	FR-010 requires every user to hold exactly one role. OD-10 asked how accounts and roles are created.
Constraints	FR-010 (exactly one role per user), least privilege, no separate Administrator role in scope (CR-009).

Field	Record
Alternatives	(a) In-app admin screen for Management to assign roles. (b) Self-registration as Requester only. Staff and Management roles assigned directly in the database by an administrator.
Decision	Chosen (b). Anyone can register, always as a Requester. A database administrator promotes Staff and Management with an UPDATE on users.role in the Neon SQL Editor.
Rationale	Least privilege: the application offers no path to raise a role. role is excluded from the User model's mass-assignable fields, so a tampered registration or profile request cannot set it, and the column defaults to requestor. No admin screen means no admin attack surface.
Trade-offs	Role changes need database access and are not recorded in an audit trail. Security depends on keeping Neon credentials to a small number of people.
Risk	R-003 (reduced: no in-app path to raise a role). Unaudited role changes (L-05) and exposure of Neon credentials.
Evidence	User.php ($fillable), CreateNewUser.php, role default in the add_role migration, RegistrationTest. Procedure verified on staging, on 16/10/2026.
Later consequence	An audited role-management screen for Management, restricted by policy, if the organisation grows.

10. GitHub & Team Governance

10.1 Repository

Control	Status / Evidence Location
Repository URL	https://github.com/Destroyer1819/SEN381_Project
Main branch protected	Yes
PR required for substantive changes	Yes

Control	Status / Evidence Location
Minimum two-reviewer approval configured	Yes
Self-approval disabled	Yes
Secrets and credentials policy	.gitignore excludes environment files. No credentials are committed. Environment values are documented in .env.example. CI secrets will be held in GitHub Actions secrets once the CI workflow exists.
Issue board / backlog in use	https://github.com/users/Destroyer1819/projects/2
Pull request template	planned
Continuous integration workflow	In place: .github/workflows/tests.yml
Required status checks on main	Not configured, merges wait for a green CI run by team rule

10.2 Team Working Agreement

Roles: All three members contribute across requirements, engineering artefacts and repository work. No member is confined to a single artefact type. Current section ownership: Jared - Sections 1, 2 and 3 (Problem/Business Need, Stakeholder Analysis, Scope Baseline), Michael - Sections 4 & 6 (Requirements, RTM), Xander - Sections 5, 7, 8, 9, and 10 (Constraints, Risk Register, Forward Engineering Considerations, Decision Log, GitHub Governance). Ownership may shift for M2 to M4 as the work changes.

Communication: The team will check in at least twice a week (in person or via chat) and immediately before each milestone deadline. Blockers are raised as soon as identified rather than at the next scheduled check-in.

Commit & PR conventions: Commits use short, meaningful messages describing the change (e.g. "Add NFR-004 performance requirement"). Substantive changes to controlled artefacts (PED sections, registers, code) go through a Pull Request. Direct pushes to main are not used for substantive change, per the Master Project Brief section 9.

Review: Every PR requires review from the two team members who did not author it before merging. Reviewers check alignment with requirements/acceptance criteria, consistency with other PED sections, and flag anything that looks rubber-stamped rather than genuinely reviewed.

Conflict resolution: Disagreements on content or approach are raised in the team chat first. If unresolved within 24 hours, the team discusses it together (in person or on a call) rather than letting it stall a PR. The lecturer is consulted only if the team genuinely cannot reach agreement.

AI use: Any material AI-assisted contribution must be logged in the AI Usage Register (Section 11) by the student who used it, including what was verified, changed or rejected, before it is merged into a controlled artefact.

10.3 Repository structure and continuous integration

The repository is organised so that documentation, decisions and code sit beside each other and a single pull request can carry a change to all three.

Path	Contents
docs/PED/	The Project Engineering Document
docs/decisions/	One file per decision record, D-001 onward
docs/requirements/, risk/, change/	Requirement notes, risk working papers, change request forms
docs/architecture/, deployment/	Diagrams and environment notes
app/, database/, resources/js/	Laravel application code, migrations and React components (from Milestone 3)
tests/	Feature and unit tests (from Milestone 3)
.github/workflows/	Continuous integration workflow
.github/pull_request_template.md	Pull request template

This structure will be created at the start of Milestone 3 together with the Laravel project (D-014).

11. AI Usage Register

Date	Student	Tool	Engineering Task	AI Contribution	Verification	Decision	Issues Found
08/09/2026	Michael	Claude	Finding sources	Provided a list of sources	Read the sources	Accepted	None
08/09/2026	Michael	Claude	Understanding the project	Provided explanations	See if the explanation makes sense, and check with other online sources	Partially accepted	Not all of the explanations were relevant to the project
07/09/2026	Xander	Claude	Drafting M1 PED template structure (all section	Generated the section structure, table layouts and instructional notes	Checked every table's required fields against the Milestone 1 Brief section 3 and Master Brief sections 9-13 field-by-field	Accepted (structure only, no project	None. structure only, no factual claims to verify

Date	Student	Tool	Engineering Task	AI Contribution	Verification	Decision	Issues Found
			headings, tables, guidance notes)	matching the M1 brief's required outputs		content yet)	
08/09/2026	Xander	Claude	Drafting Sections 5 (Constraints), 7 (Risk Register), 8 (Forward Engineering Considerations) and 9 (Decision Log) content	Drafted CivicConnect-specific constraint implications, 7 project risks, 6 forward-engineering concerns and 5 decision-log entries	Cross-checked every claim against the CivicConnect Master Project Brief and Milestone 1 Brief sections cited inline (e.g. section 2, 3, 9, 18.1), reviewed with Michael before merging	Partially accepted	Owner names were placeholders ([Name]) and needed real assignment. Some risk/decision wording needed personalising against the team's actual scope once agreed
09/09/2026	Xander	Claude	Reviewing merged team document (Michael's Sections 4 & 6) for consistency and completing AI Usage Register / Risk Register owners / Team Working Agreement	Identified that AI Usage Register did not yet log Xander's own AI-assisted sections. Drafted Section 10.2 Team Working Agreement content. Proposed a draft Risk Register owner distribution. Aligned FR-001 wording between Section 4 and the RTM	Reviewed against the Master Project Brief's Responsible AI standard (section 10) requiring every material AI contribution to be logged and verified. Owner assignments and working-agreement content flagged to team as draft, pending confirmation	Accepted, pending team confirmation of owner assignments	None found, flagged as draft/pending team sign-off rather than a factual error
04/09/2026	Michael	Claude	Linking of multiple sections, such as the ASR's to the influence of the ASR's on the stakeholders.	Found potential links and provided explanations	Read through the explanations, and verified if the links found are valid	Partially accepted	Some potential links included links with little in common to what they are linked with.
24/09/2026	Jared	Claude	Structuring the Milestone 2 work: what evolves in the PED and where each M2 deliverable belongs	Produced a section-by-section plan mapping M2 brief and Master Brief requirements to PED sections	Checked each mapping against the Milestone 2 brief sections 4, 5 and 12 and the Master Brief section 20.2 list	Accepted with changes	The first version proposed new sections for everything. The team required the M1 baseline to be extended instead

Date	Student	Tool	Engineering Task	AI Contribution	Verification	Decision	Issues Found
25/09/2026	Jared	Claude	Technology selection research and the weighted matrix structure	Listed common web bundles, proposed criteria and weights traced to constraints and ASRs, and produced illustrative scores	Team scored every cell itself and rejected the initial weighting. Free-tier and licensing claims checked against the providers' own pricing pages	Partially accepted	Early scoring treated capability as one criterion and had to be split into language familiarity and framework learning. A lock-in score was inconsistent and was corrected
26/09/2026	Xander	Claude	Drafting decision records D-007 and D-010 to D-015, and restructuring Section 9	Drafted the missing decision records in the Master Brief section 13 field structure and reformatted the log one decision per table	Each record checked against the Assignment 2 task it cites and against the design in Section 13. D-006 and D-007 rewritten by the team after the Inertia decision	Accepted with changes	The first draft recorded the SPA architecture, which contradicted the matrix result and was corrected
26/09/2026	Xander	Claude	Raising risks R-008 to R-016 and reviewing R-001 to R-007	Proposed new risks traced to the M2 decisions, and Milestone 2 review notes for the existing risks	Team confirmed or changed every probability, impact and owner. R-010 reworded because Inertia removed the CORS exposure	Accepted with changes	Suggested owners and ratings were placeholders and were reassigned by the team
26/09/2026	Jared	Claude	Consistency review of the merged document	Identified contradictions and missing cross-references between Sections 6, 8, 9, 11, 12 and 13	Every reported issue checked in the document before acting on it	Partially accepted	Some reported items were formatting artefacts of the conversion rather than real defects
04/10/2026	Michael	Claude	Database hosting research	Compared free PostgreSQL hosts	Checked providers' docs and pricing pages to see if it conforms to the constraints	Accepted (Neon, London)	None
05-06 /10/2026	Michael	Claude	Database Schema review	Reviewed types, checks, foreign keys and delete rules against the ERD	Reviewed pgAdmin backups after each change	Accepted	Some relational and datatype errors in the database found during review, and resolved.
07-10/ 10/2026	Xander	Claude	Frontend pages, components, Vitest and Playwright tests, frontend CI	Generated the first version of the React pages and tests	Vitest, ESLint, Prettier and build. E2E later run on staging	Accepted with changes	E2E tests used accounts the seeders don't create and picked a category before the form loaded (DF-06, DF-07). Fixed in 832b8ec

Date	Student	Tool	Engineering Task	AI Contribution	Verification	Decision	Issues Found
10/10/2026	Michael	Claude	Schema check	Wrote 53 tests	Requested explanations behind the logic of each test, and compared this to the tests and their outcomes	Accepted	None
10/10/2026	Michael	Claude	Dummy data generation	Created the dummy data	Schema enforcement should have caught illegal statements. Checked if statuses are valid between the requests and the change history.	Partially accepted	Password hashing was initially incorrect, as it was hashed with a bcrypt cost factor of 10 instead of the required 12. New passwords were generated.
11/10/2026	Jared	Claude	Learning PHP	Explained how to code in PHP	Used the explanation to code the application. Code worked	Accepted	None
15/10/2026	Michael	Claude	Research free tier hosting services	Found sources for comparing different free tier hosting services	Personal inspection of the sources and manual comparison between services.	Accepted	None
15/10/2026	Michael	Claude	Research free tier emailing services	Found sources for comparing different free tier emailing services that will work with the hosting services	Personal inspection of the sources and manual comparison between services. Did a test run, and a test email was successfully sent.	Accepted	None
15/10/2026	Jared	Claude	Fixes for Xanders branch	Fixed an issue where Xanders branch was outdated	Had to make sure what it deleted was unused UI files	Accepted	None
16/10/2026	Jared	Claude	Test case creation	Drafted ideas for tests	Manual inspection of tests	Accepted	None
16/10/2026	Xander	Claude	Creation of the sort control, average resolution card, and 403 and 404 screens.	Wrote drafts for these tasks	Manual inspection of the created sort control, resolution card, and the 403/404 screens.	Accepted	None so far.
16/10/2026	Michael	Claude	Sections 14-16	Found the links between sections	Inspected each link and verified if these links are valid and sensible	Accepted	None.

Date	Student	Tool	Engineering Task	AI Contribution	Verification	Decision	Issues Found
18/10/2026	Michael	Claude	M4 finalisation: gap analysis against the Final brief, scope table, RTM 6.4, Section 17 drafts, consistency review	Compared the PED and repository with the brief and drafted text	Every claim checked against the repository, CI runs and staging. Walkthrough done by hand.	Accepted with edits	Release commit corrected from Development to main after checking CI history. Role assignment reframed as a security decision (D-022)

12. Baseline Readiness Checklist

Milestone 1 Readiness Checklist

- PED v1.0 is complete, coherent, versioned and reviewed
- Scope and requirements are baselined rather than merely drafted
- RTM, Risk Register, Decision Log, Forward Engineering Considerations and AI Usage Register are current
- GitHub governance required by the Master Project Brief is demonstrable
- Team Working Agreement is agreed and controlled
- Baseline sign-off/gate evidence is ready
- All students can independently locate and explain the project foundation and its Software Engineering significance
- All students are prepared to present professionally and defend the team's evidence individually

Milestone 2 Readiness Checklist

- PED v2.0 visibly continues M1 and retains traceable history - Met: document control, change record CR-001 to CR-019.
- RTM has evolved with architecture/data/design/interface/technology/implementation/verification/status evidence - Met: Section 6.2 (implementation and verification Planned M3 under D-014).
- Risk Register, assumptions/dependencies and Forward Engineering Considerations reflect new evidence - Met: Sections 7, 5.2 and 8.
- ASRs/quality drivers are project-specific and linked to architecture decisions - Met: Section 4.3.
- Architecture selection and diagrams are defensible and proportionate - Met: Section 13.1, D-006, CR-017.
- Data/persistence decisions are documented and linked to correctness/quality needs - Met: Section 13.2, D-008, D-015.
- Technology stack is justified with evidence rather than preference - Met: Section 13.3, D-007.
- At least two genuine design problems have final pattern/approach decisions informed by A2 research - Met: Section 13.4, D-009, D-010.
- A2 research is referenced as supporting evidence, not copied - Met: D-008 to D-011.

- Initial interface/integration decisions are documented where applicable - Met: Section 13.6.
- Architecture, Technology & Initial Design Baseline is identifiable and controlled - Not recorded: no Milestone 2 approval outcome was issued (see Baseline Approval)
- Meaningful development has begun against the baseline - Deferred to Milestone 3 by D-014.
- Application/technical documentation matches the repository/application - Deferred with D-014.
- At least one requirement traces into implementation and verification evidence - Trace complete to the technology decision (D-007). Implementation and verification planned in M3 (D-014).
- GitHub history demonstrates progressive controlled work and peer review - Met for documentation: pull requests with two approvals.
- Relevant A2 SCM/CI recommendations are adopted progressively - Partly met: protected main and two-reviewer approval adopted. PR template and CI at the start of M3.
- AI use is recorded and verified - Met: Section 11.
- Every team member can defend the shared evidence – confirmed

Final readiness checklist

Check	Status/Evidence
One final release identified	v4.0, commit 288849f
Every M1 in-scope commitment has a final status	Section 17.2
Scope Completion Percentage calculated	Section 17.3: 87.9%
Final deployment state declared and justified	Section 17.4, D-021
M3 defects, risks, debt and open decisions closed	Section 17.5, 13.8, 15
RTM updated, not recreated	Section 6.4

Check	Status/Evidence
Registers concluded	Sections 7, 9, 11, 15, 16
Limitations and future work stated	Section 17.6

13. Solution Design Baseline

13.1 Architecture

13.1.1 Logical components

Component	Responsibility
Submission	Accepting, validating, and providing persistence to the new requests that are being made.
StatusAudit	Status changes, updates and appending the records of who made a change, what change was made, and when was the change made.
Assignment	Responsible for the ownership of a request, be it assigning a request, offering a request to another staff member, or accepting an offered request.
RequestQuery	Reading and then returning of data. Includes search functions, sorting, filtering, listing and pagination of data.
Authentication	Verifies who the user is (session login) and which of the three roles they hold (D-012).
Category handling	Category specific validation and routing rules.
Notification	Delivering notifications about the status changes to the requesters.

Component	Responsibility
Retention	Enforcement of the 5-year retention before permanent deletion rules.
Authorisation	ServiceRequestPolicy: decides which requests each role may see and change, applied on every query path.
Reporting	On-demand management reports: counts by status and category, the overdue list and average time to resolution.

13.1.2 Considered alternatives

Option	Description	ASRs supported	Risks / weaknesses	Decision
A. Server-rendered monolith (Blade)	Pages are rendered by Laravel Blade views and the browser submits ordinary forms. No client-side application.	ASR-001, ASR-002 (authorisation stays on the server), ASR-006 (single deployable).	Leaves the team's React and TypeScript experience unused. Less interactive filtering and status updates.	Rejected - kept as fallback B-003 (R-001)
B. Layered modular monolith with Inertia-rendered React	Laravel controllers return Inertia responses and React + TypeScript components render the pages. One deployable, session authentication, no JSON API.	ASR-002 (policies on every query path), ASR-003 (D-008 transaction in one process), ASR-004 (business rules in services), ASR-006 (one web process, one worker, one database).	Front-end and back-end are coupled and released together. Page props are not a reusable API (R-016).	Selected (CR-017)
C. Layered modular monolith with a separate React SPA over a Laravel JSON API	A React single-page application calls a Laravel API using token or cookie-based authentication.	ASR-002, ASR-003, ASR-004.	Adds a network boundary, CORS and token handling, a second deployable and duplicated validation for a client the project does not need to serve independently (Assignment 2, Task 3).	First selected, superseded by CR-017
D. Distributed services	Separate deployables for requests, notifications, reporting and identity.	None better supported than option B.	The status change and audit write become a distributed transaction (ASR-	Rejected

Option	Description	ASRs supported	Risks / weaknesses	Decision
			002, ASR-003). Several deployables exceed free-tier limits (ASR-006).	

13.1.3 Justification of selected architecture

CivicConnect adopts option B: a layered modular monolith in which Laravel controllers return Inertia responses and React with TypeScript renders the pages. Option C, a separate React single-page application over a Laravel JSON API, was the first selection and was superseded during Milestone 2 by CR-017.

Accountability is the first reason. Weak accountability for status changes is a core problem in the business need. D-002 makes the status history append-only, and D-008 writes the status change and its history row in one database transaction. That guarantee depends on both writes happening in one process against one database. Option D would turn it into a distributed transaction.

Access control is the second reason. FR-013 lets staff combine search, filter and sort freely, so there is no single screen at which permissions can be checked. In option B every page visit is an ordinary authenticated request to Laravel, and ServiceRequestPolicy applies the same rule on every query path (D-012, ASR-002). Option C would need the same rules exposed through an API, with authentication carried across a network boundary using CORS and token handling.

Integrity is the third reason. A stale write is rejected by the optimistic version check in D-008 instead of silently overwriting another staff member's change (R-008).

Cost rules out option D and weighs against option C. The cost constraint (CON-003) favours the fewest deployables, and option B runs as one web process, one queue worker and one PostgreSQL instance (D-013). A more distributed system is not automatically more advanced, and no requirement justifies the extra complexity.

The trade-off accepted with option B is coupling: the React pages and the Laravel application are released together, and page props are an internal contract rather than a reusable API. No requirement in the approved scope needs a second client, so this is accepted and tracked as R-016.

13.1.4 Physical deployment

The components in 13.1.1 are logical boundaries in the source tree, not separate processes. CivicConnect runs as three physical units:

Laravel web process: runs all the logical components in 13.1.1, handles every page visit and returns Inertia responses together with the React and TypeScript assets compiled by Vite.

Queue worker: runs php artisan queue:work from the same codebase and executes the queued NotifyRequesterOfStatusChange listener (D-011). Laravel's scheduler, which runs the retention purge (FR-008), also runs from this codebase (13.7.3). PostgreSQL database: stores all application data and the job queue (database queue driver). It is a single point of failure, accepted at MVP under the cost constraint and mitigated by backups (13.7.5).

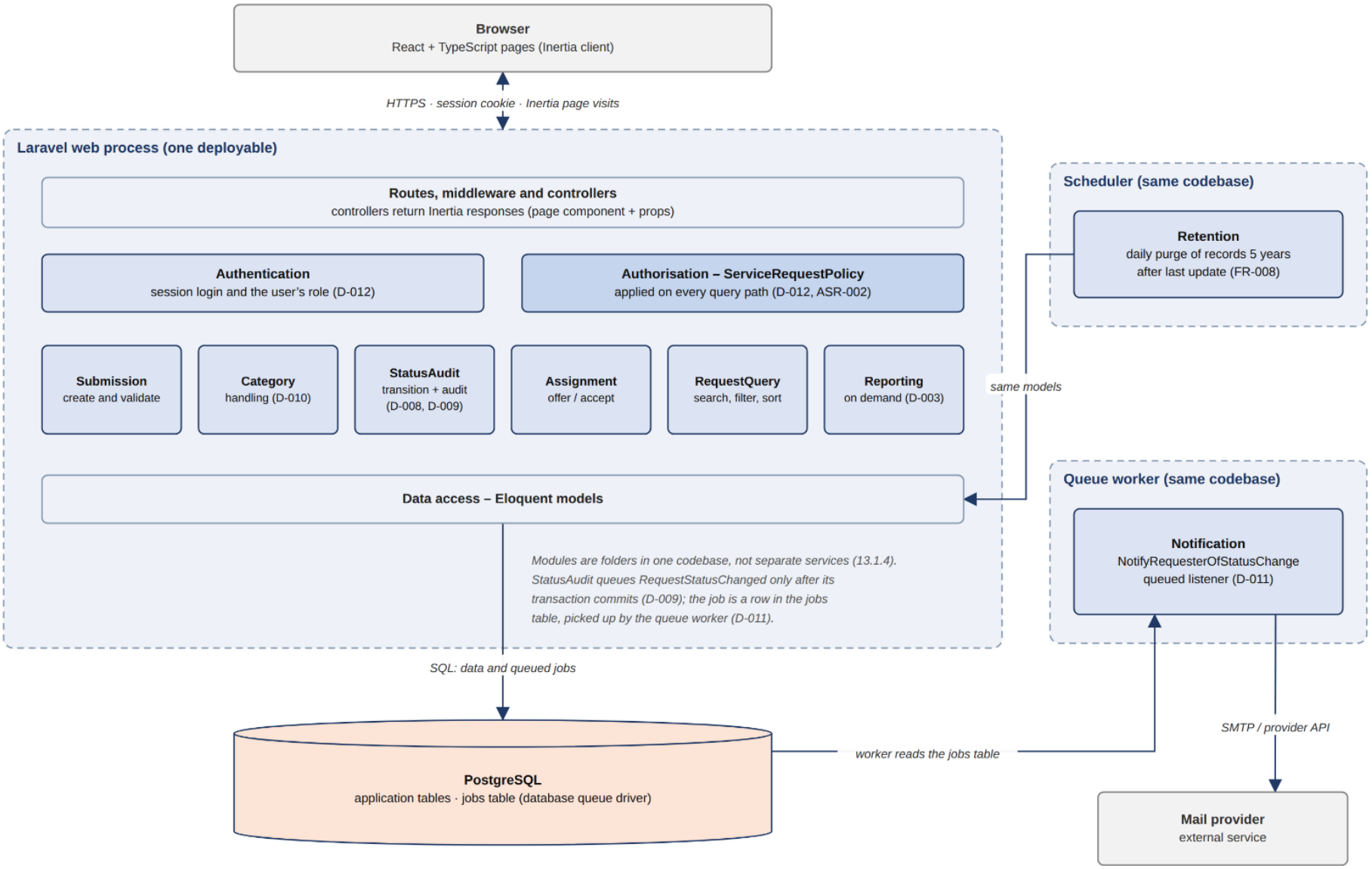
Final state (M4): the deployed system is one Render free-tier web service built from the Dockerfile in the Frankfurt region, with the queue running synchronously and no queue worker or scheduler (D-018), connected to the Neon staging branch (D-016) and sending mail through Resend over HTTPS (D-019). The queue worker and scheduler described in this section and shown in Figure 13.1 are the target design. Their absence is recorded in R-009, TD-05 and TD-08.

13.1.5 Architectural diagrams

shows the logical components inside the single Laravel deployable, the queue worker and scheduler that share its codebase, and the external mail provider.

Figure 13.1: CivicConnect logical architecture

One Laravel deployable serving Inertia pages · one queue worker · one PostgreSQL instance (D-006, D-013)



13.2 Data and persistence baseline

Intro

This section is the design-level explanation, and identifies the core data entities CivicConnect needs at this stage, sets out the initial data model, and commits the persistence approach that keeps a status change and its audit entry consistent. The full decision record is D-008 in Section 9.

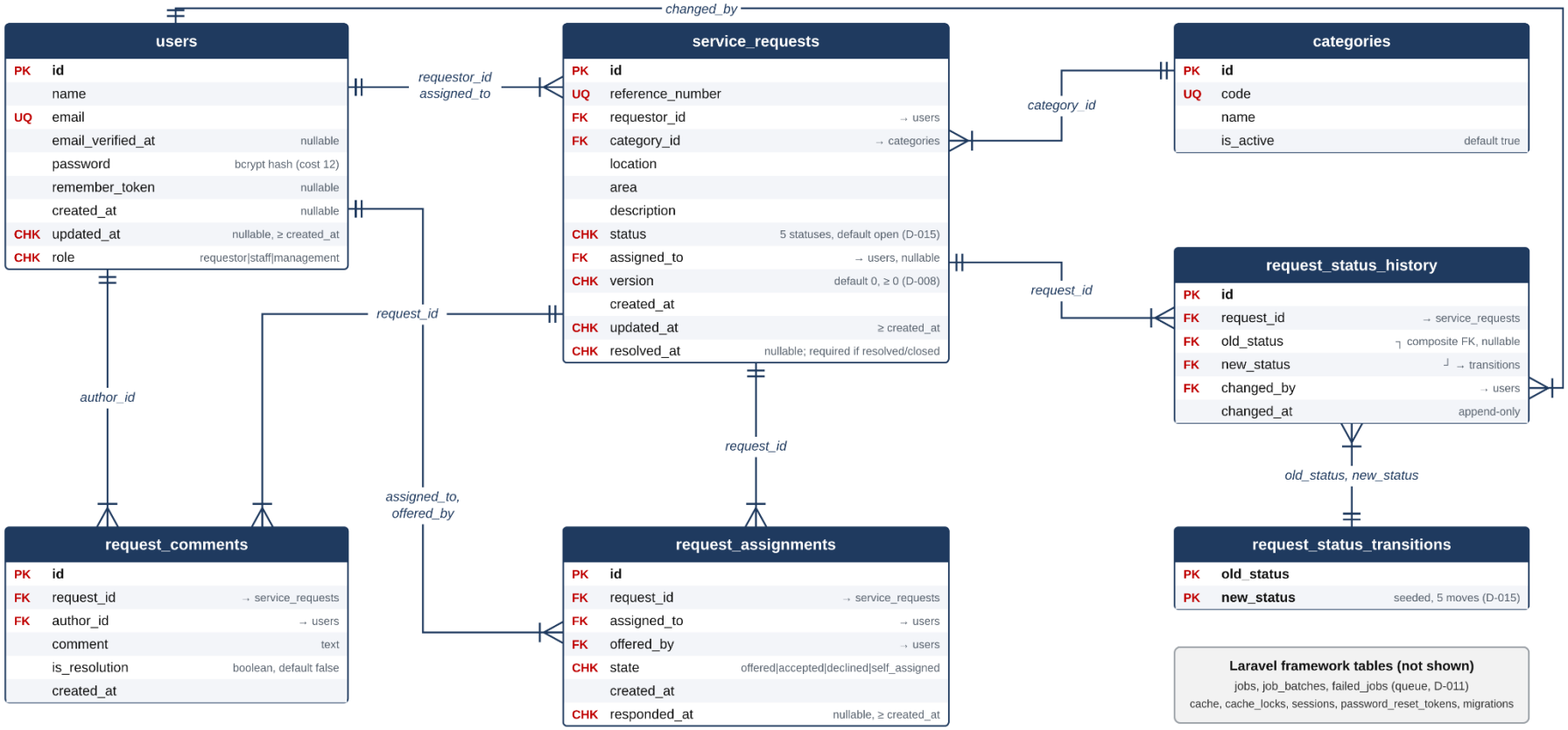
13.2.1 Core data entities

Table	Purpose	Traces back to
users	Account data for every user: name, email, password hash and one of three roles.	FR-010, NFR-001, NFR-002
categories	Seeded, controlled list of request categories, including 'Other'.	FR-005, D-001, D-010
service_requests	The root entity: requester, category, location, area, description, current status, assignee and version.	FR-001, FR-006, FR-023, D-008
request_status_transitions	Seeded list of legal old-status / new-status pairs.	FR-017, D-015
request_status_history	Append-only record of every status change: request, previous and new status, who and when.	FR-014, FR-019, NFR-003, D-002
request_assignments	Offers, acceptances and self-assignments, with who and when.	FR-015, FR-016
request_comments	Staff comments and resolution notes, with author and time.	FR-014, FR-020
jobs, failed_jobs, Cache, cache_locks, job_batches, sessions, migrations, password_reset_tokens	Laravel database queue tables for queued notifications.	D-011

13.2.2 ERD

Figure 13.2 shows the initial schema. Table ownership and lifecycle are in 13.2.3

Figure 13.2: CivicConnect entity relationship diagram (as built by the migrations, M3)



Every status change writes service_requests.status/version and one request_status_history row in a single transaction (D-008).

All foreign keys are ON DELETE RESTRICT. A self_assigned assignment must have assigned_to = offered_by. A history row with old_status NULL must have new_status = open.

13.2.3 Data Ownership and Lifecycle

Table	Owning component	Written by
users	Authentication	Account creation (OD-10) and profile updates.
categories	Category handling	Seeded by migration, changed only through a migration and a change record.
service_requests	Submission (create), StatusAudit (status), Assignment (assignee)	Created by the submission service. Status changed only by RequestStatusService (D-008, R-011). Category changed only by an audited staff reclassification.
request_status_transitions	StatusAudit	Seeded by migration (D-015).
request_status_history	StatusAudit	Inserted inside the D-008 transaction. Never updated. Purged 5 years after the request's last update.
request_assignments	Assignment	Written on offer, accept, decline and self-assignment.
request_comments	StatusAudit	Written when staff add a comment or resolution note.
jobs, failed_jobs	Notification	Written by the framework when the listener is queued or fails.

13.2.4 Persistence model choice

The database that will be used is a relational model in PostgreSQL. The evidence for this decision is as follows:

Factor	Evidence	Implication
Structure	Users, categories, requests, assignments, comments and status history are separate entities linked by foreign keys, with one-to-many relationships from each request (13.2.1).	A relational model represents these relationships directly. A document store would duplicate user and category data inside every request.
Integrity	Status values and legal transitions must be enforced (FR-017, D-015), and every history row must point to a real request and user (FR-019).	PostgreSQL CHECK constraints, foreign keys and a composite foreign key to request_status_transitions act as a backstop behind service-layer validation.
Consistency	A status change and its audit row must succeed or fail together (D-002, D-008), and concurrent edits must not overwrite each other (R-008).	ACID transactions (DB::transaction) plus a version column for optimistic concurrency. No eventual consistency is needed at MVP.
Access patterns	Staff and management search, filter, sort and paginate lists (FR-009, FR-013). Overdue is derived from timestamps (FR-023).	Indexes on status, category_id, assigned_to and updated_at. Pagination at 50 rows per page (NFR-004, ASR-001). Final status: the indexes on status and updated_at were not created (TD-04).
Sensitivity	Security and IT-support requests may contain personal or safety-related information (CON-005, FEC 6).	Authorisation is applied at the data-access layer (D-012). Passwords are hashed (NFR-001). Encryption at rest depends on the provider (OD-05).
Growth	Records are kept 5 years after the last update (FR-008) on a free tier with a storage cap (CON-003, ASR-006).	Rows are small text records. A 5-year volume estimate is checked against the provider's cap before OD-01 closes (AS-003), and the purge job removes expired rows.

13.2.5 SPOF, scalability and bottlenecks

The single PostgreSQL instance is a single point of failure. This is accepted at MVP under the cost constraint (CON-003) and mitigated by the provider's backup and recovery facilities (13.7.5).

Horizontal scaling is not required at this stage. CivicConnect serves a single organisation, load grows gradually rather than in spikes, the architecture is a single deployable (D-006), and additional instances would exceed the free-tier constraint. NFR-004 is a response-time target and is met through indexing and pagination (13.2.4), not additional servers. If growth erodes that target, vertical scaling is the first response. Horizontal scaling becomes necessary only if load exceeds what one server can handle, or if the NFR-005 availability target requires removing the single point of failure.

Three bottlenecks are expected. The first is the database connection limit: free-tier PostgreSQL caps concurrent connections (CON-003), and every page request, the queue worker and each D-008 transaction hold one, so transactions are kept short. The second is reporting load: FR-024 aggregates over request_status_history, the fastest-growing table, on the same instance that serves day-to-day work. On-demand reporting (D-003) and indexes limit this. The third is the retention purge (FR-008): deleting a large batch of expired rows at once can lock tables, so the purge runs off-peak in batches.

13.2.6 Milestone 3 Update: schema prototype and verification.

Before the Laravel migrations were written the schema was built into a local database in PostgreSQL as a prototype. This was for the purpose of verifying the integrity rules before being carried over to the migrations, avoiding the need to replace the migrations if issues are found.

Additional rules were added beyond what figure 13.2, the ERD, shows. This is done as the ERD would have been difficult to read if these additional rules were added to the ERD itself.

Furthermore, there were 53 database-level checks on the tables and their relationships. They were run on 10/10/2026 and all passed the checks. The results are traced back to the RTM in section 6.3. The tests were re-run on 11/10/2026 and 12/10/2026 to ensure the database on NEON works as well.

Test data has been created in the form of Laravel seeders and migrations, using completely fictional users to mitigate the damage if the database is breached. Passwords are hashed using bcrypt at a cost factor of 12.. Test data includes all categories specified in D-001. Each status types is also covered in the test data. This data will be used for local development and will be used for the NFR-004 timing checks as well.

13.3 Tech Stack Analysis

13.3.1 Common web bundles

Bundle ID	Bundle	Frontend	Backend	Database	Source
B-001	MERN / PERN	React (JS/TS)	Node + Express	MongoDB (MERN) / PostgreSQL (PERN)	(Express, 2026)
B-002	Next.js full-stack	React via Next.js	Next.js route handlers / server actions (Node), usually Prisma	PostgreSQL	(Next.js, 2026)

Bundle ID	Bundle	Frontend	Backend	Database	Source
B-003	LAMP / LEMP	Server-rendered PHP views (Blade) + light JS	PHP on Apache or Nginx, typically Laravel	MySQL / MariaDB (PostgreSQL also supported)	(Laravel, 2026a)
B-004	Laravel API + SPA	React or Vue SPA (TypeScript)	Laravel JSON API (PHP)	PostgreSQL or MySQL	(Laravel, 2026b)
B-005	Django	Django templates, or React with Django REST Framework	Django (Python)	PostgreSQL	(Django Software Foundation, 2026)
B-006	ASP.NET Core	Razor Pages, Blazor, or React	ASP.NET Core (C#) + Entity Framework	SQL Server (PostgreSQL supported)	(Microsoft, 2026)
B-007	Spring Boot	Angular or React	Spring Boot (Java) + JPA/Hibernate	PostgreSQL or MySQL	(Spring, 2026)
B-008	Rails	ERB views with Hotwire/Turbo, or React	Ruby on Rails	PostgreSQL	(Ruby on Rails, 2026)
B-009	Laravel + Inertia	React + TypeScript via Inertia	Laravel (PHP)	PostgreSQL	(Inertia.js, 2026) (Laravel, 2026c)

The bundles were selected from the most widely used web frameworks among developers (Statista, 2026)

PostgreSQL capabilities are taken from the PostgreSQL Global Development Group (2026).

13.3.2 Candidates not recommended

MERN was excluded before scoring because MongoDB is not a relational database, and the data design (D-008, D-015) depends on transactions, foreign keys and CHECK constraints. Its relational variant, PERN (PostgreSQL), was kept and scored as B-001

13.3.3 Weight matrix

Scale: 1 = poor fit or high risk, 2 = weak, 3 = adequate with effort, 4 = good, 5 = strong, with evidence. Weights total 100. Maximum weighted score 500.

Criteria	Weighting	Options								
		B-001	B-002	B-003	B-004	B-005	B-006	B-007	B-008	B-009
Requirement & ASR fit	15	4	3	5	5	5	5	5	5	5
Team capability	8	3	3	2	2	4	4	4	2	2
Framework & surface learning	12	2	3	3	2	3	3	1	2	3
Dev environment availability	10	5	5	4	4	4	5	5	3	4
Security ecosystem	12	3	3	5	4	5	5	5	5	5
Maintainability	10	4	3	4	4	4	4	4	4	4
Testing & tooling	10	4	3	5	4	4	4	4	5	5
Deployment & ops	10	4	2	5	3	4	3	3	4	5
Cost	5	4	4	5	4	5	3	3	5	5
Lock-in	4	4	2	3	5	3	3	5	3	4
Fallback consequence	4	4	2	3	4	3	3	2	3	4
Weighted total (/500)		366	307	416	369	412	402	382	384	424
Percentage (%)		73	61	83	74	82	80	76	77	85

13.3.4 Weight matrix for database

Criteria	Weight	PostgreSQL	MySQL / MariaDB	SQL Server
Integrity support: CHECK constraints, composite foreign keys, transactional DDL	30	5	3	4
Team capability	20	5	4	2

Criteria	Weight	PostgreSQL	MySQL / MariaDB	SQL Server
Managed free-tier availability	20	4	4	2
Migrations and tooling support in the chosen stack	15	5	4	4
Cost and lock-in	15	5	5	3
Weighted total (max 500)		480	385	305
Percentage		96	77	61

13.3.5 Results of the matrices

B-009 (85%), B-003 (83%), B-005 (82%) and B-006 (80%) fall within five percentage points of one another. The matrix therefore narrows the field to four defensible candidates rather than producing a single winner and the final selection is recorded in D-007 together with the stated tiebreakers.

Sensitivity check: halving the two capability criteria to ten points combined and reallocating those ten points to requirement and ASR fit gives B-009 90%, B-003 88%, B-005 86% and B-006 84%. The order of the leading four is unchanged, so the result is not an artefact of the weight placed on learning curve. A second check moved the weight the other way. Raising the two capability criteria to 30 points combined (team capability 12, framework learning 18) and reducing requirement and ASR fit to 5 gives B-009 80.0%, B-005 79.2%, B-003 78.4% and B-006 77.2%. B-009 remains first, but its lead falls to 0.8 points, so the ranking is sensitive to how heavily the learning curve is weighted. This is why the tiebreakers in D-007 matter.

13.3.6 Selected versions and compatibility

Target versions for the selected stack. Each is confirmed against composer.lock and package.json when the project is created. Any change after the baseline goes through the change record.

Component	Target version	Reason / compatibility
PHP	8.3 or later	Laravel 13 supports PHP 8.3 to 8.5. Availability on the BC Desktop still to be verified (R-014, AS-002).

Component	Target version	Reason / compatibility
Laravel framework	13.x	Current major release (March 2026): bug fixes until Q3 2027 and security fixes until Q1 2028, which covers the project. Laravel 12 bug fixes ended in August 2026.
Inertia (inertiajs/inertia-laravel, @inertiajs/react)	3.x (confirmed from composer.lock at the start of M3)	Inertia v3 was released in March 2026. v2 bug fixes ended on 26 September 2026 (security fixes until 26 March 2027), so v3 is preferred for a new project.
React	19.x (confirmed from package.json at the start of M3)	Rendered through the Inertia React adapter.
TypeScript	5.x (confirmed from package.json at the start of M3)	Used by the official React starter kit.
Node.js	20 LTS or later (confirmed against Vite's minimum at the start of M3)	Needed for npm and the Vite build.
PostgreSQL	Version 18, both locally and on NEON.	The local Docker image must match the provider's version (environment parity, Master Brief section 17).
Composer	2.x	PHP dependency manager.

13.4 Design Decisions

The two Assignment 2 design problems as final, defended Milestone 2 decisions. The full ADR entries are D-009 and D-010 in Section 9. This subsection is the design-level explanation with diagrams and code links.

13.4.1 Design Problem 1: Status-Change Side Effects

Problem

A status change on a request is not a single-field update. FR-016 and FR-019 require an audit entry recording who made the change and when, and FR-007 requires the requester to see the update reflected on their side. Writing all of this directly inside the method that changes the status collects several unrelated reasons to change and becomes harder to extend, for example if an SLA-overdue escalation reaction is added at a later milestone.

Assignment 2 Alternatives

Assignment 2 Task 1 compared a direct sequential-call approach against the Observer pattern and a full domain-event/pub-sub dispatcher, and recommended a uniform Observer applied at the ServiceRequest entity. A full dispatcher was set aside there as more infrastructure than a single event-raising entity needed.

Final Decision (adapts the Assignment 2 recommendation)

Once Laravel was confirmed as the stack, its built-in event/listener mechanism removed the infrastructure cost that ruled out a dispatcher in the Assignment 2 research, but Assignment 2 Task 2's transaction requirement meant a uniform Observer would not work: an after-commit reaction cannot also be the audit write. The reactions were split: the audit history write happens synchronously inside RequestStatusService::transition(), inside the same database transaction as the status update (D-008). Every other reaction, currently the requester notification, is a queued listener (NotifyRequesterOfStatusChange) on the RequestStatusChanged event, dispatched only once that transaction has committed. RequestStatusChanged implements ShouldDispatchAfterCommit, so Laravel dispatches it only after the D-008 transaction commits. If the transaction rolls back, the event is never dispatched (Laravel, 2026c).

Trade-off

The status-change reaction logic is no longer one uniform mechanism. A misconfigured listener still silently skips its reaction, the same risk a plain Observer would carry.

Class / Flow Diagram

Figure 13.3: Status-change write path (D-008, D-009, D-011)

Synchronous inside one transaction · asynchronous only after commit

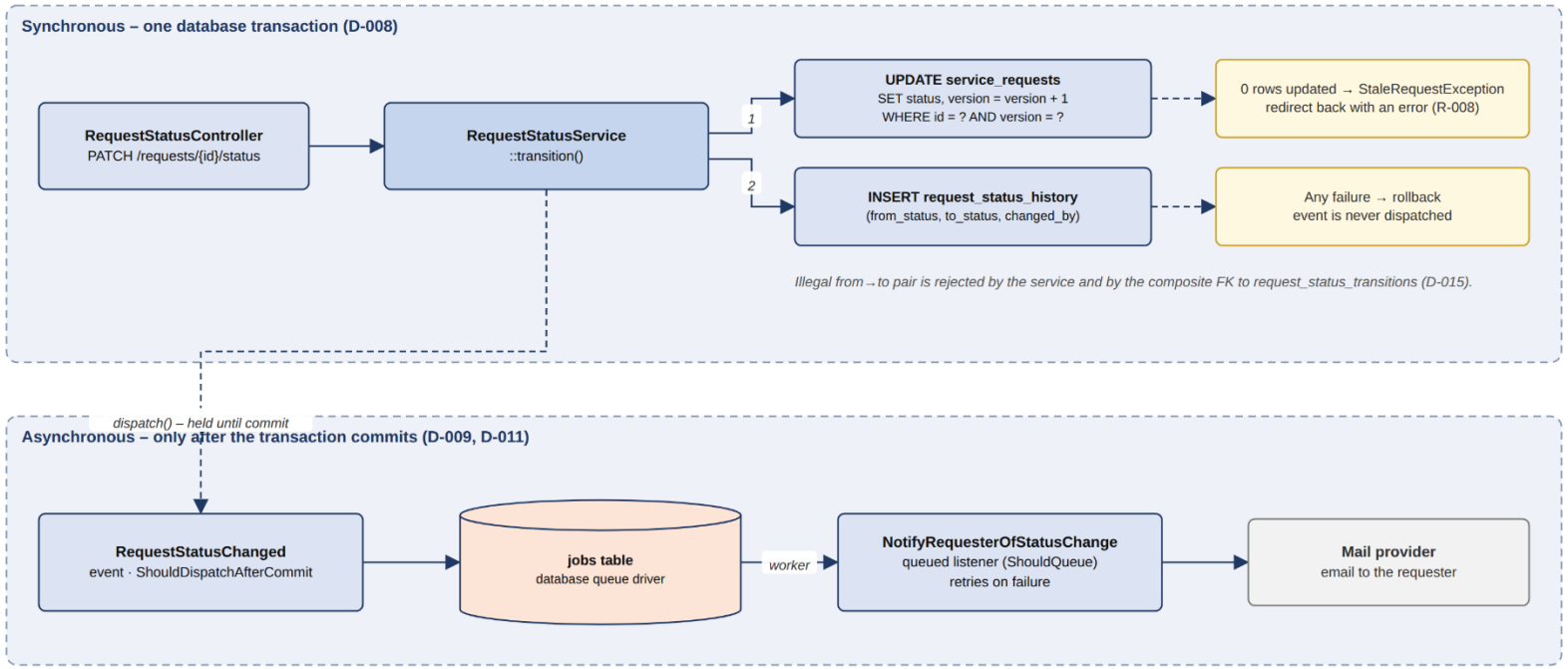


Figure: synchronous audit write inside the D-008 transaction, asynchronous notification only after commit.

Implemented in https://github.com/Destroyer1819/SEN381_Project/blob/main/app/Services/RequestStatusService.php (transition, version check and history row in one transaction), https://github.com/Destroyer1819/SEN381_Project/blob/main/app/Events/RequestStatusChanged.php and https://github.com/Destroyer1819/SEN381_Project/blob/main/app/Listeners/NotifyRequesterOfStatusChange.php. The listener is written as a queued listener with 3 tries and backoff. On Render's free tier the queue runs synchronously, so the email is sent during the request and a mail failure is logged instead of undoing the status change (D-018, TD-05). Verified by StatusTransitionTest and StatusNotificationTest.

Service	app/Services/RequestStatusService.php - transition()
Event	app/Events/RequestStatusChanged.php
Listener	app/Listeners/NotifyRequesterOfStatusChange.php (implements ShouldQueue)
Notification	app/Notifications/RequestStatusChangedMail.php
Test	tests/Feature/StatusTransitionTest.php

13.4.2 Design Problem 2: Category-Specific Request Handling

Problem

Decision D-001 fixed the category vocabulary itself, but not how category-specific validation and routing behaviour is implemented. Writing that behaviour as one long conditional inside the request-handling code means every new category requires editing and retesting that same method.

Assignment 2 Alternatives

Assignment 2 Task 1 compared the Strategy pattern against Factory Method and a configuration-only lookup. The configuration-only approach was set aside because CivicConnect's categories differ in behaviour, not only in the fields they require. Factory Method was set aside because the categories differ in behaviour, not in data shape. A subclass per category would force a class hierarchy onto the persisted request and complicate the data model (Assignment 2, Task 1).

Final Decision

The Strategy pattern is adopted as recommended. Each category implements a shared CategoryHandler interface (rules(), defaultAssignee()). CategoryRegistry, bound in AppServiceProvider, resolves the handler for a given category code. An unrecognised code falls back to OtherHandler. StoreServiceRequest merges the resolved handler's rules into its own FormRequest validation.

Trade-off

Requires a small registry/lookup mechanism. If the category list stays very small with simple, near-identical rules, this is more structure than the problem needs.

Class Diagram

Figure 13.4: Category-specific handling – Strategy pattern (D-010)

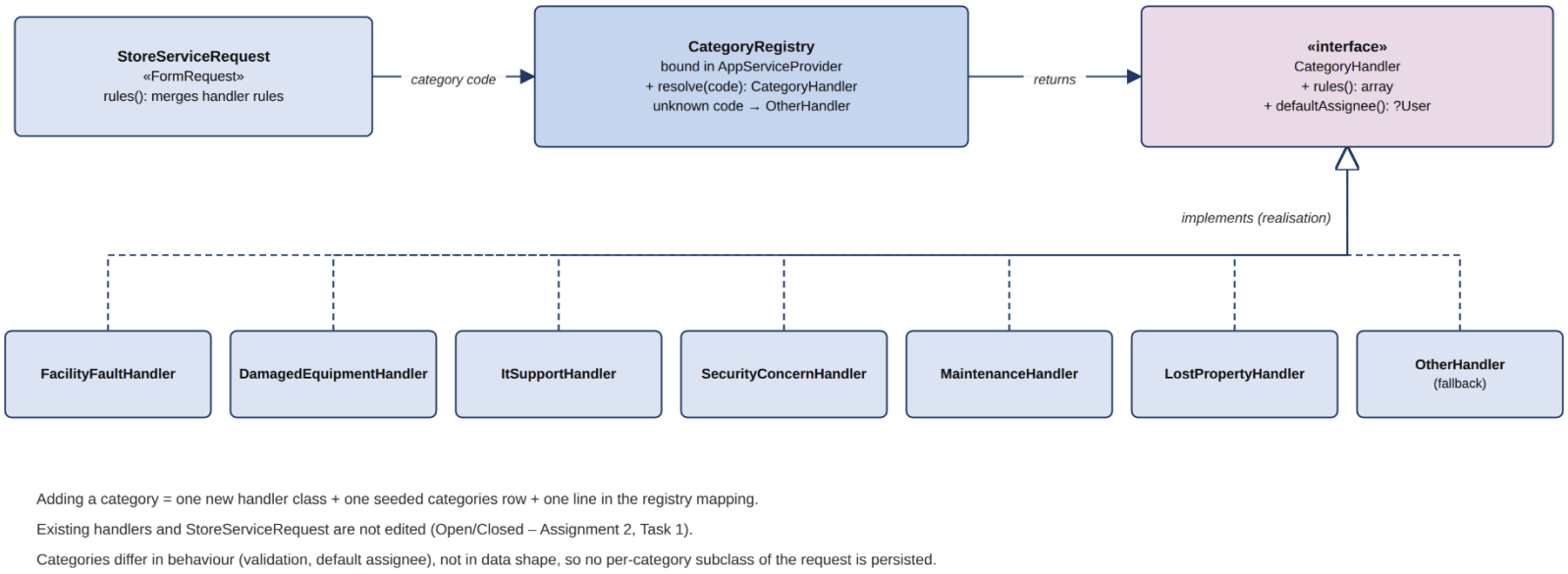


Figure: CategoryRegistry resolves a CategoryHandler per category code. OtherHandler is the fallback.

Implemented in https://github.com/Destroyer1819/SEN381_Project/blob/main/app/Domain/Categories/CategoryHandler.php (interface), CategoryRegistry.php, and seven category handlers in app/Domain/Categories/Handlers/ (Damaged Equipment, Facility Fault, IT Support, Lost Property, Maintenance, Other, Security Concern) that share BaseCategoryHandler. Verified by tests/Feature/CategoryHandlingTest.php.

Interface	app/Domain/Categories/CategoryHandler.php
Registry	app/Domain/Categories/CategoryRegistry.php
Handlers	app/Domain/Categories/Handlers/ (FacilityFault, DamagedEquipment, ItSupport, SecurityConcern, Maintenance, LostProperty, Other)
Binding	app/Providers/AppServiceProvider.php
Test	tests/Feature/CategoryHandlingTest.php

13.5 UI & Information Architecture

The role-based site map, low-fidelity wireframes for the six screens in scope for Milestone 2, and the usability and accessibility rationale behind them (NFR-007, NFR-008).

13.5.1 Role Sitemap

Navigation is organised by role rather than by feature. Auth/Login is the single entry point for all three roles.

Which pages a logged-in user can reach is read from users.role, not chosen at login. Pages are React components in resources/js/Pages, rendered through Inertia. Laravel route middleware limits each route to the roles that use it, as a navigation convenience. The actual access control is ServiceRequestPolicy on the server (ASR-002).

Every page is reachable from the role landing page in at most two clicks, satisfying the three-click rule in NFR-007.

Figure 13.5: Role sitemap – Inertia pages in resources/js/Pages

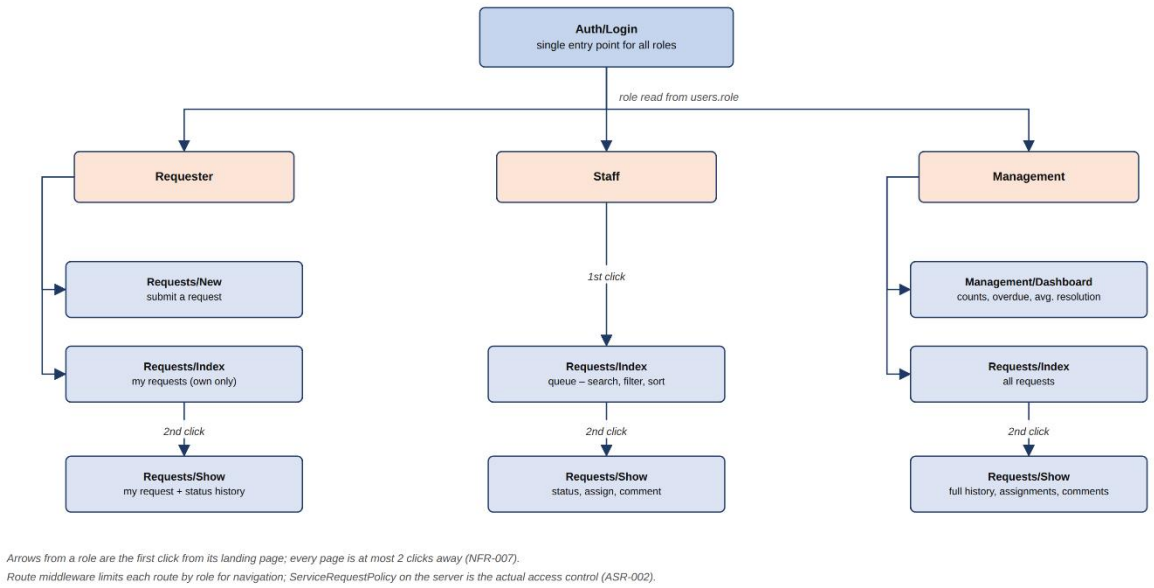


Figure: role sitemap - Inertia pages in resources/js/Pages.

Every screen is reachable from LoginPage in at most two clicks, satisfying the three-click rule in NFR-007. Route-level role guards are a usability convenience only. The actual access control is enforced server-side by ServiceRequestPolicy (ASR-002).

13.5.2 Wireframes

Low-fidelity: they fix what information and controls each screen needs, not final visual design.

Figure 13.6: Low-fidelity wireframes

Layout and content only – not final visual design

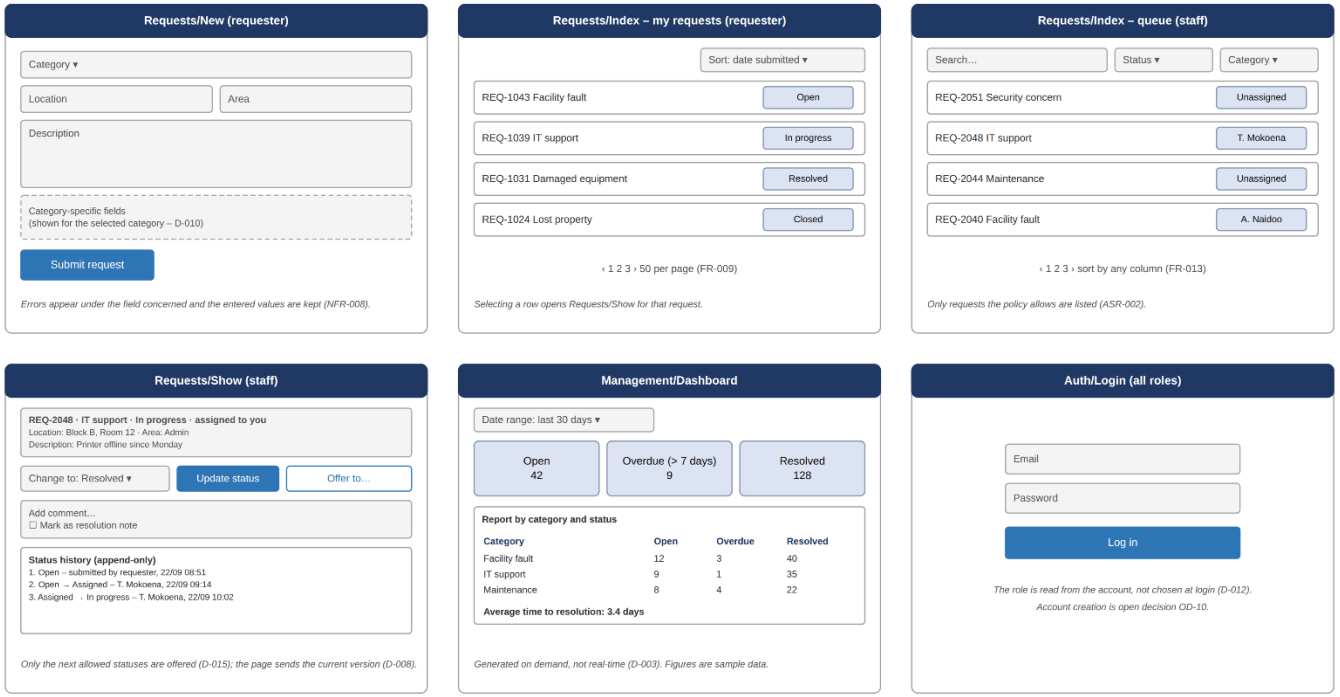


Figure: low-fidelity wireframes - Requests/New, Requests/Index (requester and staff views), Requests/Show, Management/Dashboard and Auth/Login.

13.5.3 Usability and Accessibility Rationale

Requirement	How the screens satisfy it
NFR-007 (3 clicks from main screen)	Every page is at most 2 clicks from the role landing page reached after Auth/Login. Requests/New and Requests/Index are 1 click from the requester landing page. Requests/Show is 1 click from Requests/Index.
NFR-008 (validation failures produce actionable messages)	When validation fails, Laravel redirects back to Requests/New with an error next to each field and the submitted values preserved. If the version is stale, Requests/Show reloads with the message “This request changed since you opened it. Please review and try again.”
FR-004 (submission feedback)	Submitting Requests/New always ends in either a confirmation on the created request or field-level errors. There is no silent failure state.

Requirement	How the screens satisfy it
FR-012 (views provide relevant information)	Requests/Index shows the category, status, time submitted and assignee in each row.

Formal accessibility testing is not scoped to Milestone 2. These wireframes fix the information architecture the accessibility work at Milestone 3 will build on.

13.6 Interfaces and Integration

Because D-006 selects an Inertia presentation layer, CivicConnect exposes no public API at Milestone 2. The interfaces described below are the boundaries that do exist, documented at the level currently established. Where an interaction is not yet established it is recorded as a forward engineering consideration rather than specified speculatively

13.6.1 Interface inventory

	Boundary	Type	Crosses a network boundary	Decision reference
1	Browser to application	Inertia page visit over HTTPS, authenticated by session cookie	Yes, client to server	D-006, D-012
2	Web process to queue worker	Job row written to the database queue and picked up by the worker	No, same codebase and same database	D-011
3	Application to PostgreSQL	Database connection from the web process and the worker	Yes, to the managed database instance	D-007, D-008
4	Application to mail provider	SMTP or provider API call made by the queued listener	Yes, external service	D-011, D-013

The only internal boundary the team introduced by choice is number 2, and it exists because D-011 requires a notification failure to be isolated from the status-change transaction. Boundaries 1, 3 and 4 are unavoidable for a browser-based application with a managed database and email delivery.

13.6.2 Page contracts

Inertia page visits replace the endpoint table an API-based design would have. Each route names the page component it renders, the props it sends, the input it accepts and how it responds. These props are an internal contract between a controller and its page component. They are not a published API.

Route	Method	Page component	Props sent to the page	Input accepted	Responses	Requirements
/login, /register	GET (POST handled by Fortify)	auth/login, auth/register	status message	email, password (and name on register)	Redirect to /requests, validation errors on the form	FR-010, D-022
/logout	POST (Fortify)	-	none	none	Redirect to the login page	FR-010
/requests/create	GET	requests/create	Active categories	none	Page	FR-001, D-010
/requests	POST	-	-	category_id, location, area, description	Redirect to the new request with "Request	FR-001, FR-003, FR-004

Route	Method	Page component	Props sent to the page	Input accepted	Responses	Requirements
					submitted. Your reference is CC-…", field-level errors	
/requests	GET	requests/index	Paginated requests scoped to the role, filters, categories, statuses	q, status, category_id, from, to, mine, active, overdue, sort, direction, page	Page (50 per page)	FR-002, FR-005, FR-009, FR-011, FR-013
/requests/{id}	GET	requests/show	Request, history, comments, assignments (empty for requesters), allowed actions	none	Page, 403 when the policy denies	FR-006, FR-014, FR-022
/requests/{id}/take	POST	-	-	none	Redirect, "Someone else took this request first" if already taken, 403 when not allowed	FR-015, FR-018
/requests/{id}/status	POST	-	-	status, version, note (required when resolving)	Redirect with "Status updated.". Stale version shows an error. 403 when not allowed	FR-017, FR-018, FR-019, FR-021
/requests/{id}/offer	POST	-	-	assigned_to (an eligible staff member)	Redirect, error if the offer is not possible, 403 when not allowed	FR-015, D-020
/assignments/{id}/respond	POST	-	-	decision (accept or decline)	Redirect, 403 unless the offer is to the current user	FR-015, D-020
/requests/{id}/comments	POST	-	-	body (max 2,000 characters)	Redirect with the comment added, 403 when not allowed	FR-020

13.6.3 Failure behaviour

Condition	Response	Requirement
Input fails validation	Field-level messages returned to the originating form, with the submitted values preserved	FR-003, FR-004, NFR-008
Policy denies the action	403 with a generic message. The attempt is logged	FR-011, FR-018, NFR-002
Version supplied is stale	Redirect back to Requests/Show with the message “This request changed since you opened it” and the current state reloaded	R-008, D-008
Legal transition not found	Validation failure naming the allowed next statuses	FR-017, D-015
Unexpected server error	Generic error page with no internal detail exposed	NFR-008

13.6.4 Internal event contract

RequestStatusChanged is the one internal payload that behaves like a contract, because a listener consumes it rather than sharing the caller’s memory of the request.

Field	Purpose
request_id and reference number	Identifies which request the notification concerns
from_status, to_status	Describes the transition that occurred
actor_id	The staff member who made the change, for the message and for traceability
occurred_at	The timestamp recorded in the history row
requester contact address	Where the notification is delivered

The event deliberately does not carry the full request entity, which follows the Assignment 2 Task 3 analysis of what information an interaction actually needs. Listeners are written as queued with retries and backoff. On staging they run synchronously, so a failed email is logged and not retried (TD-05). A permanently failed job is recorded in the failed jobs table and does not affect the committed status change.

13.6.5 External interface: mail provider

The queued listener is the only component that calls an external service. Credentials and the provider endpoint are supplied as environment variables and are never committed (section 13.7.2). The mail provider is Resend (D-019), called over HTTPS through Laravel's Resend driver, with the key held in environment variables. On staging, mail is sent synchronously (D-018), so a delivery failure must be caught and logged and must not undo the saved status change (D-011).

13.6.6 Change and versioning

Page props are internal: a controller and its page component change together in a single pull request, so no interface versioning scheme is required at this stage. If a JSON API is added later for a non-browser client, it becomes a published contract and requires versioning, deprecation handling and its own authentication decision. That possibility is recorded as an open decision in section 13.8 and as a trade-off in D-006 and R-016.

13.6.7 Interactions not yet established

The following interactions are recorded here rather than specified, because no requirement currently establishes them: third-party or partner integration. An export interface for reporting beyond the on-demand view in D-003. And any inbound channel from email, SMS or WhatsApp, which section 3.2 places out of scope. Each would require its own interface decision before implementation.

13.7 Deployment & Environments

The environment strategy for the selected stack (Laravel with Inertia serving React and TypeScript, and PostgreSQL) and the deployment risks it interacts with. The hosting provider for the application will be Render(D-018), but the database host has been decided (D-016).\

Final state (M4): the deployed system is one Render free-tier web service built from the Dockerfile in the Frankfurt region, with the queue running synchronously and no queue worker or scheduler (D-018), connected to the Neon staging branch (D-016) and sending mail through Resend over HTTPS (D-019). The queue worker and scheduler described in this section and shown in Figure 13.1 are the target design. Their absence is recorded in R-009, TD-05 and TD-08.

13.7.1 Environment Structure

Environment	Composition	Purpose
Development	Local host running PostgreSQL for testing, switching to NEON for staging and production. Laravel served by php artisan serve (or a container). The Vite dev server for React assets. A local queue worker (php artisan queue:work).	Local development and manual testing with the same database engine and queue driver as production.
Test (CI)	GitHub Actions workflow (.github/workflows/tests.yml): PHP and pnpm installs from lockfiles, composer and pnpm security audits, Pint, php artisan test on SQLite (.env.testing), TypeScript type-check, ESLint, Prettier and Vitest. PostgreSQL-specific constraints are covered by the 53 schema checks run on Neon. PostgreSQL in CI is future work (TD-03).	Every pull request is built and tested against real PostgreSQL before merge.
Staging / Production	One Laravel deployable serving Inertia pages and the compiled assets, one queue worker process, the scheduler, and a managed PostgreSQL instance on a free tier. Staging and	Demonstration, then real deployment.

Environment	Composition	Purpose
	production use separate databases. Provider for the application will be hosted on Render (D-018), and the database host is NEON (D-016).	

13.7.2 Configuration and Secrets

All environment-specific values are read from environment variables and never hard-coded: APP_KEY, APP_ENV, APP_URL, DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD, MAIL_MAILER, RESEND_KEY, MAIL_FROM_ADDRESS, MAIL_DEMO_TO and the mail provider credentials, SESSION_DRIVER, SESSION_SECURE_COOKIE and QUEUE_CONNECTION=database.

Render's free tier, selected in D-018, runs no background processes, so AS-001 is not met. Staging therefore uses two replacements:

Notifications: QUEUE_CONNECTION=sync. The listener stays queued in the code but runs straight after the status change has been committed, sending through Resend's HTTPS API (D-019). Because the status change is already saved, a mail failure is caught and logged and does not undo it (D-011). The cost is that the request waits while the email is sent, and there is no automatic retry.

Retention purge: the purge is scheduled in Laravel (routes/console.php, daily at 02:00). Render's free tier runs no scheduler, and the planned GitHub Actions workflow to run php artisan model:prune against Neon was not set up, so the purge does not run on staging (TD-08, L-02).

Moving to a host with background processes (Render's paid tier, or option (b) in D-018) only requires QUEUE_CONNECTION=database and a worker process. The code does not change. This is tracked as R-009.

13.7.3 Queue worker and scheduler

D-011 makes NotifyRequesterOfStatusChange a queued listener. On a host that runs background processes, a queue worker (php artisan queue:work) delivers it after the response, and Laravel's scheduler runs the retention purge (FR-008) daily at 02:00 (routes/console.php).

The retention purge (FR-008) runs from Laravel's scheduler, so the host must also support either a cron entry running php artisan schedule:run every minute or a persistent php artisan schedule:work process.

Both are covered by Risk R-009 and assumption AS-001: a host without background processes would accept notifications that are never sent. Fallback: run queue:work --stop-when-empty from the scheduler, accepting delayed delivery.

13.7.4 Free-Tier Limits

Two limits interact directly with this work and are tracked as Risk R-013: a free-tier PostgreSQL instance that pauses when idle would make the demo unreliable, and the storage cap (R-002) interacts with the 5-year retention period on service_requests, request_status_history and request_comments. Mitigation: record the chosen host's actual limits once D-013 is finalised, seed a small, realistic dataset for the demo, and warm up the database connection before presenting.

Milestone 3 adheres to the limits by making use of NEON's free plan. This allows for a free account capable of storing 1 GB of data per project, Meaning it can take an estimated 250,000 requests, as 310 test requests has taken up an estimated 1.08 MB. It provides 100 compute-unit hours and the suspension of the database when idle after 5 minutes (NEON, 2026). London was chosen to host the database as there was no option for it to be hosted in the African region.

Furthermore, the application will be hosted on Render. It also adheres to the limits as it is a free-tier service. It does however have limits of its own, such as the 15 minutes before sleeping, about a minute before it wakes up 750 hours of usage a month and 512MB of RAM. It also blocks SMTP. This is resolved by using Resend, which has its own limits of 100 emails a day, and only one recipient without a domain.

13.7.5 Backup and Recovery

NFR-006 requires data to be recoverable within 48 hours. Free tiers do not always include automatic backups (Supabase's free plan, for example, does not) (supabase, 2026), so the baseline is a scheduled pg_dump of the database, run daily and stored outside the database host, with a restore rehearsed in Milestone 3. If the chosen provider includes daily backups or point-in-time recovery, that is used as an additional layer. A single PostgreSQL instance is accepted as a single point of failure at MVP under the cost constraint.

Neon's free plan offers instant restore within a 6-hour window, which does not meet NFR-006's 48-hour requirement, so the pg_dump baseline stands. Final status: the daily pg_dump and the restore rehearsal were not carried out (NFR-006 not met, L-04).

13.8 Open and Deferred Decisions

Milestone 2 section 6 requires open decisions and deferred concerns to be identified separately from the approved baseline. None of the items below blocks the Architecture, Technology and Initial Design Baseline. Each is open because the evidence needed to decide it does not yet exist, and each names that evidence, an owner and the point at which it must be closed.

ID	Open decision	Why it is still open	Evidence still needed	Owner	Decide by	Related records	Final Status
OD-01	Hosting provider	D-013 fixes the topology but not the provider	Background-worker support, backup and recovery facility, idle-pause behaviour, storage cap and pricing beyond the free tier	Michael	Before Milestone 3 staging deployment	D-013, R-002, R-009, R-013, FEC 4	Closed. Database decided by D-016, and application decided by D-018
OD-02	Email delivery provider	Depends on the host and on free-tier send limits	Daily and monthly send allowances, sender-domain verification requirements, log retention	Jared	With OD-01	D-011, R-009, section 13.7.4	Closed by D-019

ID	Open decision	Why it is still open	Evidence still needed	Owner	Decide by	Related records	Final Status
OD-03	Staff sub-roles by department	No stakeholder requirement for department-scoped visibility has appeared	Whether staff need to see only their department's requests	Jared	Milestone 3, if a need appears	FEC 1, D-012, CR-009	Closed. No stakeholder need has appeared.
OD-04	Depth of audit history exposed to staff	Retention is decided, how much history each role sees is not	Management input on whether staff need full history or only the current state	Xander	Milestone 3	FEC 3, CR-012, FR-014	Closed. Staff/management sees full history and assignment, requesters see only their history/comments
OD-05	Encryption at rest for sensitive categories	Depends on what the chosen provider offers and at what cost	Provider encryption options, key handling, cost impact	Xander	With OD-01	FEC 6, D-012, ASR-002	Partly closed. The Provider's default is relied on.
OD-06	Static analysis as a blocking quality gate	Adopted as a warning first, following the Assignment 2 Task 4 recommendation	One clean run against real application code	Michael	Milestone 3	Section 10.3, NFR-009	Closed. Pint, ESLint, type-check and audits are blocking CI steps (tests.yml).
OD-07	Verification method for NFR-004	The target is measurable but the measurement method is not chosen	A seeded data volume representative of five years and a repeatable timing approach	Michael	Milestone 3 test strategy	NFR-004, ASR-001	Closed: tests/perf/request-list.mjs on 5,000 seeded requests (14.8).
OD-08	Accessibility testing scope	Section 13.5.3 fixes the information architecture only	Which accessibility checks are feasible within the Milestone 3 window	Xander	Milestone 3	NFR-007, NFR-008	Not done, carried as future work (L-08).
OD-09	Whether a JSON API is added alongside Inertia	No second client exists and none is in scope	A stakeholder need for a non-browser client	Jared	Milestone 3	D-006, R-016	Closed: not added, no second client exists.

ID	Open decision	Why it is still open	Evidence still needed	Owner	Decide by	Related records	Final Status
OD-10	Account creation	No requirement defines how requesters and staff get accounts	Whether requesters self-register (starter-kit registration) and whether staff and management accounts are created by Management	Jared	Before the Milestone 3 authentication slice	FR-010, D-012, 13.2.3	Closed. New users are defaulted to “requestors”. Authorisation is given through the database (D-022).

Deliberately deferred rather than open:

construction itself is deferred to Milestone 3 under D-014, with the consequences recorded in R-015 and Forward Engineering Consideration 7. That is a decision with a date, not an unresolved question, which is why it appears in Section 9 rather than in this table.

14. Milestone 3: construction, quality and release readiness

14.1 Overview

The release candidate v3.0-rc1 (main, commit 288849f, PR #25, CI run #21) is one Laravel 13 + Inertia + React application on PostgreSQL. Built:

- registration and login with three roles.
- request submission with validation and a reference number.
- role-scoped request list with search, filters and 50 rows per page.
- detail page with history and comments.
- staff Take.
- management dashboard with status, category and overdue counts.
- 5-year retention purge (scheduled in code, does not run on Render, TD-08).
- mail through Resend.
- status changes (start work, resolve, close), offers and declines, status-change emails and category handlers.

14.2 M2 baseline conformance

M2 decision	Implemented	Evidence	Status
D-006/D-007 Inertia monolith, Laravel + React + PostgreSQL	As planned (Laravel 13, React 19, PostgreSQL 18)	routes/web.php, composer.json	Conforms
D-012 Session auth, policies	Role column, ServiceRequestPolicy, scopeVisibleTo()	RequestVisibilityTest	Conforms
D-015 Seeded transitions	Transition table, composite foreign key	Schema checks, TakeRequestTest	Conforms
D-008 Status service with version check	RequestStatusService, StaleRequestException on a version mismatch, Take uses a conditional update	RequestAssignmentController	Changed (CC-3)
D-009/D-011 Event + queued mail	RequestStatusChanged event, NotifyRequesterOfStatusChange listener, sent through Resend	config/mail.php	Conforms (runs synchronously on Render, D-018)
D-013 Deployment	Neon (D-016), Render (D-018), no worker	Section 9	Changed (CC-1)

M2 decision	Implemented	Evidence	Status
CR-012 Retention purge	Prunable, daily at 02:00	ServiceRequest.php, routes/console.php	Implemented, but not running on staging (TD-08)
D-010 Category handling	CategoryHandler, CategoryRegistry, 7 category handlers	CategoryHandlingTest	Conforms

14.3 Controlled changes

M2 decision	Implemented	Evidence	Status	Consequence
CC-1 (D-018, CR-037).	Hosting deferred to M4 (D-017)	M3 postponed, same final deadline	Render free tier	No worker: synchronous mail, purge through GitHub Actions but not set up (TD-08).
CC-2 (D-019, CR-030)	Queued email, provider open (OD-02)	Render free tier blocks SMTP ports	Resend over HTTPS, demo inbox redirect	Emails reach one inbox, no retry
CC-3 (D-008, CR-038)	Version check in RequestStatusService	Two staff pressing Take at the same moment could both pass a read-then-write version check. A single conditional update (only an open, unassigned row is changed) lets the database decide, so exactly one Take succeeds.	Take updates only open, unassigned rows and increments version	Double Take impossible, other transitions use the version check in RequestStatusService.

14.4 Construction and configuration

Personal branches merge into Development through pull requests (#3 to #23), then into main which is protected (shown as a protected branch on GitHub). The team rule is two approvals per pull request ([https://github.com/Destroyer1819/SEN381_Project/rules/23877310?ref=refs%2Fheads%2Fmain]10.2, NFR-009).

Review evidence: Most reviewing happened as team discussions before pull requests were opened, causing the pull requests themselves to carry few review comments.

Secrets stay in .env (git-ignored), Render environment variables and GitHub secrets. .env.example lists the keys empty. Tests use .env.testing with SQLite. Neon has separate staging and production branches, and production is never seeded.

Staging is hosted on Render's free tier (D-018), built from the Dockerfile and connected to the Neon staging branch, with Resend mail redirected to MAIL_DEMO_TO. Differences from production are free-tier sleep, synchronous mail, one demo inbox and seeded accounts. Rollback: redeploy the previous tag and migrations are additive.

14.5 CI and automated verification

The Tests workflow runs on every pull request and on pushes to main and Development. dependency install from lockfiles, Pest (php artisan test), TypeScript type-check, ESLint and Vitest. Any failure marks the pull request as failing. Main is protected by the “Main Protection” ruleset (pull request and 2 approvals required), but a passing CI run is not yet a required status check, so the team rule is to merge only after a green run, as with PR #18 and PR #23.

Passing run: run #17, https://github.com/Destroyer1819/SEN381_Project/actions/runs/38868569090.

Failed runs that blocked a merge: runs #7 and #8 on PR #18 failed at the frontend install step (pnpm lockfile mismatch, DF-08) and the fix passed in run #9 before merging. run #15 on PR #23 failed the type-check (DF-09) and passed in run #16 after 1fb7142.

Static and security gates: Pint checks PHP code style, composer audit and pnpm audit (high and above) check dependencies for known vulnerabilities.

Gaps: no secrets scan, Playwright and PostgreSQL not in CI, no coverage collected.

14.6 Quality strategy applied

Risk	Stronger verification
Data shown to the wrong role (ASR-002, FR-010, FR-011)	Decision-table tests over role x assignment state, 403 tests, TC-E2E-02
Invalid or lost submissions (ASR-003, FR-001, FR-003, FR-004)	Equivalence partitions and boundary values, server and client
Two staff take one request (R-008, FR-015)	Conditional update and an "already assigned" test
Wrong dashboard figures (FR-023, FR-024)	Integration test with controlled data
Status and history integrity (FR-017, FR-019)	53 PostgreSQL schema checks, also on Neon

14.7 Black-box, integration and system validation

Against the brief's minimums: 32 Vitest unit/component cases. Black-box cases in SubmitRequestTest and RequestVisibilityTest. Pest integration tests through HTTP, policy and database. 3 Playwright E2E tests. Techniques: equivalence partitioning, boundary values, decision table, state transition. Negative cases: guest redirects, 403 on Take, Comment and Dashboard, invalid input. Execution evidence for automated rows: the CI run above.

ID	Basis	Technique / level	Input	Expected	Status
TE-01	FR-003	BVA / feature + unit	Description 9, 10, 2000, 2001 characters	9 and 2001 rejected, 10 and 2000 accepted	Pass
TE-02	FR-003, FR-004	EP, negative	Empty form, unknown category	Field messages, nothing saved	Pass
TE-03	FR-011	Decision table / integration	3 roles x 4 assignment states	Each role sees exactly its set, others 403	Pass
TE-04	FR-015, R-008	State, negative / integration	Already assigned, non-staff tries Take	403, assignment unchanged	Pass
TE-05	FR-023	Integration	4 controlled requests, 1 older than 7 days	total 4, overdue 1	Pass
TE-06	Core journey	E2E (TC-E2E-01 to 03)	Seeded accounts	Submit to resolve, access blocked, dashboard filter	Pass (Render staging, 16/10/2026)

14.8 Performance and reliability

Operation: the request list with search and filters (GET /requests), the most frequent staff and management operation. NFR-004 allows 1 s per 100 rows. A page holds 50 rows, so the target is at most 500 ms per page.

Test: tests/perf/request-list.mjs (Playwright) logs in as the management user and requests each scenario 20 times after one warm-up request. Environment: Windows, app served with php artisan serve, local PostgreSQL 18 database. Data: 5,000 generated requests (SEED_REQUESTS=5000) plus the scenario data.

Scenario	Median	95th percentile	Max	Errors
No filters (page 1)	83 ms	120 ms	182 ms	0 / 20
Status + category filter	76 ms	84 ms	119 ms	0 / 20
Text search (“room”)	84 ms	90 ms	193 ms	0 / 20

Result: every request was well under the 500 ms target, with no errors, so NFR-004 is met in this environment. This does not, however, include the latency added by the distance due to the NEON hosted database being hosted in London.

14.9 Defects and quality interpretation

Ten defects were recorded (Section 16) and all are fixed. Most came from environment and tooling (Windows server start-up, pnpm versions, the type-check, clashing test data) or from test timing, rather than from business rules, and CI or the test suites caught them before they reached Development. The most serious, DF-04, could have shown requests to the wrong staff member. RequestVisibilityTest found it and now guards against it. The main remaining concern is untested behaviour rather than known defects: PostgreSQL-only constraints and row locks (including the lock that stops two simultaneous offer declines) are not exercised by the SQLite test run, and the status-change, offer and category-handler code was merged last, so it has the least time in use beyond its automated tests.

14.10 Release readiness

Area	Result	Interpretation / action
Build & CI	Pass	Install, Pint, composer and pnpm audits, Pest, type-check, ESLint and Vitest pass on the release candidate https://github.com/Destroyer1819/SEN381_Project/actions/runs/38919834328
Automated regression	Pass	Visibility, submission, Take, comments, dashboard protected.
API / integration	Pass	Built features work through HTTP, policy and database (SQLite)
System / E2E	Pass	TC-E2E-01 to 03 passed against Render staging (14.4). 3 passed in 42.6 seconds total.
Static / security	tsc, ESLint, Pint and audits pass	Secrets scan not configured.
Performance	Median 76 to 84 ms, max 193 ms, 0 errors	Meets NFR-004 (≤ 500 ms per page) locally. Not tested at scale or on Neon
Known defects	0 open	Section 16
Acceptance	24 of 24 FRs implemented	22 have automated tests, FR-009 (50 per page) and FR-012 (list columns) were checked by hand (RTM 6.3)

Classification: CONDITIONALLY READY. All required checks pass and the core journey works end to end. The conditions are non-blocking: status emails reach one demo inbox, PostgreSQL-only behaviour is checked by schema tests rather than in CI, and the offer rule (D-020) awaits final team confirmation.

Residual risk	Why it remains	Mitigation / next action
PostgreSQL constraints and locks untested automatically	CI runs on SQLite	53 schema checks on Neon. Add PostgreSQL to CI

Residual risk	Why it remains	Mitigation / next action
Free-tier cold starts and limits	Render and Neon sleep when idle	Warm up before the demo. Paid tier before real use
Emails reach one inbox, no retry	Resend without a domain, synchronous sending	Verify a domain and restore a worker
Scale untested	Single-user local test	Add indexes, retest in a representative environment

15. Technical Debt Register

ID	Item	Consequence	Next action
TD-01	Visibility rule in two places (scopeVisibleTo, policy)	Can drift and leak data	One shared rule
TD-02	Transition table and status CHECK kept in step by hand	New status needs two edits	Add a consistency test
TD-03	Tests run on SQLite	CHECKs and locks untested	PostgreSQL in CI
TD-04	No index on status or updated_at (13.2.4 says there is)	Slower filters and purge	Add one migration
TD-05	Synchronous mail, no worker (D-018)	Slower status changes, no retry	Worker host
TD-06	Seeded accounts share one known password	Risky on public staging	SEED_PASSWORD
TD-07	No notification list, password reset or timezone rule for dates (frontend)	Usability gaps	Next milestone
TD-08	Retention purge never runs on Render (no scheduler)	Data kept beyond 5 years. FR-008 and FR-016 partially met	GitHub Actions cron calling model:prune, or a host with a scheduler
TD-09	Playwright E2E not in CI	E2E journeys only re-verified when run by hand	Add an E2E job against a CI-served app

No debt was repaid during finalisation: the code was frozen at the M3 release candidate, so every item is retained with its consequence stated

16. Defect Register

ID	Defect	Severity	Fix	Regression evidence	ID
DF-01	artisan serve failed on Windows	Low	d404dbd	Manual	DF-01
DF-02	Frontend test failure	Low	32e57b6	Vitest	DF-02
DF-03	Home redirected to /settings/profile	Low	1b1c449	Manual	DF-03
DF-04	Visibility scope returned the wrong set	High	3b62324	RequestVisibilityTest	DF-04
DF-05	Schema-check test IDs clashed with seeded rows	Medium	IDs moved to 900001+	53/53 re-run	DF-05
DF-06	E2E used accounts the seeders don't create	Medium	832b8ec	E2E login passes	DF-06
DF-07	E2E picked a category before the form loaded	Low	832b8ec	TC-E2E-01 submit step	DF-07
DF-08	CI install failed: pnpm 12 ignored package.json overrides	Medium	5752253, 3bf3e07 (CI runs #7, #8 failed, #9 passed)	CI install step	DF-08
DF-09	Unused UI components broke the type-check	Medium	1fb7142	CI run #15 failed, #16 passed	DF-09
DF-10	E2E-01 checked the status before the detail page had loaded, so other resolved requests could give a false result	Low	dbc08b7	TC-E2E-01 final step	DF-10

17. Final delivery and conclusion

17.1 Final product release baseline.

The assessed product is release v3.0-rc1: commit 288849fd91f39c35ecdd15fdeec31455884f32c6 (merge of PR #25, which brought PR #24 from Development). This is the same code as the M3 release candidate. No code changed after M3. PED v4.0, RTM 6.4 and the demonstration all refer to this commit. It is one Laravel 13 + Inertia + React application on PostgreSQL 18, deployed to Render (staging) with the database on the Neon staging branch.

Completed on 16/10/2026 after the M3 evidence in Section 14 and RTM 6.3 was recorded (CI run #17), and therefore first evidenced in v4.0: the offer and decline workflow (D-020, OfferTest), sorting (RequestSortTest), staff status changes with resolution notes (StatusTransitionTest) and the average resolution time on the dashboard (DashboardTest). The code was frozen at the M3 release candidate. Finalisation was documentation and closure only.

Critical user journeys that work in v4.0: a requester submits a request and follows its status. Staff find, take or offer, progress and resolve it with a note. Management monitors all requests on the dashboard. All three are covered by TC-E2E-01 to 03 against staging.

17.2 M1 scope baseline to final delivery

This table shows each commitment in the scope baseline made in milestone 1, and uses the final status categories for the Final Project brief. Deferred and out of scope items are listed separately so as to not be read as failed commitments. All scope changes after Milestone 1 went through, and were logged in, the Change Record.

M1 scope item	M1 status	Final status	Acceptance criteria	Evidence	Conclusion / reason for variance
Request submission with category, location, area and description (3.1)	In scope	Implemented	Met (FR-001 to FR-004)	SubmitRequestTest, TE-01, TE-02, TC-E2E-01	Delivered as committed.
Selectable category (3.1)	In scope	Implemented	Met	CategorySeeder (7 categories, D-001), CategoryHandlingTest	Delivered as committed.
Staff view, search, filter, sort, assign, accept, comment, update status (3.1)	In scope	Implemented	Met (FR-012 to FR-021)	RequestListFilterTest, RequestSortTest, TakeRequestTest, OfferTest, CommentTest, StatusTransitionTest	Delivered. Offer and decline rule added through D-020.
Management dashboard by status, category, overdue and basic reporting (3.1)	In scope	Implemented	Met (FR-022 to FR-024)	DashboardTest, TE-05, TC-E2E-03	Delivered, including average time to resolution.

M1 scope item	M1 status	Final status	Acceptance criteria	Evidence	Conclusion / reason for variance
Role-based access (3.1)	In scope	Implemented	Met (FR-010, FR-011, NFR-002)	RequestVisibilityTest, TE-03, TC-E2E-02	Three roles after CR-009.
Email notification of a status change (3.1, CR-011)	In scope	Partially implemented	Met for the code path, delivery limited	StatusNotificationTest, D-019 evidence (15/10/2026)	Sends through Resend. Without a verified domain, all mail goes to one demo inbox (D-019).
Separate Administrator role	Deferred (CR-009)	Still deferred	Not applicable	Section 3.3, CR-009	Correctly outside the committed scope. Management carries admin functions.
Push notifications	Deferred	Still deferred	Not applicable	Section 3.3	Correctly outside the committed scope.
Power BI style analytics	Deferred	Still deferred	Not applicable	Section 3.3	Correctly outside the committed scope.
Native mobile app, multi-language, WhatsApp, SMS, telephone	Out of scope	Not implemented (by design)	Not applicable	Sections 3.2, 3.4	Excluded for cost (CON-003). Not a failed commitment.

17.3 Scope Completion Percentage

Each acceptance criterion in the approved M1 baseline (Sections 4.1 and 4.2, as amended by approved change requests) was given a final value: Met = 1.0, Partially met = 0.5, Not met = 0.0. The PED uses High / Medium / Low priorities instead of MoSCoW. High is treated as critical (Must-have).

ID	Priority	Value	Final status	Evidence	Note / reason
FR-001	High	1.0	Met	SubmitRequestTest, TC-E2E-01	
FR-002	High	1.0	Met	RequestVisibilityTest (requester cases)	
FR-003	Medium	1.0	Met	TE-01 (BVA), TE-02, schema checks	
FR-004	Medium	1.0	Met	TC-E2E-01 success message, field errors (TE-02)	

ID	Priority	Value	Final status	Evidence	Note / reason
FR-005	Low	1.0	Met	RequestSortTest	Sort by submitted, last changed, reference, status and category.
FR-006	Medium	1.0	Met	StatusTransitionTest, status-badge.test.tsx	
FR-007	Medium	1.0	Met	Every page load reads the database, StatusNotificationTest	The 4.1 criterion (a refresh shows the new status) is met. Email notification, added by CR-011, works but reaches one demo inbox (L-01), which is why 17.2 shows that scope item as Partially implemented.
FR-008	Low	0.5	Partially met	RetentionPruneTest (purge logic)	Purge is scheduled daily, but Render's free tier runs no scheduler, so it does not run on staging (TD-08).
FR-009	Medium	1.0	Met	paginate(50), manual check	
FR-010	High	1.0	Met	Role CHECK schema tests, AuthenticationTest	
FR-011	Medium	1.0	Met	RequestVisibilityTest (3 list + 6 detail cases), TE-03	
FR-012	Low	1.0	Met	List columns, TC-E2E runs	
FR-013	Low	1.0	Met	RequestListFilterTest, RequestSortTest, filter-bar.test.tsx	
FR-014	Medium	1.0	Met	RequestVisibilityTest (detail), history on requests/show	
FR-015	High	1.0	Met	TakeRequestTest, OfferTest (15 tests), TE-04	Decline rule from D-020.
FR-016	Low	0.5	Partially met	OfferTest, assignment schema checks	Every offer, acceptance and decline is recorded. Automatic purge after 5 years does not run on staging (same cause as FR-008).
FR-017	Low	1.0	Met	StatusTransitionTest, 6 schema checks	

ID	Priority	Value	Final status	Evidence	Note / reason
FR-018	Medium	1.0	Met	StatusTransitionTest, TakeRequestTest (403 cases)	
FR-019	Low	1.0	Met	StatusTransitionTest, 5 schema checks	
FR-020	Medium	1.0	Met	CommentTest, resolution note required when resolving	
FR-021	High	1.0	Met	StatusTransitionTest, "Active only" filter	Resolved requests stay in the system with history.
FR-022	Medium	1.0	Met	RequestVisibilityTest (management cases)	
FR-023	Low	1.0	Met	DashboardTest, TE-05	
FR-024	Low	1.0	Met	DashboardTest: "the dashboard shows the average resolution time" and "…empty when nothing is resolved"	
NFR-001	High	1.0	Met	BCRYPT_ROUNDS=12, PasswordUpdateTest	
NFR-002	High	1.0	Met	RequestVisibilityTest, 403 tests, TC-E2E-02	
NFR-003	Medium	1.0	Met	History rows with actor and timestamp, schema check	
NFR-004	Medium	1.0	Met	Section 14.8: max 193 ms per 50-row page on 5,000 rows	Measured locally (14.8). Server-to-database latency on staging (Render Frankfurt to Neon London) was not measured (L-09).S
NFR-005	Medium	0.0	Not met	None	Availability was not measured. Render's free tier sleeps after 15 minutes idle.
NFR-006	Low	0.0	Not met	Section 13.7.5	Neon free tier keeps 6 hours of history. The planned daily pg_dump and restore rehearsal were not done.

ID	Priority	Value	Final status	Evidence	Note / reason
NFR-007	Low	0.5	Partially met	Walkthrough was fully tested on 16/10/2026	Three-click rule checked by walkthrough. The 80% first-attempt user test was not run as we do not have a test group large enough to run the test.
NFR-008	Low	1.0	Met	Walkthrough was fully tested on 16/10/2026. StoreServiceRequest messages	
NFR-009	Medium	0.5	Partially met	Main Protection ruleset, PR history	Two approvals enforced on main, but most review happened in discussion.

Scope Completion % = (sum of final values / number of applicable criteria) x 100 = (29 / 33) x 100 = 87.9%

Result: 27 met, 4 partially met, 2 not met. 87.9% falls in the brief's 80 to 89% "expected substantial completion" range. All 7 High-priority (critical) criteria are met. Functional completion is 23 of 24 points (95.8%): 22 met, 2 partially met. The shortfall sits in non-functional criteria that depend on infrastructure the free tier does not provide (availability, backups, scheduled jobs), which is also why real production is a no-go (Section 17.4).

17.4 Final deployment state and release decision

Final deployment state: STAGING, with the Staged Application conditionally accepted and real production a NO-GO (D-021).

Release v4.0 runs on Render's free tier at https://civicconnect-staging-kski.onrender.com, connected to the Neon staging branch. Secrets are held in Render environment variables and GitHub secrets, never in the repository (14.4). The deployment is identified by commit 288849f. Post-deployment verification: On 18/10/2026 the team logged in as each role, submitted, took and resolved a request with a note, received the status email in the demo inbox and checked the dashboard figures. All steps passed.

The staged application is accepted because the core lifecycle is complete and verified. Real production is a no-go because releasing would cause the following:

Blocking concern	Affected requirement / risk	Consequence if released now	Must be resolved before go-live
Status emails reach one demo inbox (L-01)	FR-007, D-019	Real requesters would never be told their request changed	Verified sending domain, remove MAIL_DEMO_TO
Retention purge never runs (L-02)	FR-008, FR-016, POPIA s14, R-009	Personal data kept longer than the agreed 5 years	Scheduler (GitHub Actions cron or a host with one)
No backups beyond 6 hours. Restore not rehearsed (L-04)	NFR-006	Data loss could not be recovered within 48 hours	Daily pg_dump off-host, rehearse a restore

Blocking concern	Affected requirement / risk	Consequence if released now	Must be resolved before go-live
Free tier sleeps after 15 minutes (L-03)	NFR-005, R-013	First request after idle takes about a minute	Always-on host

17.5 Closure of M3 issues

Defects: all 10 M3 defects (DF-01 to DF-10, Section 16) remain fixed. The automated suites passed on the final commit (CI run #21, 16/10/2026: https://github.com/Destroyer1819/SEN381_Project/actions/runs/38919834328). No code changed after that commit, so no rerun was needed, and no new defects were found during finalisation.

M3 residual risk	Final status
PostgreSQL constraints and locks untested in CI	Carried (TD-03). The 53 schema checks still pass on Neon. PostgreSQL in CI is future work.
Free-tier cold starts and limits	Carried (L-03). Mitigated for the demo by warming the app up beforehand.
Emails reach one inbox, no retry	Carried (L-01, TD-05). A blocking concern for production (17.4).
Scale untested	Carried (TD-04, L-09). Local test of 5,000 rows meets NFR-004. Not repeated on Neon.
Offer rule awaiting team confirmation	Closed: D-020 confirmed through CR-032.

Section 14.10 stated “24 of 24 FRs implemented”. The corrected final figure is 22 met and 2 partially met (FR-008, FR-016), per Section 17.3.

Risks, technical debt and open decisions are concluded in their own registers: Section 7 (final review), Section 15 and Section 13.8.

17.6 Known limitations

ID	Limitation	Affects	Next step
L-01	Status emails reach one demo inbox (Resend without a verified domain)	FR-007	Verify a domain, remove MAIL_DEMO_TO
L-02	Retention purge does not run (no scheduler on Render free tier)	FR-008, FR-016	GitHub Actions cron or a host with a scheduler

ID	Limitation	Affects	Next step
L-03	App sleeps after 15 minutes idle. About a minute to wake	NFR-005	Paid or always-on host
L-04	No backups beyond Neon's 6-hour history, restore never rehearsed	NFR-006	Daily pg_dump stored off-host, rehearse a restore
L-05	Role changes are made through SQL and are not audited (D-022)	FR-010	Audited role-management screen for Management
L-06	Seeded demo accounts share one known password on staging	NFR-002	SEED_PASSWORD per environment. In production, Users will create their own passwords
L-07	No user test for the 80% first-attempt criterion	NFR-007	Run a small usability test
L-08	Accessibility not tested	13.5.3	Automated accessibility checks plus a manual pass
L-09	App (Render, Frankfurt) and database (Neon, London) are in different regions. That latency was not measured.	NFR-004	Measure on staging, move region when available

17.7 Project reflection and conclusion

Original problem and value: CivicConnect set out to replace email, phone and spreadsheets with one controlled record of each request's lifecycle (Section 1.3). The final product does this: every request has one owner, a full audit history and a status visible to its requester, and management can see totals, overdue work and resolution times.

Scope: 87.9% of the M1 acceptance criteria were met, with every High-priority criterion delivered. All changes were controlled. What remains incomplete is mostly infrastructure the free tier does not provide.

Quality: confidence comes from the CI suites (Pest, Vitest, type-check, lint, audits), the 53 PostgreSQL schema checks and three E2E journeys on staging. What remains uncertain is behaviour at scale, PostgreSQL row locks under real concurrency, and usability with real users.

Decisions with the clearest consequences:

- D-006 / CR-017 (Inertia monolith instead of a separate SPA): positive. One deployable and session authentication removed CORS and token handling, which made a single free Render service enough.
- D-015 and D-008 (seeded status transitions enforced in the service and the database): positive. Invalid transitions are impossible, and the schema checks caught problems the SQLite tests could not.
- D-018 (Render free tier): negative. It has no worker or scheduler, which caused synchronous mail, a purge that never runs and, through blocked SMTP, the one-inbox email limit. It is the main reason real production is a no-go.

Team and project execution: the biggest effects on delivery were D-014 (construction deferred to M3), M3 being postponed while the final deadline stayed fixed, the whole team learning PHP at once (R-012), and reduced team capacity during finalisation.

Future: if the project continued, the next work is the four production blockers in 17.4, followed by PostgreSQL and Playwright in CI, the missing indexes, and a small usability and accessibility test.

Conclusion: the project delivered a working, tested MVP that meets its critical requirements and addresses the problem defined in M1. It is not yet safe for real users. The team chose to state that clearly rather than release it.

References

Copes, F., 2026. *Every hosting provider's free tier, side by side.* [Online]
Available at: https://flaviocopes.com/hosting-free-tiers/
[Accessed 15 Oktober 2026].

Django Software Foundation, 2026. *Django at a glance.* [Online]
Available at: https://docs.djangoproject.com/en/stable/intro/overview/
[Accessed 26 September 2026].

Express, 2026. *Express - Node.js web application framework.* [Online]
Available at: https://expressjs.com
[Accessed 26 September 2026].

Inertia.js, 2026. *How it works.* [Online]
Available at: https://inertiajs.com/how-it-works
[Accessed 26 September 2026].

ISO, 2023. *ISO/IEC 25010:2023 Systems and software engineering - Systems and sofware Quality Requirements and Evaluation (SQuaRE) - Product Quality model,* International Organisation for Standardization: Geneva.

Jeff, S., 2011. *What Is A Good Task-Completion Rate?.* [Online]
Available at: https://measuringu.com/task-completion/
[Accessed 13 September 2026].

Laravel, 2026a. *Blade Templates.* [Online]
Available at: https://laravel.com/docs/blade
[Accessed 26 September 2026a].

Laravel, 2026b. *Laravel Sanctum.* [Online]
Available at: https://laravel.com/docs/sanctum
[Accessed 26 September 2026b].

Laravel, 2026c. *Starter Kits.* [Online]
Available at: https://laravel.com/docs/starter-kits
[Accessed 26 September 2026c].

Meta, 2026. *Pricing on the WhatsApp Business Platform.* [Online]
Available at: https://developers.facebook.com/documentation/business-messaging/whatsapp/pricing
[Accessed 13 September 2026].

Microsoft, 2026. *Overview of ASP.NET Core.* [Online]
Available at: https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core
[Accessed 26 September 2026].

NEON, 2026. *Neon pricing.* [Online]
Available at: https://neon.com/pricing
[Accessed 12 10 2026].

Next.js, 2026. *Documentation.* [Online]
Available at: https://nextjs.org/docs
[Accessed 26 September 2026].

Nielsen, J., 1993. *Response Times: The 3 Important Limits.* [Online]
Available at: https://www.nngroup.com/articles/response-times-3-important-limits/
[Accessed 13 September 2026].

POPIA, 2019. *Section 14 Retention and restriction of records.* [Online]
Available at: https://popia.co.za/section-14-retention-and-restriction-of-records/
[Accessed 27 September 2026].

PostgreSQL Global Development Group, 2026. *About PostgreSQL.* [Online]
Available at: https://www.postgresql.org/about/
[Accessed 26 September 2026].

Render, 2025. *changelog.* [Online]
Available at: https://render.com/changelog/free-web-services-will-no-longer-allow-outbound-traffic-to-smtp-ports
[Accessed 18 Oktober 2026].

Ruby on Rails, 2026. *Getting Started with Rails.* [Online]
Available at: https://guides.rubyonrails.org/getting_started.html
[Accessed 26 September 2026].

Spring, 2026. *Spring Boot.* [Online]
Available at: https://spring.io/projects/spring-boot
[Accessed 26 September 2026].

Statista, 2026. *Most used web frameworks among developers worldwide as of 2025.* [Online]
Available at: https://www.statista.com/statistics/1124699/worldwide-developer-survey-most-used-frameworks-web/
[Accessed 26 September 2026].

supabase, 2026. *https://supabase.com/pricing.* [Online]
Available at: https://supabase.com/pricing
[Accessed 13 September 2026].

Tovarys, J., 2022. *Availability Table.* [Online]
Available at: https://betterstack.com/community/guides/incident-management/availability-table/
[Accessed 13 September 2026].

Translated, 2025. *Software Localization Cost: Application Translation Pricing & Budget Guide.* [Online]
Available at: https://translated.com/resources/software-localization-cost-application-translation-pricing-budget-guide
[Accessed 13 September 2026].

VerifyNow, 2025. *https://www.verifynow.co.za/how-to-comply-with-rica-in-south-africa-your-comprehensive-guide.* [Online]
Available at: https://www.verifynow.co.za/how-to-comply-with-rica-in-south-africa-your-comprehensive-guide
[Accessed 13 September 2026].